INDEX

S. No	Topic	Page No.
	Week 1	
1	Biological Neuron	1
2	From Spring to Winter of AI	8
3	The Deep Revival	18
4	From Cats to Convolutional Neural Networks	25
5	Faster, higher, stronger	29
6	The Curious Case of Sequences	31
7	Beating humans at their own games (literally)	38
8	The Madness (2013-)	41
9	(Need for) Sanity	50
10	Motivation from Biological Neurons	54
11	McCulloch Pitts Neuron, Thresholding Logic	62
12	Perceptrons	73
13	Error and Error Surfaces	82
14	Perceptron Learning Algorithm	85
15	Proof of Convergence of Perceptron Learning Algorithm	98
	Week 2	
16	Deep Learning(CS7015): Linearly Separable Boolean Functions	108
17	Deep Learning(CS7015): Representation Power of a Network of Perceptrons	113
18	Deep Learning(CS7015): Sigmoid Neuron	124
19	Deep Learning(CS7015): A typical Supervised Machine Learning Setup	136
20	Deep Learning(CS7015): Learning Parameters: (Infeasible) guess work	146
21	Deep Learning(CS7015): Learning Parameters: Gradient Descent	157
22	Deep Learning(CS7015): Representation Power of Multilayer Network of Sigmoid Neurons	179
	Week 3	
23	Feedforward Neural Networks (a.k.a multilayered network of neurons)	209
24	Learning Paramters of Feedforward Neural Networks (Intuition)	222
25	Output functions and Loss functions	229
26	Backpropagation (Intuition)	244
27	Backpropagation: Computing Gradients w.r.t. the Output Units	255
28	Backpropagation: Computing Gradients w.r.t. Hidden Units	267
29	Backpropagation: Computing Gradients w.r.t. Parameters	279
30	Backpropagation: Pseudo code	287

31	Derivative of the activation function	292
32	Information content, Entropy & cross entropy	294

Week 4

33	Recap: Learning Parameters: Guess Work, Gradient Descent	312
34	Contours Maps	324
35	Momentum based Gradient Descent	333
36	Nesterov Accelerated Gradient Descent	346
37	Stochastic And Mini-Batch Gradient Descent	355
38	Tips for Adjusting Learning Rate and Momentum	368
39	Line Search	377
40	Gradient Descent with Adaptive Learning Rate	383
41	Bias Correction in Adam	404

Week 5

42	Eigenvalues and Eigenvectors	412
43	Linear Algebra : Basic Definitions	424
44	Eigenvalue Decompositon	434
45	Principal Component Analysis and its Interpretations	442
46	PCA : Interpretation 2	465
47	PCA : Interpretation 3	477
48	PCA : Interpretation 3 (Contd.)	482
49	PCA : Practical Example	484
50	Singular Value Decomposition	491

Week 6

51	Introduction to Autoncoders	510
52	Link between PCA and Autoencoders	543
53	Regularization in autoencoders (Motivation)	556
54	Denoising Autoencoders	564
55	Sparse Autoencoders	582
56	Contractive Autoencoders	590

Week 7

57	Bias and Variance	599
58	Train error vs Test error	608
59	Train error vs Test error (Recap)	624
60	True error and Model complexity	631
61	L2 regularization	639
62	Dataset augmentation	656

63	Parameter sharing and tying	660
64	Adding Noise to the inputs	662
65	Adding Noise to the outputs	667
66	Early stopping	671
67	Ensemble Methods	679
68	Dropout	686

Week 8

69	A quick recap of training deep neural networks	700
70	Unsupervised pre-training	706
71	Better activation functions	726
72	Better initialization strategies	752
73	Batch Normalization	774

Week 9

74	One-hot representations of words	786
75	Distributed Representations of words	793
76	SVD for learning word representations	802
77	SVD for learning word representations (Contd.)	803
78	Continuous bag of words model	814
79	Skip-gram model	839
80	Skip-gram model (Contd.)	846
81	Contrastive estimation	856
82	Hierarchical softmax	861
83	GloVe representations	871
84	Evaluating word representations	877
85	Relation between SVD and Word2Vec	885

Week 10

86	The convolution operation	888
87	Relation between input size, output size and filter size	907
88	Convolutional Neural Networks	919
89	Convolutional Neural Networks (Contd.)	935
90	CNNs (success stories on ImageNet)	951
91	CNNs (success stories on ImageNet) (Contd.)	972
92	Image Classification continued (GoogLeNet and ResNet)	974
93	Visualizing patches which maximally activate a neuron	989
94	Visualizing filters of a CNN	993
95	Occlusion experiments	999

96	Finding influence of input pixels using backpropagation	1003
97	Guided Backpropagation	1008
98	Optimization over images	1012
99	Create images from embeddings	1022
100	Deep Dream	1032
101	Deep Art	1039
102	Fooling Deep Convolutional Neural Networks	1044

Week 11

103	Sequence Learning Problems	1050
104	Recurrent Neural Networks	1057
105	Backpropagation through time	1067
106	The problem of Exploding and Vanishing Gradients	1080
107	Some Gory Details	1089
108	Selective Read, Selective Write, Selective Forget - The Whiteboard Analogy	1095
109	Long Short Term Memory(LSTM) and Gated Recurrent Units(GRUs)	1103
110	How LSTMs avoid the problem of vanishing gradients	1126
111	How LSTMs avoid the problem of vanishing gradients (Contd.)	1133

Week 12

112	Introduction to Encoder Decoder Models	1152
113	Applications of Encoder Decoder models	1169
114	Attention Mechanism	1185
115	Attention Mechanism (Contd.)	1205
116	Attention over images	1207
117	Hierarchical Attention	1216

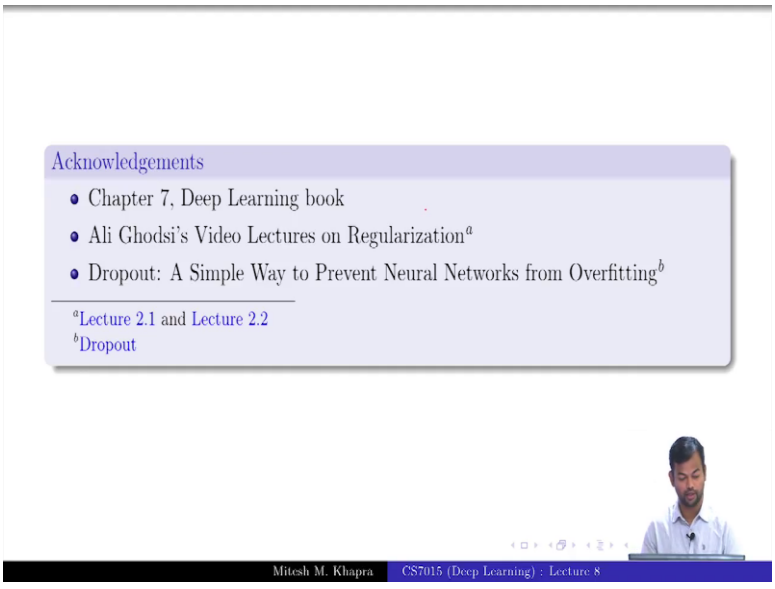
Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Lecture - 08
Regularization: Bias Variance Tradeoff, L2 regularization, Early stopping, Dataset augmentation, Parameter sharing and tying, Injecting noise at input, Ensemble methods, Dropout

So, in this lecture we are going to talk about a bunch of regularization techniques for Deep Neural Networks. You might find some very familiar terms here. For example, L2 regularization, perhaps something else also, but I promise you that we will see a very different interpretation of this from what you have done in your earlier courses, right.

So, again as is the trend in this course, I will start with some basic concepts, I will take today's lecture to finish off the basic part which is the bias variance tradeoff and I will try to make it more informative. Then, what you have done in your earlier courses and in the rest of the lecture which will happen on Friday, we will build upon these basics and then, try to look at these as the regularization forms.

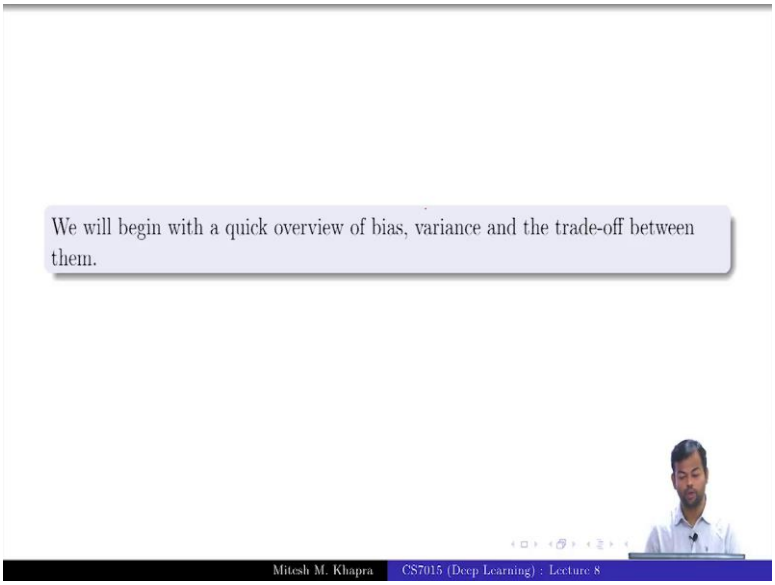
(Refer Slide Time: 00:59)



So, let us start. So, these are the sources which I have looked at. So, one of them is the chapter 7 from deep learning book. Other is this very good lecture by Ali Ghodsis on Regularization and of course, this paper on dropout. So, let us start with Bias and

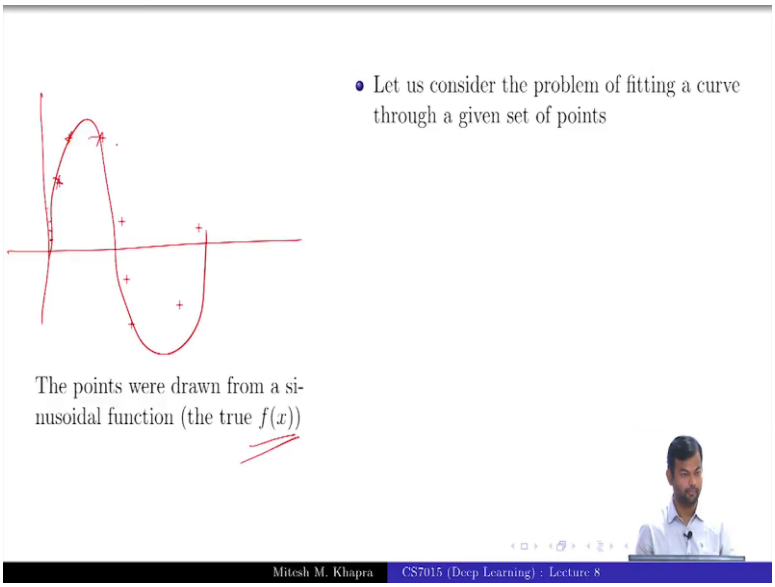
Variance. Again some 5-10 minutes would be similar to what you have seen in the middle class, but then I will go on to something different.

(Refer Slide Time: 01:22)



So, we will begin with a quick overview of Bias Variance and the tradeoff between them.

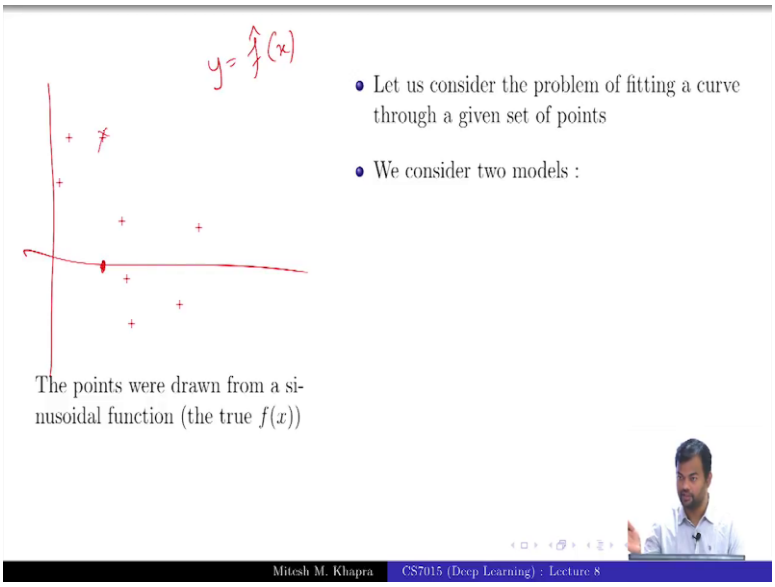
(Refer Slide Time: 01:26)



So, let us consider the problem of fitting a curve through a given set of points, ok. Now, remember I have always been telling you that there is always this true relation between x and y which is f of x, right and which we never know.

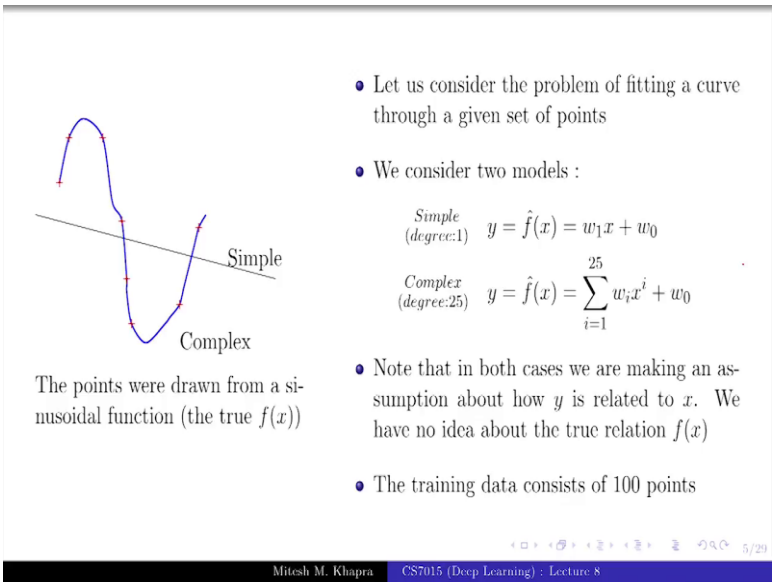
So, we do not know what this is. In the movie example, we do not know what this is in the credit card fraud detection or in the oil mining, example. In this particular example I know it right? So, what I have done is, I know that the true relation between x and y is the sinusoidal curve. I know this, but instead of giving you every point on this sinusoidal curve what I have done is, I have such sampled some points from it. I have taken some points and given to you.

(Refer Slide Time: 02:11)



So, from now on we will behave as if we do not know that this is how it came. It is a big secret, and we now want to fit a curve to this. That means, I want to learn the function f hat of x ok? Which of course will have some parameters and what will be my goal is that now let us look at this. Again my goal would be if I feed at this point after the model is train, the output should be as close to this point as possible. That is our training criteria. Everyone gets this?

(Refer Slide Time: 02:48)



So, we consider two models. The first model is a simple model. How many parameters does it have?

Student: 2

Two parameters, right? The other model and this is what happens when I train the simple model. Of course, I will get a line, but do you see something special about this line. Why did I get this as a line or this as a line? So, on average it is trying to minimize the distance from all the points. If I have this as the line, then I will have a very high error for these points, right? So, just something which goes along the average, and hence the sum of the squared errors would be minimized right?

So, it is important that when you see these figures, you should make these connections to the math behind it. So, this is the geometry, you have to make connections to the math behind it, right and I hope all of you make that connection. Now, I take a complex model which is a degree 25 polynomial ok? So, this is w 1 x w 2 x square w 3 x cube and so on. It is a degree 25 polynomial that I have used and I again learn the parameters of this using how will you learn the parameters. You have a quiz 2 days from now on Gradient Descent.

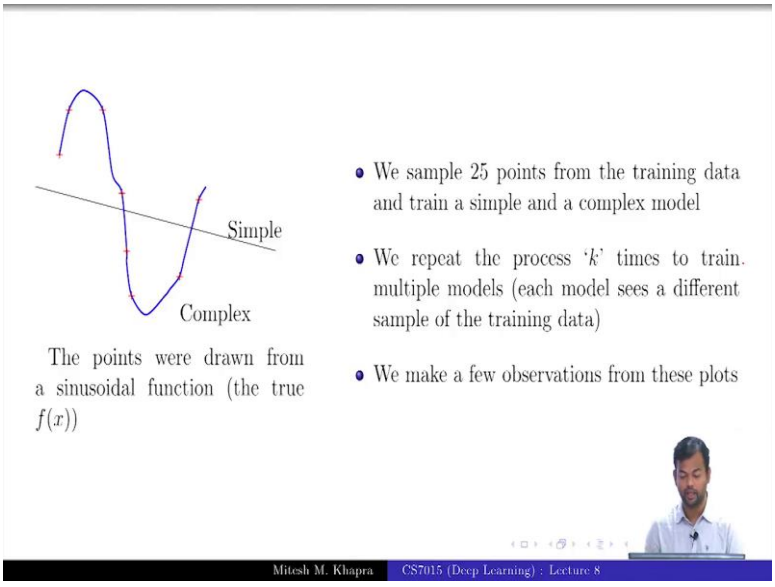
What else do you know? If you know any other algorithm, of course you know, but getting this end right what else will you use? You can use gradient descent for learning.

These parameters, the same idea, right? You will define a loss, you will compute the gradients with respect to all these parameters. How many of them are there? Here 26 and just update those parameters till a fixed number of iterations or any convergence criteria, ok and this is the curve which I get for the complex model. Note this in both these cases, we are making an assumption about how y is related to x, right? In this case, I made a simple assumption. In this case, I made a slightly complex assumption, but in both the cases, we do not know what is true relation is?

The true relation is actually the sine curve, but we do not know that we are just making an assumption. So, you remember the five things in machine learning, you have a data, you make an assumption about how the input is related to the output. So, these are my two assumptions. Then, I have some parameters. You know the number of parameters in these cases? I use a learning algorithm which happens to be gradient descent and then, I minimize an objective function which would be squared error loss in this case, fine.

Now, the training data actually consists of 100 points ok? But you do not see 100 points here.

(Refer Slide Time: 04:58)

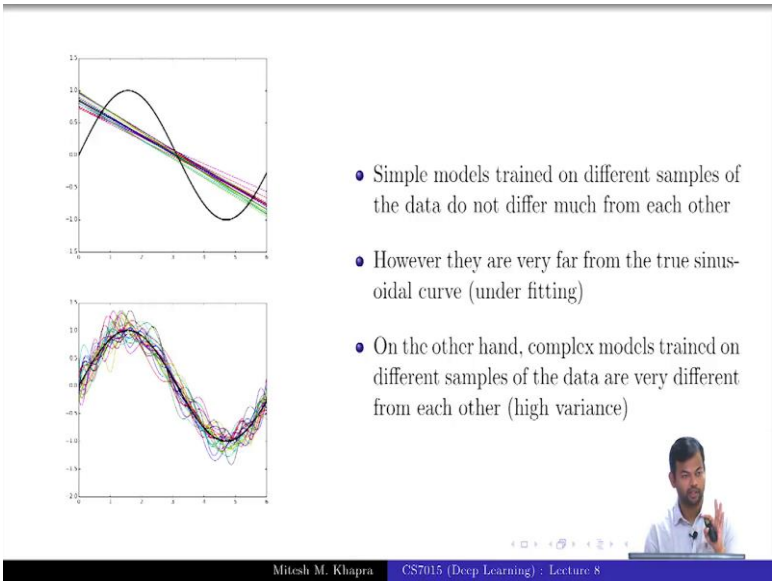


So, what I have done is, I have sampled some 25 points from here and use that as the training data. So, I have learned my parameters w 1 and w naught or w 25 up to w naught. Using these 25 points, now I will repeat this experiment k times. What I do is every time I will get a different sample of 25 points and I will try to learn the parameters

of the model. Will I get the same curve every time? Will I get the same function every time? No, my parameters would change slightly, right because my training data is different.

So, I am trying to learn it differently to adjust to that training data. So, my function is going to be different. It is the same form. It is either the linear function or the polynomial function, but the parameters, the coefficients are going to be different.

(Refer Slide Time: 05:43)



I will actually draw these different functions and we will make some observations from that. So, this is the black curve that you see is the true sinusoidal curve from which the data has come. The blue line is one of these functions which I have trained from one random sample of the data, right?

Now, I train different functions from different random samples of the data and see what happens. I get different lines. This obvious can you relate to this every time. I am basically learning a different value of w 1 and w naught. Is that ok? And I have done this 25 times and plotted these lines. What do you observe with respect to each of these if you compare any line to any other line?

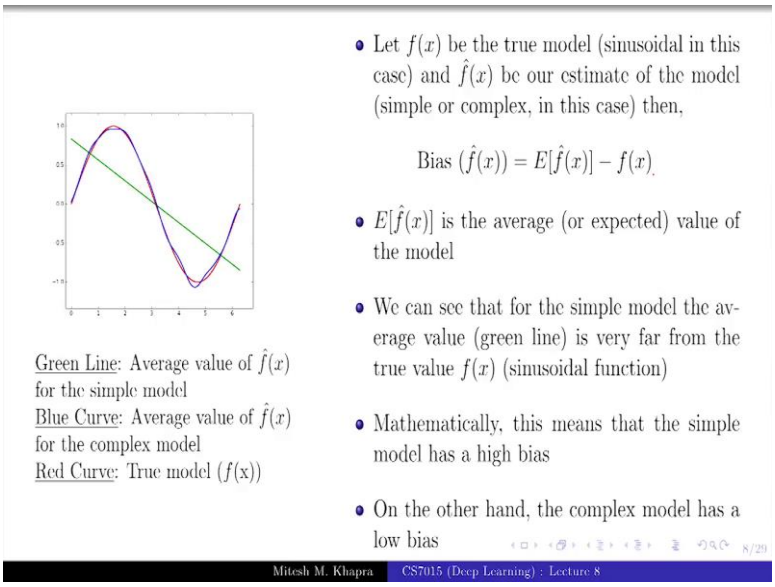
So, if you compare one of these lines to the remaining 24 lines, what do you observe? They are very close to each other. They are not very different from each other, however there is a problem. They are very far from dash the actual function; that means we are

under fitting. We have very few parameters in fact only 2. That is why we are under fitting. Let us look at the other case, fine. This is the function, the polynomial, the blue curve that you see is the polynomial that I learned from one random sample of the data.

Now, I am going to learn this from a different sample of the data. You see what happens? You see that the green curve is actually very different from the blue curve. You see that here actually this was peaking whereas, this is going down. Similarly this was peaking, but this is going down and so on. So, you see that there are clear differences between the two curves and if I draw the next curve, you see it is even more different. The same function learnt from different data point is turning out to be very different. Why, because it is over fitting on those 25 points that I have given. The simple model did not even have the capacity to do or fit because it is just two parameters.

How much can I over fit? I will just end up drawing the average line right, but here it is really able to overt fit and you see that these 25 curves or I do not know how many curves that I will draw, all of these are going to be very different from each other. You see that, and everyone agrees that this would happen if you actually try to do this ok? So, complex models train on different samples of the data are very different from each other. What is happening there is over fitting ok?

(Refer Slide Time: 07:54)

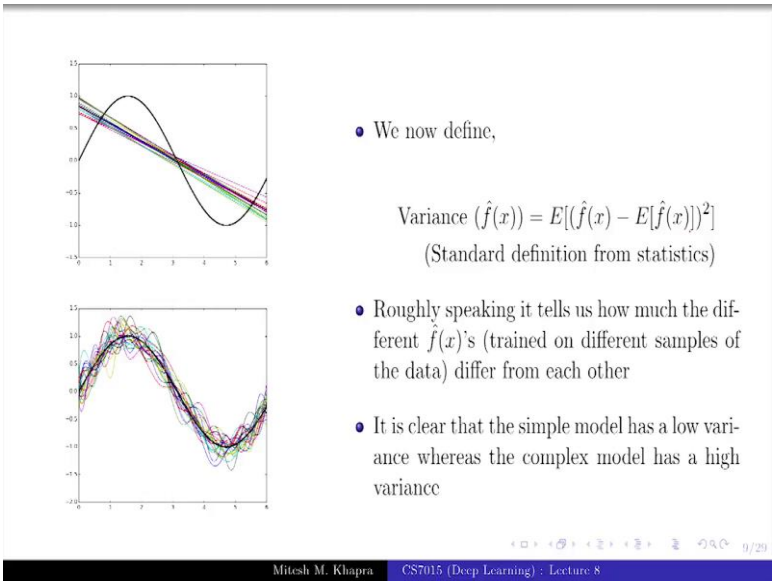


Now, let me define two concepts. From statistics 1 is bias. Bias is very simple. It tells us that this is the true function. If you are trying to learn the approximate function and you

do it many times, then you will get an expected value of the function. So, it tells you how much does this expected value differ from the true function, ok. You get the definition. The definition is straight forward, ok. Now, for the simple line or the simple model, the green line that you see is actually the average of all those 25 lines that you had seen, ok. What can you say about the bias, very high right because this difference is very high.

This green line is very different from the red curve which is my true function, right predicted and true function. Now, what about complex model? The blue curve that you see is actually the average of all those 25 different curves that I had drawn. So, what is the bias? It is very low. Does that make sense? This means that the simple model has a high bias and the complex model has a low bias. Is it clear to everyone?

(Refer Slide Time: 09:06)



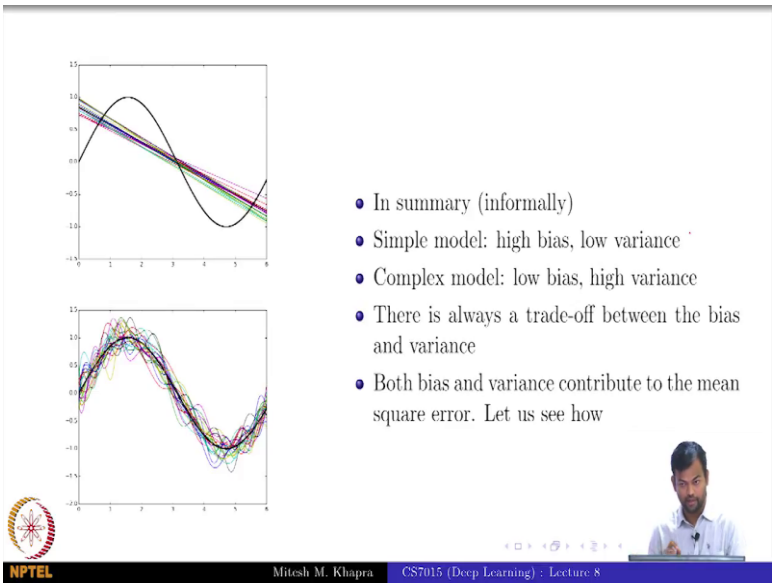
Now, let us define another quantity which is variance. Everyone knows what variance is. So, this is one of the functions that I have learned. This is the average of that function and the variance tells me the spread. Now, based on the figures that you have seen, can you tell me what would happen for the simple model; low variance or high variance?

Student: Low variance.

Low variance, because all these models were very close to each other; there was not much spread in the models. What about the complex model?

High variance: all these models were very far from each other. The spread was very high, ok. So, roughly speaking it tells us how much the different f f act f x is that you are learning, how different are they from the average f of x.

(Refer Slide Time: 09:50)



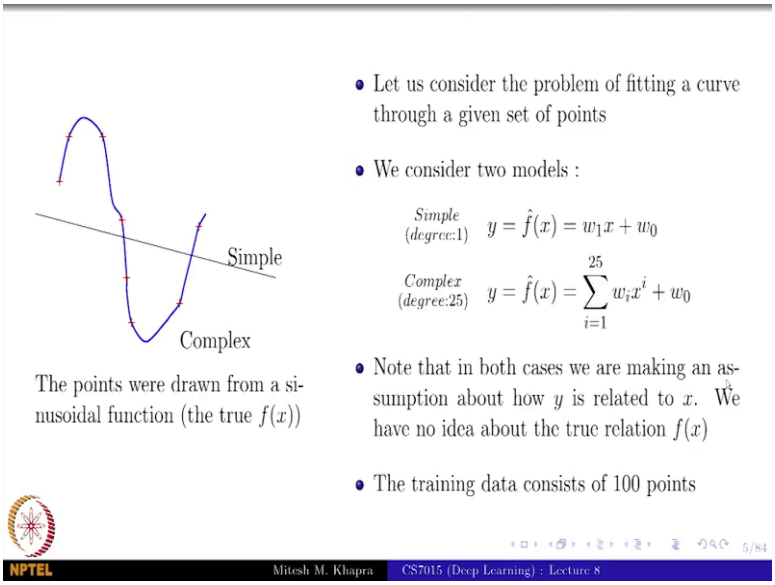
So, informally I can say the following simple model has a high bias low variance, complex model has a low bias high variance and as always going to be a tradeoff between the bias and variance.

So, why is there always a tradeoff between the bias and variance? People have done ML course. Why is there a tradeoff? How many of you know the mathematical answer to that? You have not done this in the ML course, no. So, it turns out that both bias and variance contribute to the mean square error and let us see how.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

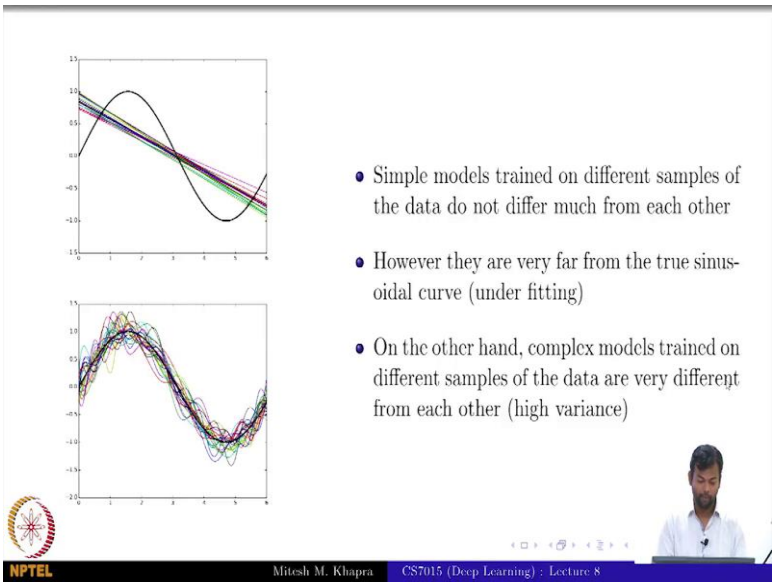
Lecture - 08
Train error vs Test error (Recap)

(Refer Slide Time: 00:12)



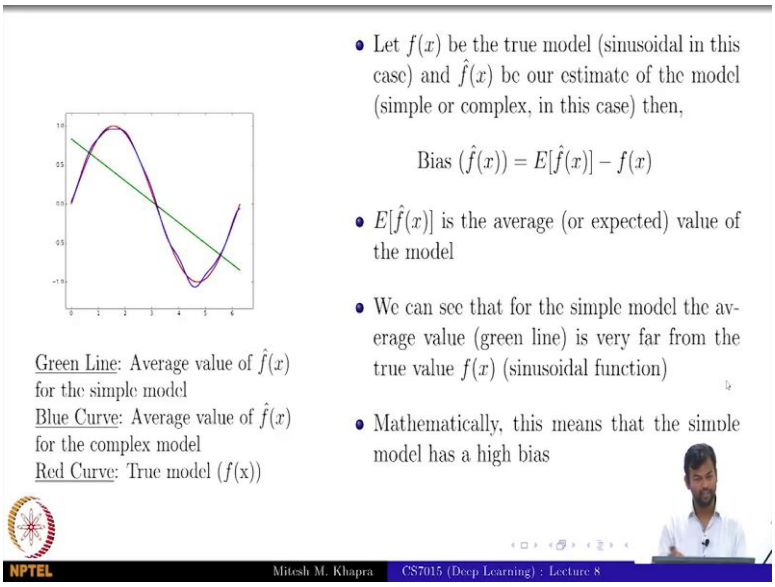
So, we spoke about bias and variance and we saw that simple models have a high bias, but low variance and complex models have a low bias high variance and so on. And we saw it some illustrative examples that what that is what that means.

(Refer Slide Time: 00:18)



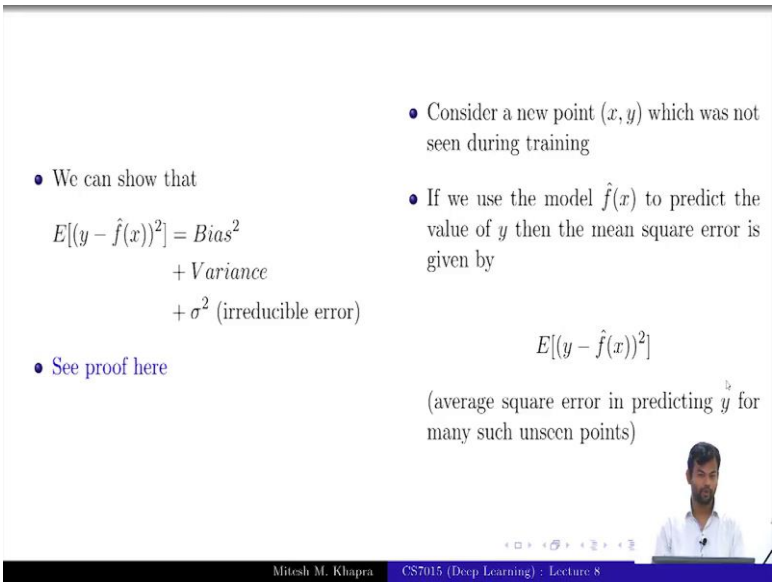
And, then, the important thing to note was these two formal definitions of bias.

(Refer Slide Time: 00:26)



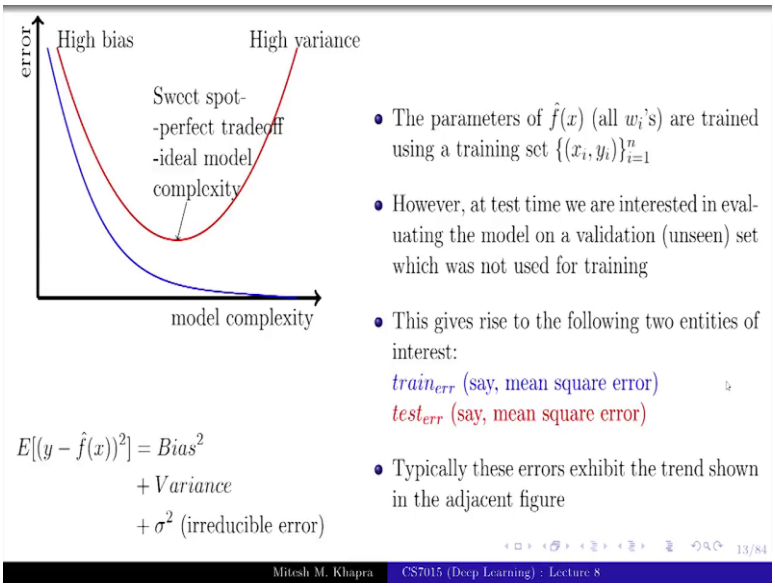
And formal definition of variance which you all know anyways, and then the important concept that we spoke about was the train error versus test error, right?

(Refer Slide Time: 00:36)



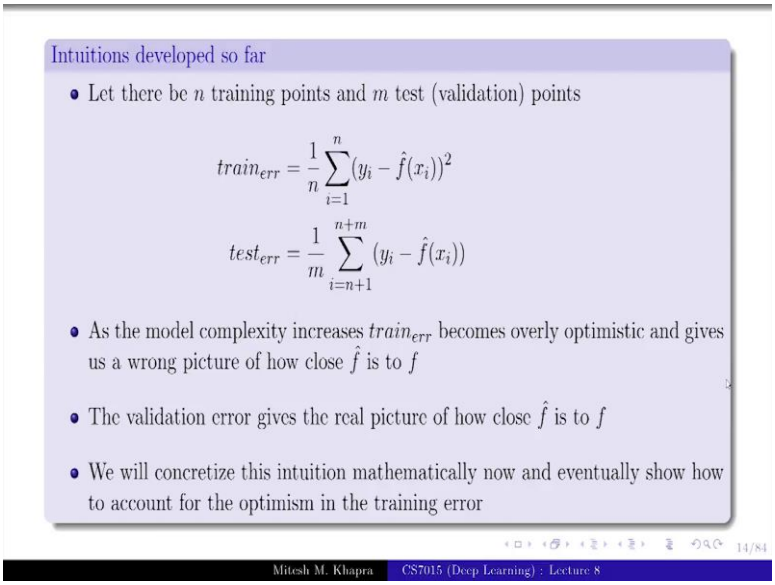
So, this was the curve that we were interested in and one corner of this curve was related to high bias low variance. And the other corner was related to low bias high variance, right?

(Refer Slide Time: 00:39)



So, I am looking for something in the middle. That is what our quest is in this lecture, right and we want to find ways of falling somewhere in middle.

(Refer Slide Time: 01:06)

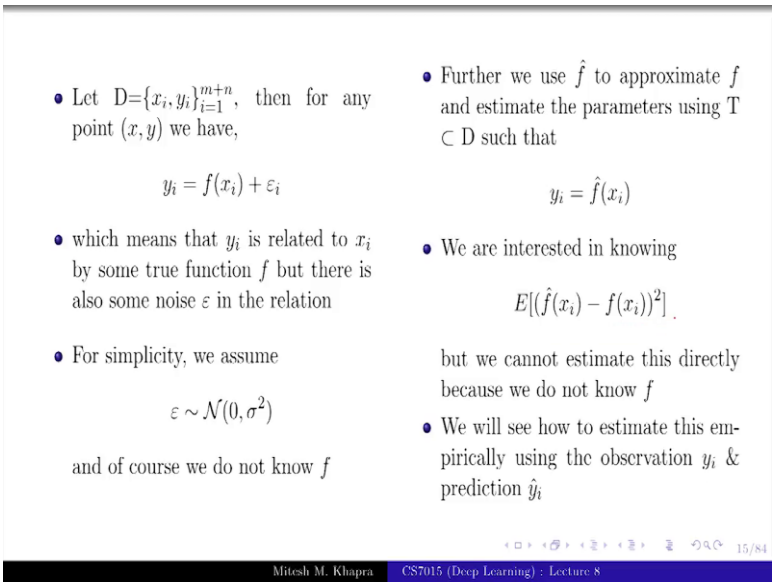


And This led to the definition of two quantities of interest or training error and test errors. So, training error is computed from the training points. These are the points that you actually look at while you are solving this optimization problem. So, the training always involves solving an optimization problem which is the objective that you want to optimize or maximize and the test error is something that you want to use it for at eventually.

So, you all have these two quantities of interest that we design and we realize that the training error is more optimistic whether the test errors actually gives us the real picture of what we do and we tied those back to things that you have done previously in the machine learning; or other courses that we always split the data into training, valid and test. Training it on the training data, do some validations on the validation data, but never look at the test data. That is for the final evaluation.

So, that's the this is this intuition which I have been trying to build with these two curves is the explanation for why we do things that way.

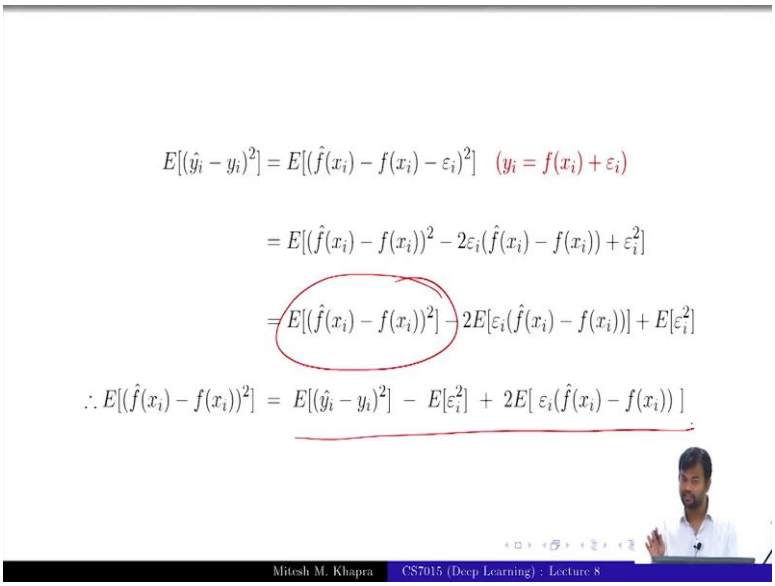
(Refer Slide Time: 02:03)



Now, we are interested in doing a more mathematically rigorous analysis of this intuition, right? So, that is where we left off. So, what we are interested in? So, now I will just start from this point is that we are given some data which is m, n. m training points and n testing points and we know that there is a true function between the outputs and the inputs, and we are also expecting or accepting some noise in this relation just as in any other relation so which means that y is related to xi, but by some true function. But there is also this noise and for simplicity we assumed as this noise comes from a normal distribution with zero mean and some small variance and as usual we never know f. right But we are trying to approximate this f hat, and we come up with some parametric form for f hat and then, try to learn the parameters of f hat from the training subset of the data that is given to us.

So, this is what we always do and we have already seen different variations of f hat. One of them being the deep neural network and what we are actually interested in is this quantity, the expected difference or square difference between the predictions made by our model and the true value of the output with respect to the true function right? Then, we asked I asked you whether we can actually estimate this quantity and all of you said no. Why? It is because you do not know what f of xi is, right? So, we will see how to estimate this empirically.

(Refer Slide Time: 03:44)



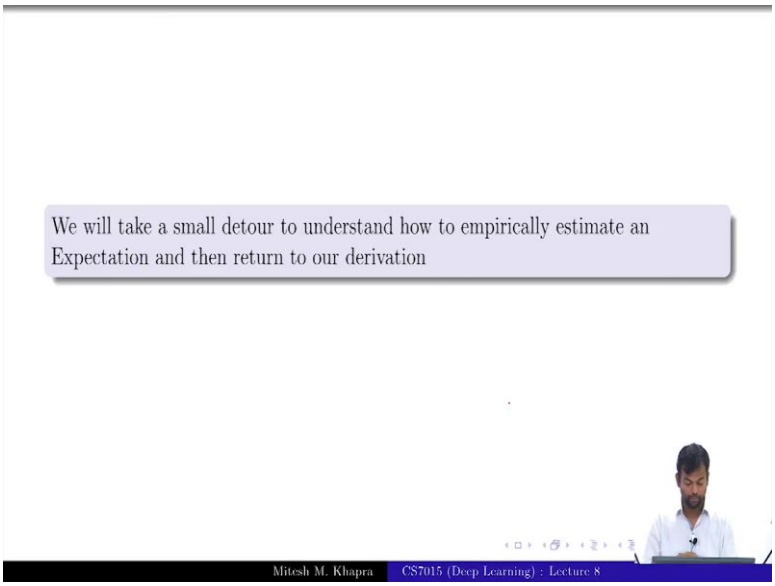
So, then we started off with this information that we have we know what y i hat is because that is the prediction that we make and we know yi. What yi is, we do not know the function, but we see the output of the function in the form of the training data points given to us or any data points given to us.

So, we wrote this by making this particular substitution where we notice that yi that we see is actually the true function plus some noise and then, we did some trickery and try to simplify this. And then, we just realize that this is the term that we are interested in. So, we moved it to the other side of the equation and came up with this neat left hand side or neat right hand side that we need to analyze now. So far everything is clear.

This is where we ended the last class, right? You just went to it very quickly, but I assume everything is clear at this point, ok fine. So, we are left with a bunch of expectations, right and we have I am assuming we have no clue how to estimate this. Right I mean. So and Remember that when you are dealing with expectations as always this true expectation and then, there is this empirical estimation right? So, what we are going to move towards? So, these all equations when I write E here, capital E here, I am talking about the true expectation.

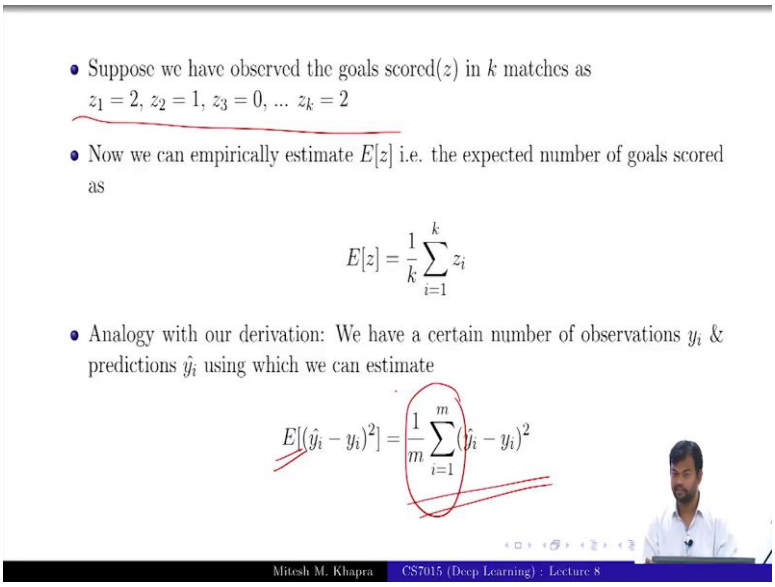
Now, we will see how to approximate the true expectation with an empirical expectation and then, based on that we will make some observations.

(Refer Slide Time: 05:12)



So, that is what we will do now.

(Refer Slide Time: 05:14)



So we will just take a small d 2 and I will just tell you what expectations are or what empirically expectation is, how to compute them. So, suppose we have observed the goals scored in k matches. There is some k football matches that we have seen and we have seen that the goals scored were the following.

Now, if I asked q what is the expected value of the goal? Now, the number of goals for what will you do; take the average of this. This is what you will do. So, what is it that

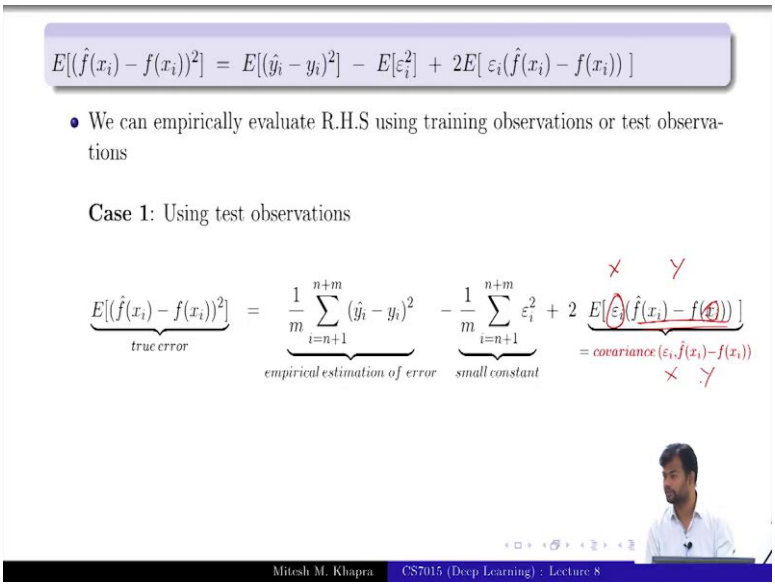
you are doing here? You are taking a dash estimate of the expectation, empirical estimate. You are making some observations. These are the observations given to you, these are the k matches. Watch as much, as many football matches as you want after the semester ends and then, notice the number of goals that were scored in them and then, you can compute this expectation, right and this is how you do empirically. So, there is something that we do on a regular basis, but I just want you to realize that what you are doing is actually an implicit estimate of the true expectation.

Now, can you relate this to the quantity that we are interested in? We are interested in computing a certain expectation which is this. Can you take an analogy and tell me how you would do this? The hint is we have done this a million times in the course already, fine. So, this is how we will do it and have actually done this a million times in the course. So, when you compute this, we are actually doing an empirical estimate of the data.

So, let us just take a minute to understand this. We are given some data, we are interested in this to expectation which we cannot compute. So, we will take this data, we will assume there is enough of this. We are given m samples which are enough and from that we will make an empirical estimate and just as in the case of these goal scored, right? As you see more and more matches, you will have a better understanding of how many goals can be scored when two particular teams are playing. In the same analogy goes here, as you see more and more data, your estimate would become better, but that is how you will do the estimation.

So, now we will come back to, so now do not get surprised when I am going to replace all these e's by this, all the e's that we had in our original equation. I am going to replace them by these summations, ok fine.

(Refer Slide Time: 07:22)



So, this was our original equation that we had derived and we were interested in this left hand side quantity which is a sum of some terms on the right hand side. So, now this expectation I told you that we can estimate it from data, but which data? Training data or test data, both. So, we will try to estimate it from both and see if there is any difference which arises when you estimate it from one data and the other data ok?

So, the first thing that I am going to do is, I am going to use test observations to estimate this. So, can you tell me what are my summations going to look like? It is summation over n plus 1 to n plus m right? We assume that the first endpoints are training points and the remaining points are test points.

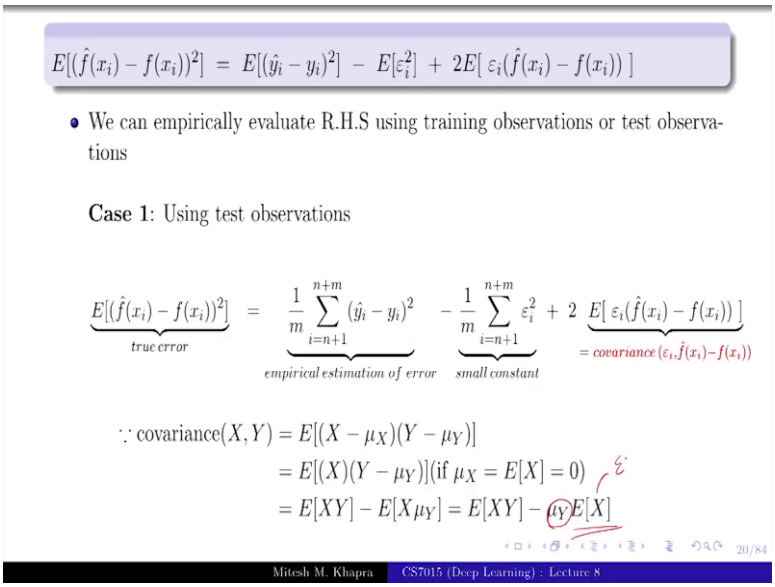
So, the quantity on the left hand side is true error. Remember that because that has f x which we do not know quantity on the right side, the first thing is empirical estimation of the error, ok. The second thing is a small constant. However, the epsilon i square and we assume that comes from a normal distribution with a small variance. What is the third quantity? Actually I have given you the answer already, but I want you to think about it. I am saying it is the co-variance between two things.

When I say it is the co variance between two things, what is the first thing that I need to prove is that the two things are dash random variables. I mean first thing we need to see is that the two things are random variables epsilon i clear. It is a random variable.

What about this other thing or rather epsilon is a random variable what about the other thing and depending on the training instance that you have sampled, this ongoing difference is going to differ, right? You are having your training or test instance whatever is this x i this is going to differ because these x's are different. They are all random variables. So, there is difference between these two quantities also going to be a random variable. Is that fine ok? But still is this the.

So, then I have told you this is x and this is y and what I am saying is that the co variance between x and y is just e of x x into y. Is that correct?

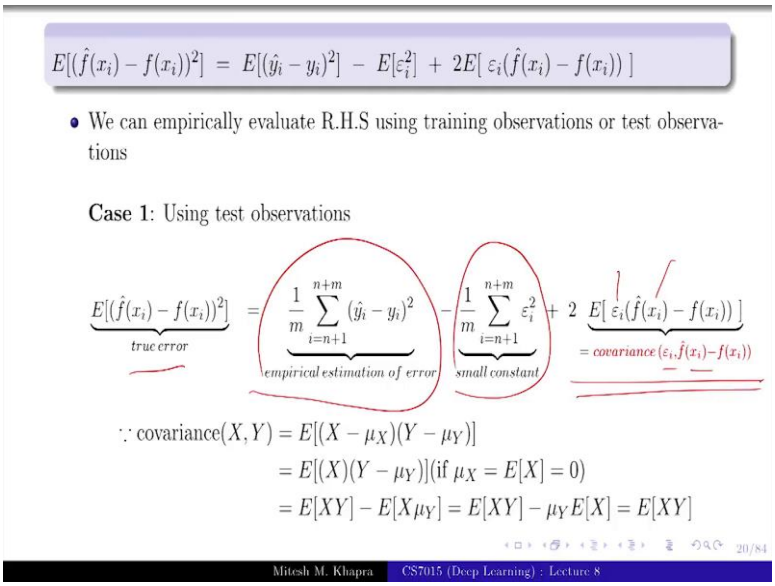
(Refer Slide Time: 09:34)



That is how you define co variance. What is the definition of co variance; if you have bothered to look at the prerequisites, no expectation in the form of e. So, co variance is e of x minus mu of x into y minus mu of phi, what is our x epsilon and what is our y. What is mu of x 0?

So, I will just simplify this a bit ok. I will open up the product. What is mu of y into e of x? What is e of x? What is the expected value of the noise 0?

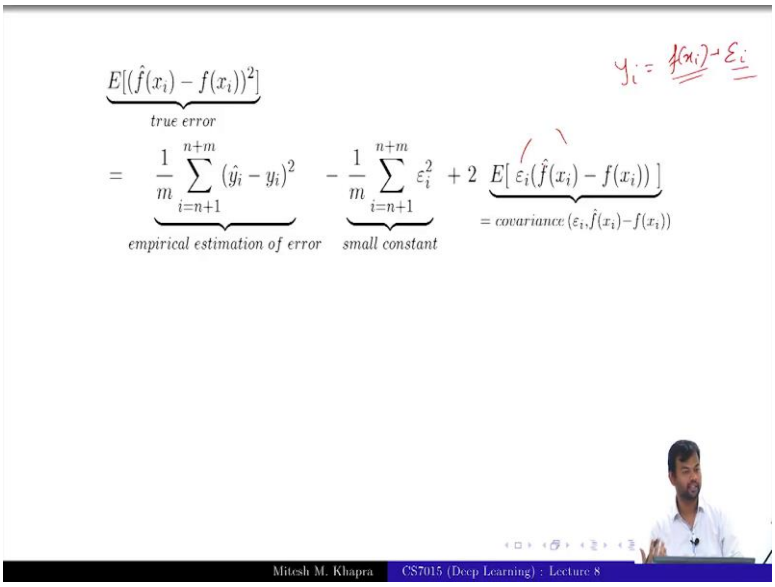
(Refer Slide Time: 10:01)



So, then this turns out to be as that. Is that fine? That is why we are writing the co variance is just the product of the two things.

So, let us just take a minute to again understand this. The true error is the empirical estimation of the error plus I mean plus or minus a small constant ok. And then, this nasty quantity that we do not know what to do with it. So, let us look at this quantity and see what we can say about it.

(Refer Slide Time: 10:30)



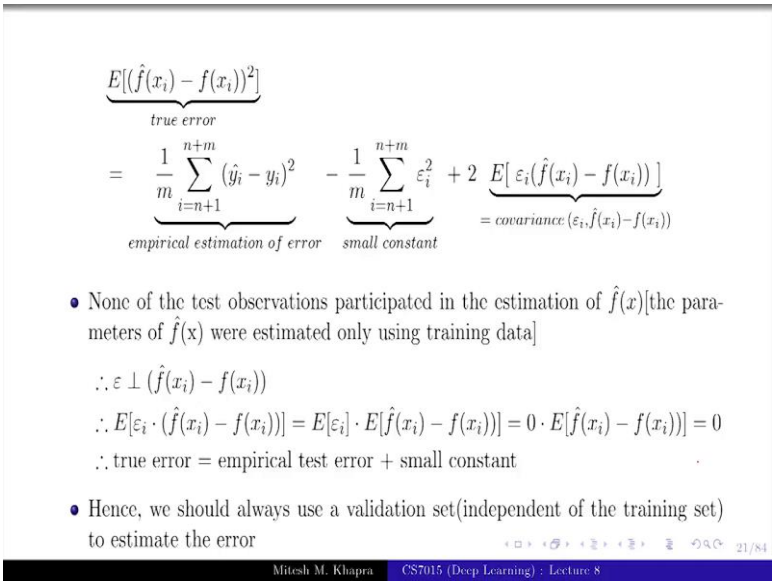
Now, what is the co variance between these two? I am trying to compute this expectation from the test data. Just remember that. So, each i here is a test instance are these two random variables dependent or independent is the question that I am trying to ask. It is independent. So, let us look at it piece wise. So, remember that we had said that y is equal to f of x i plus epsilon i, right? This epsilon I had no relation to f of x i. I mean I could choose any x i, but this noise is going to be random. So, there is no relation between these two.

Now, is there a relation between f hat of x i and epsilon i? We are doing tests. So, how did we come up with f hat of x i? How did when I say how did we come up with f hat is i mean? How did we learn the parameters of a f hat using the training data and what are we computing expectation with respect to now test data. These these epsilon improve, influence the parameters that we had learned further from the training data? No, since there is no dependence between these two guys.

So, that is why epsilon i is independent of the other random variable that you see in this expectation. Is that clear? Do you get the intuition f hat x i further? No, but this is the mean. This noise is what is present in the test data and you have not seen this add training time. When you are training the parameters, you did not look at this noise. You are looking at the noise in the training data.

So, this is not participated in the estimation of the parameters of f hat, but that was for the training data right, but this now I am doing the expectation from a test data.

(Refer Slide Time: 12:10)



So, these two random variables are independent. That means, I can write this as is this fine. What will happen to this 0?ok So, what did we eventually conclude that the true error is equal to empirical test error plus a small constant, right?

So, what does this tell you? Now, tell me forget the math. Tell me in English, right? What does this take? What does this mean? Can you relate it to; now why you do this training error, validation error, test error? So, what does this tell me? This tells me that if I have trained a model and now if I take an estimate of the error on some data which I had not used for the training, then that error which I see is actually very close to the true error. It only differs by this small constant.

How many of you get that? That is why when I look at the validation error, it is not being overly optimistic. It is giving me a true picture of what the actual error is, right? So, there are two things that you need to understand here. One, this is the quantity that we are interested in which we cannot estimate. We are trying to estimate it by using this; we are trying to make an approximation. So, we are trying to see how good this approximation is. What this derivation is telling us is that if you are approximated it using the test error or the test data, then this approximation is actually very close to the true error and how close it is actually? It just differs by this small constant.

So, you get the importance of what we are seeing here right? Ok. Now, to truly appreciate this I need to tell you what would have happened if you had used the training

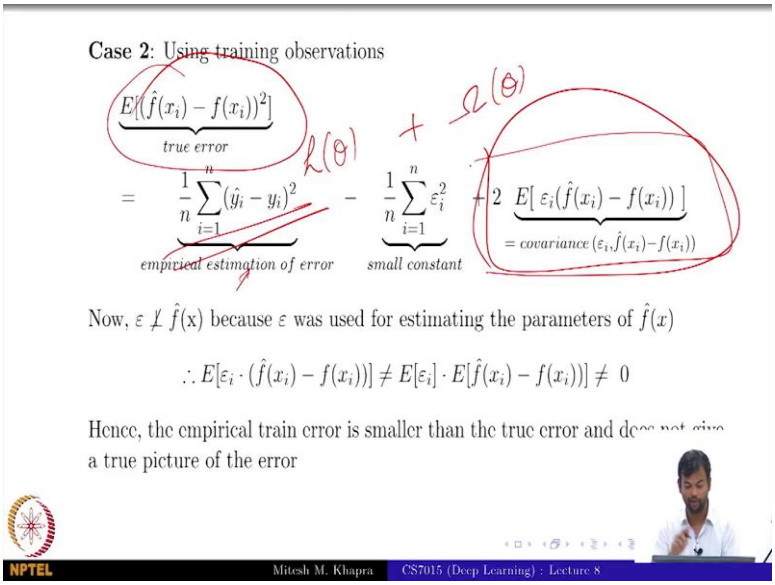
data for this estimation, right? It is largely dependent, but that is again a normal assumption that you make. So, this is ok. Good that you asked at this point. I will be doing a couple of things today where we will be deriving some things. We will try to prove some things mathematically, but all of these would have underlying some assumptions.

So, if you remember the Adam derivation with this we did there, also we had made this funny assumption that the gradients are actually coming from a stationary distribution which will not happen in practice. So, this reminds me of this joke from Big Bang theory. which says that I have a solution, but it only works for squared eggs in a vacuum, right? So, it is basically all these things always have some assumptions underlying them. But the idea is to kind of ignore those assumptions and see what happens in a neat setting and at least see whether in a neat setting everything works fine or not.

So, that is what is happening here. So, is a valid point that you are assuming that the noise comes from a zero mean distribution. Now, if the noise did not come from a zero mean distribution, then this would have not gone down to zero and the mean would have been higher than this is no longer a small constant and so on. So, those things are there. So, this is going to happen in some of the other derivations that I do today. It is not that I am teaching you something wrong. It is just that you have to take it with a pinch of salt in the sense that these assumptions are there and the original derivations these are not my assumptions. And they work only under those assumptions.

So, you have to be careful about that, but the idea is that still with these assumptions, can we at least make something meaningful out of it. right Is that fine with everyone? Can we all work with that basic premise? So, what I have done so far is told you that if you are estimating the errors from the validation data, you are doing a good job.

(Refer Slide Time: 15:37)

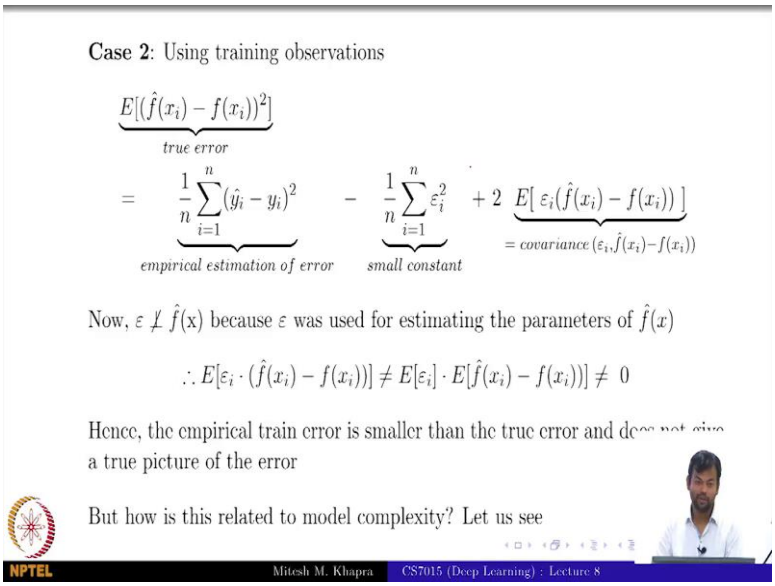


Now, let us see if I would estimate the error from the training data. Take a guess what would happen? What would my argument for this be? Now, this will not disappear, right because these two are not independent now. I cannot write it as a product of two expectations. That means, it will not go down to 0. So, that is the argument which I am going to make.

So, hence actually the true error if you see, right it is equal to the empirical estimation plus some quantity. That means the true error is dash as compared to the empirical error. That means the empirical error that we see is pessimistic or optimistic? Optimistic: that is what I started with that. You gave a very optimistic estimation of your error if you are looking at this empirical estimation from the training data because you have ignored this quantity. Is it fine? So, what is missing in the story?

Let us see now what was this quantity. So far all our discussions L theta right, but now suddenly I have realized that my true error is actually L theta plus something else right? You see where I am headed with this ok. So, that is what we need to see now ok. Now think it would be we should, but I am pretty sure it is positive. I cannot work it out right now but I am pretty sure it is positive, and you can see and if you find it is not then let me know.

(Refer Slide Time: 17:09)



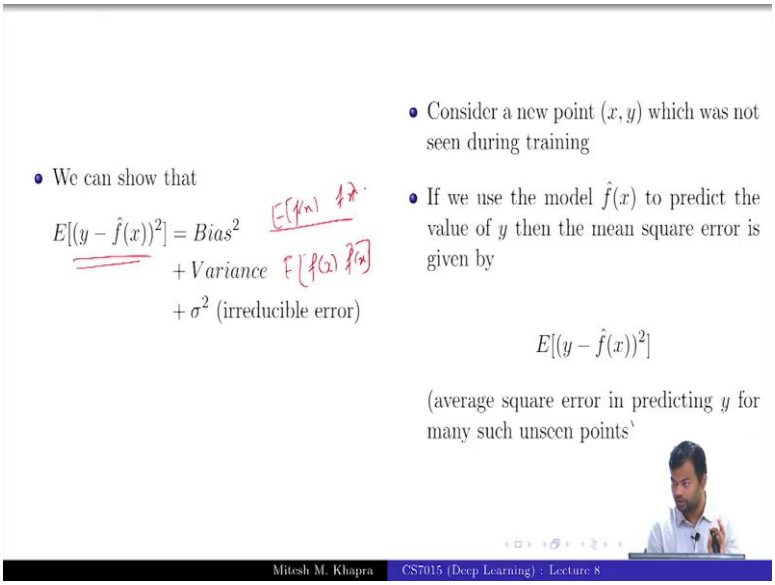
So, how is all this related to model complexity? We started off with this idea that model complexity tells you how much is the bias, how much is the variance and because of that you get these two curves that you are not happy with. One curve being very optimistic and the other curve being a bit pessimistic. Now, how does this discussion tie up to model complexity?

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 8.2
Lecture - 08
Train error vs Test error

So, we would start the next module where we will talk about Training Error versus Test Error and before that we will see this Bias Variance Tradeoff.

(Refer Slide Time: 00:20)



So, now what have we done so far in these complex models and the simple models, we have trained them using the dash data training data and what are we interested in always a test data, right? I already know; what was the oil amount of oil mined from the training data locations that I was given and I am not interested in predicting those. I am less interested in learning those, so that if you give me a new location, I should be able to do the right prediction.

So, I am always interested in the test data. So, now consider a new point which is not seen during the test data and there are several such points that you could see. Now, if you use the model f hat x to predict the value of y, then the mean square error is given by you get this. It is just the expected value of this squared error that I will get. So, what is the

randomness here? Y expected value because the x that I am going to feed at test time is going to vary for each of these different xs, I will get a different error.

So, hence that is a random variable. Do you get that? So, please focus on these things, right? I mean just do not take a formula for granted. Just see what is it trying to see. So, whenever you see an expectation over something, always question; what is a random variable here? So, what is the random variable here? It is the squared error loss. Why is it random? It is because it changed the input x. You are going to try it over a multitude of test examples. You will take 1000 text examples, 10,000 text examples and so on right? For each of this, you will get a different squared error. That is the randomness. So, you want to see what is the expected value of this or very loosely speaking the average value of this. Now, it turns out that this.

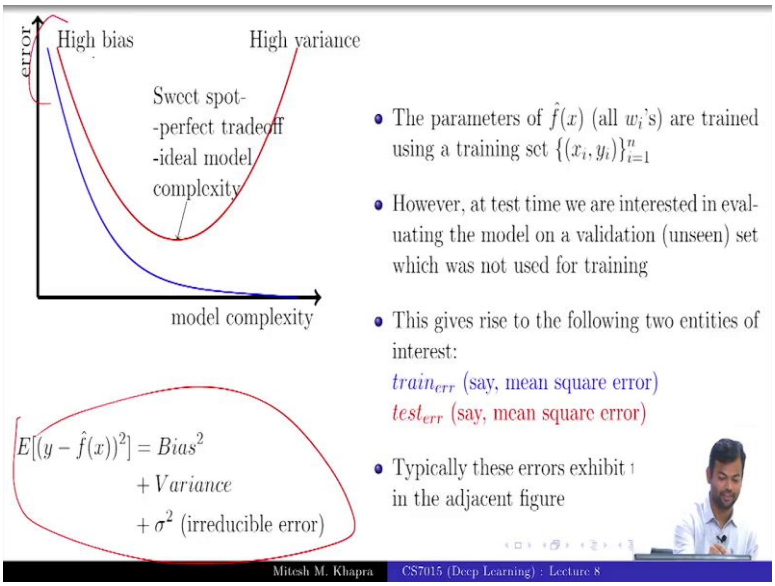
Now, just try to remember that this is also some expectation and you had the terms f x and f x hat here. This also had some expectation and term f x and f x hat and so on, right? If you do not remember the exact formula it is ok, but you do remember there were some expectations inside and the terms f x and f x hat whether they are. So, this is just simple. You are dealing with a minus b the whole square on the left hand side.

If you if you open it up, rearrange some terms, you will get this, right? So, you can show that the mean average or the expected square error on the test data is actually the bias square plus the variance. That is a small amount of irreducible error. You can go back and work this out and actually the proof is given here on the link, ok but I hope you get the intuition. You have this a minus b the whole square.

If you open it up and rearrange the terms, you should be able to get this. Now, what does this tell you? What happens if the bias is high? The squared error is going to be high. What happens is if the variance is high, it is going to be high. So, that is why you do not want a very high bias, you do not want a very high variance also. You want this sweet spot in between where the bias and variance are just about optimal. You get that? That is why there is a tradeoff between bias and variance.

You cannot rely on simple models which have high bias; you cannot rely on complex models which have high variance. You want something in between.

(Refer Slide Time: 03:18)



Now, the parameters of f hat x remember that they are trained using the training data which consists of these end points that you have. At test time we are interested in evaluating the model on a validation set which was different from the training data. This gives rise to the following two quantities. One is the training error which you deal with at dash time training time. That is the error that you are trying to minimize, right? But a test time you have a different error which is the test error and that is the error that you care about.

Typically these two errors exhibit a certain trend. Do you know what the trend is? Now, on the x axis, I have model complexity and on the y axis, I have error. As a model complexity increases, what would happen to the training error? It will go to almost 0. That is exactly what happened from the linear function to the polynomial function. This is how it will behave. As the model complexity increases, As the model complexity increases what would happen to the validation error? It will decrease up to a certain point, right because you are still not over fitting on the training data. Your answers are still generalized.

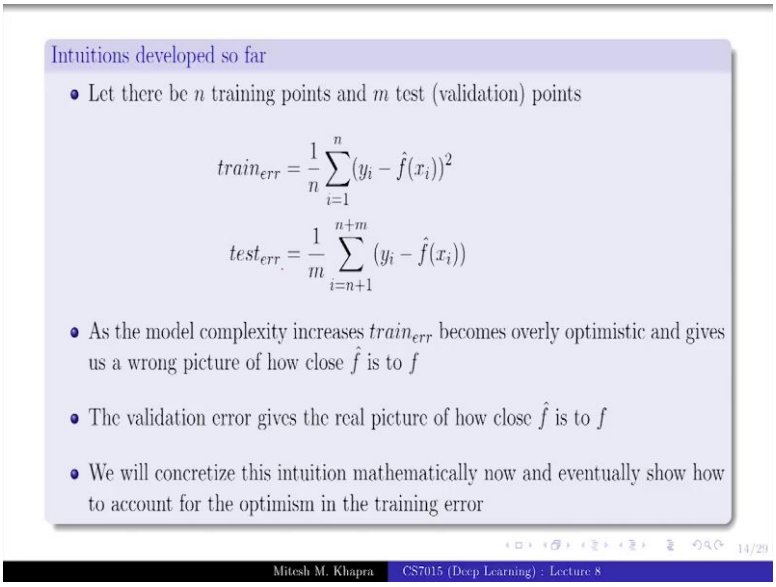
So, you had this degree 1 polynomial degree 25 polynomial. If I take in something in between, then probably this is where I would have ended up with the training error and that would not have been too bad for the test error. You see this. Ok. Now, you see I will mark two points two regions rather one of this corresponds to high bias, the other one corresponds to high variance. Tell me which one is, which do this? I cannot understand. So, let me ask this is? This is ok?

Good. So, you see that there are; these two extreme and we want somewhere to be in between, ok. At least you get the intuition behind this fine ok? And you are looking for this sweets spot which is the perfect tradeoff between the bias and the variance, right? So, now everyone gets why there is a tradeoff and how this relates to model complexity and therefore, we are looking for the ideal model complexity. How do we achieve the ideal model complexity? Well, we cannot really. Ideal is ideal, but we try to do this using dash. What is the title of this lecture?

Student: Regularization.

Regularization, I will try to use regularization to achieve this, ok. So, let us formalize this a bit more, and remember that this curve is actually because of this equation that you see, right high bias, you will be in this region. I am actually inserting it, ok fine ok?

(Refer Slide Time: 05:50)



So, the intuitions that we have developed so far is that if there are n training points and m test points. Then we have a train error which goes over the training points and we have a test error which goes over the m test points, ok?

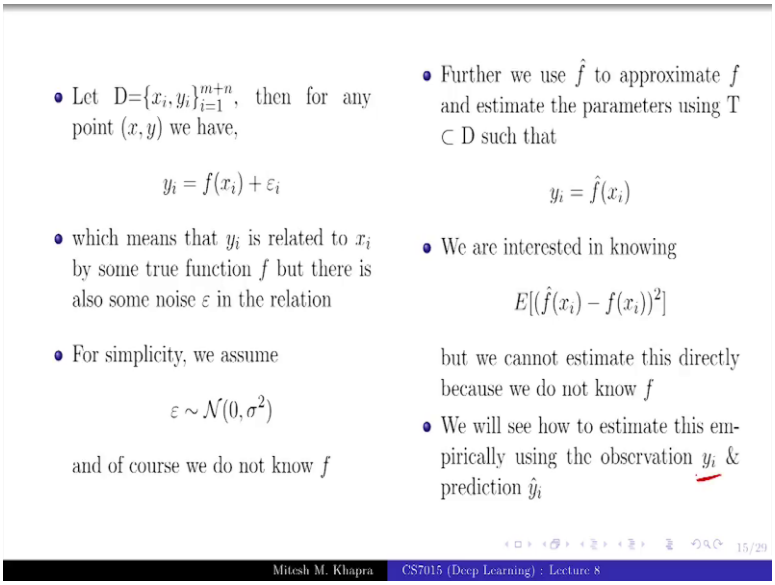
So, I am just taking a total of n plus m points. The first n is training the next, last m is test. Now, as the model complexity increases, what happens to the training error, it becomes very optimistic and gives you a very wrong picture of how close the predicted function is to the true function whether it makes you feel that you have done a perfect job, this you

have actually discovered the true function, but that is not correct. It is giving you a false picture of that. Therefore, we should always look at the dash error.

Student: Validation error.

Validation error: so now you see that why you always do this train validation and test split. Test is unseen. You try to optimize on the training error ok but you should always tune for the validation error. Your optimization algorithm is going to take in the training error. It is going to be very optimistic, it is going to try to drive to 0, but you should look at the validation error and try to see that you are not over fitting on the training data everyone gets this intuition.

(Refer Slide Time: 07:09)



So, now this is all intuition. We will have to concretize this mathematically. So, that is what we will do now. So, that d be these training test points that we have, we know that this relationship holds. We do not know what f is, but we know that this relationship holds. So, what am I trying to say here that we know that there is a true relation between y and x which is given by the function f, but I am also willing to admit some noise that may not be a very neat function, but a small noise might exist.

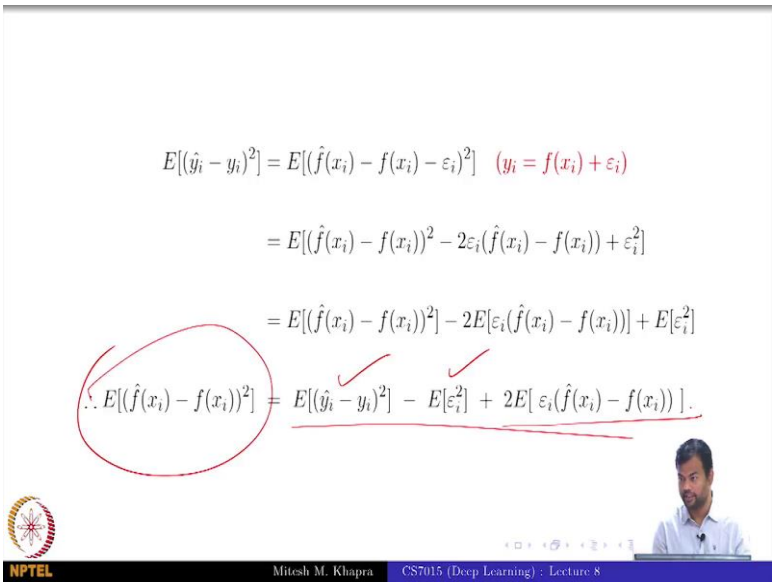
That is the epsilon i, ok and I am going to assume that epsilon comes from a normal distribution with zero means. So, on average the noise is going to be 0, but there is a small variance. Everyone gets this. This is a true relation, but I am willing to admit some noise in

the relation, ok fine and of course, we do not know f we never know f, right? Now, going by our paradigm where we have these five components, we use f hat to approximate f f hat will have some dash which will I try to learn from the training data. What is this dash parameters, right which will try to learn from the training data? The training data t is a subset of your total data which is thus those endpoints, right and we are interested in knowing this quantity. This is what we are actually interested in. Can we compute this quantity? How many of you say yes? How many of you say no? We cannot. Why cannot we compute it?

We do not know f. So, why cannot you raise your hands if you all can answer in chorus? So, we do not know what f is then how do we compute this quantity right, but what do we actually know. So, now we are going to see something which is true expectation and something which is empirical estimate expectation. How many of you know this? What is the difference between the two? Most of you should, but it is not confident about it ok. So, we do not know what fxi is the true thing, but what do we know, we are given some training data, right?

We know these yi's for was training data and we know these yi hats for those training data.

(Refer Slide Time: 09:11)



So, this is something that we can estimate. Yes or no, this is given to us. So, this expectation is going to be an empirical estimate, right because we are going to look at some 1000 10000 20000 training points and estimate this, right? It is an empirical estimate.

How many of you get that? Now, I am just going to rewrite some of this. So, what I have done is, I just defined that yi is equal to fxi plus epsilon i. So, I have just replaced yi by that, ok. Is that fine? Now, this is of the form a minus b the whole square. So, I am going to treat it as that and just open up the bracket. So, I will have a square minus 2 a b plus b square and now, this is a sum or difference of expectation. So, I can push the expectation inside. So, this is what I get this, is this fine ok?

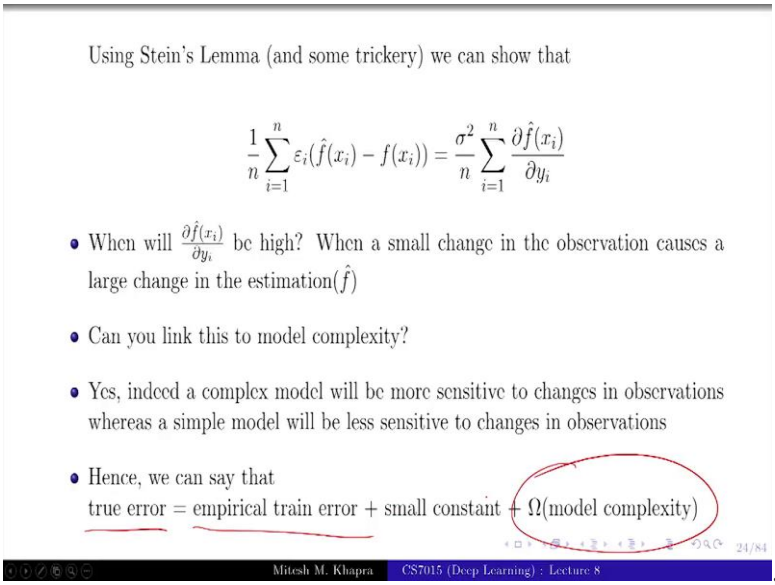
Now, I am just going to rearrange the term. So, remember this was the quantity that we were actually interested in, but this is the quantity that we had a handle over because these were the data points given to us. So, I will just rearrange the terms and I can write this which was my quantity of interest as this. Can you estimate everything on LHS, on RHS? This? this? What is this variance? Sigma square; we assumed it came from 0, sigma square distribution and this? can estimate the answer is no for the same reason. We do not know what f of x is ok?

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute Of Technology, Madras

Module - 8.3
Lecture - 08
True error and Model complexity

So, now we will try to see that how does this true error that we see depend on the model complexity.

(Refer Slide Time: 00:19)



So, using Steins Lemma and some trickery, we can show the following. What is Steins Lemma? So, I had this deal with my students last year. You do not ask me what Steins Lemma is, I will not ask you what Steins Lemma is, ok. So, it is some lemma which tells us that, this quantity, what was this quantity the last term which was troublesome rate that covariance term which was troublesome. That is this quantity. This quantity is actually equal to this quantity.

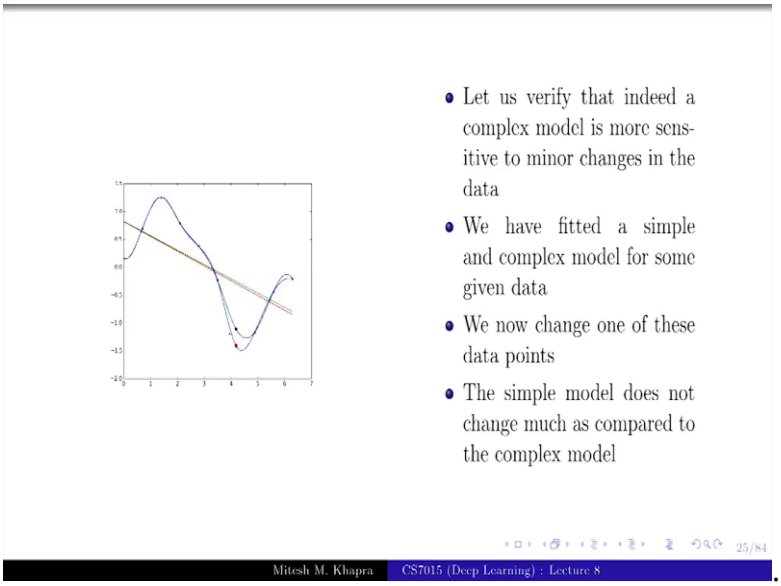
So, let us buy that. Let us all of us agree that Steins Lemma is correct and it tells us that this is the case ok, and you saw the quiz 1 paper ok fine? From last year I mean, ok. So, now we will work with this premise and we will see what it actually tells us. Now, when will this quantity be high? So, what this is telling us? I mean jokes apart. Let us try to focus again that this quantity is actually equal to the summation of this quantity.

Now, let us take one term in this summation when would dou f hat x i by dou y i be large. What does it actually tell you? If I change one of these yi's a bit when the prediction for it is going to change by a lot. Do you get that? How many of you get this? Some of you do not get this. Just think about it. When would this be high? What does the derivative capture? If the derivative is high, that means a small change in the denominator is going to lead to a large change in the numerator.

What is the denominator? Actually the true y that, we have observed. What is the numerator? That is the predicted y. So, what you are saying is that if there is a small change in y i, then there is going to be a large change in the prediction, ok. When would this happen? Would this happen for simple models or complex models? Complex models; how many of you say complex models? So, this is the link to model complexity rate and I will make a more intuitive case for this, but at least some of you get this that if your model is very complex, that means it is even one of your data points changes and the prediction of the model is going to change largely.

So, now, relate this back to that sinusoidal model that we had and we had this complex model, every model that I was training which was strained on a different set of 25 examples, the model was vastly different and that is exactly what was happening. When you were changing even one data point, your predictions were changing largely. That means, your model was changing largely. Do you get that intuition? So, indeed a complex model will be more sensitive to the changes in the observation whereas, a simple model will be less sensitive to it, and hence, we can say that the true error is actually equal to the empirical train error plus something which relates to the model complexity.

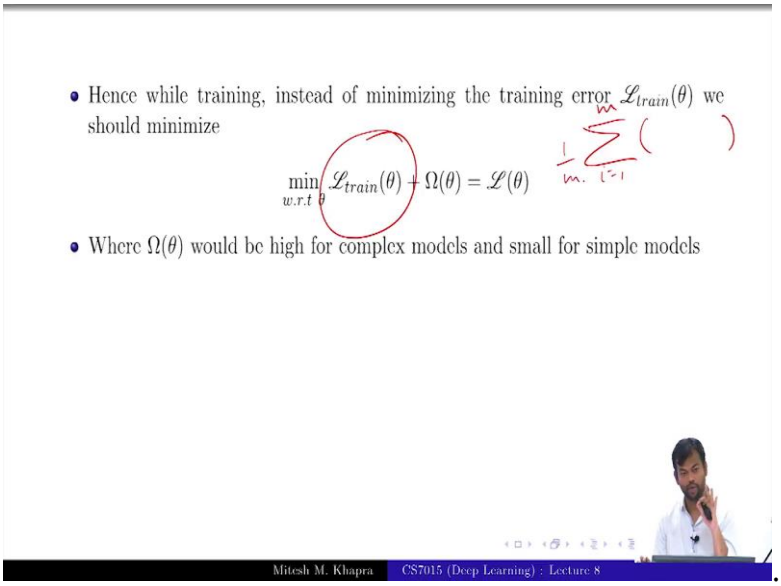
(Refer Slide Time: 03:17)



Now, let us first verify that indeed a complex model is more sensitive to minor changes in the data. So, this is some data that I had sampled from the same distribution and I trained one simple model which is the green line which you see that was a linear model and I trained one complex model which was a 25 degree polynomial which you see, ok. Now, what I am going to do is, I am going to take one of these points and change it a bit and I retrain the model.

What happens to the simple model, it does not change much, but what happens to the complex model, it is more sensitive to these observations that I have and that is exactly the quantity that we were interested in. That means, a complex for a complex model which is more sensitive that summation that we care about is going to be high. That means, that difference between the true error and the estimated error is going to be high.

(Refer Slide Time: 04:10)

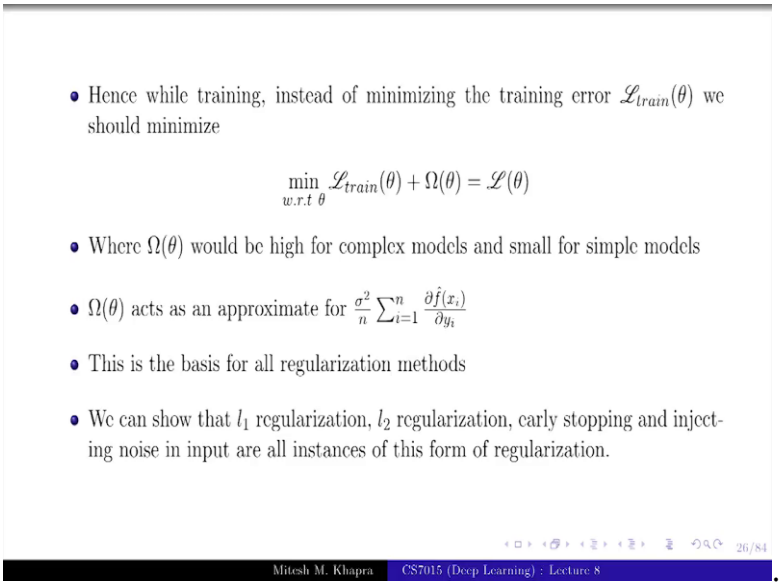


So, that is why instead of minimizing the train error, we should always minimize the train error plus some quantity which is linked to the model complexity. This is the basis for all dash methods Regularization method.

So, now you see where this comes from. So, ok where omega theta would be high for complex models and simple for simple models, ok you get the intuition for this and the rest of the lecture, we will spend in taking various cases where we will actually show that omega theta would be high and we are trying to control for omega theta. This quantity for the rest of this lecture and for the rest of this course I will assume that we all know how to deal with.

We have done enough of this, we have done a lot of back propagation, we have done enough derivations of the laws with respect to the output layer and so on everything, right? So, all of us understand how to deal with L train theta where L train theta is this L equal to 1 to m squared error loss or your log likelihood or any of these losses, right? So, we all know how to deal with this today. We are going to focus on this other term which brings in the regularization.

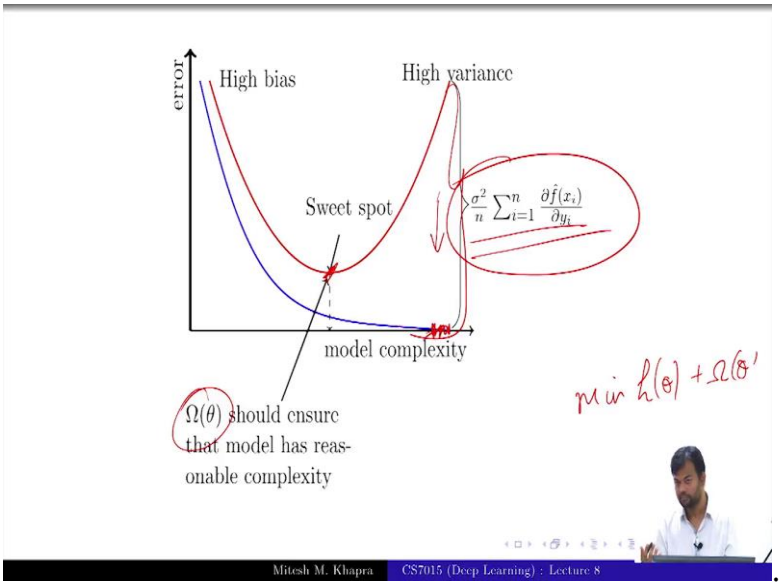
(Refer Slide Time: 05:15)



So, what omega theta does is actually acts as an approximation for this. So, what I should have actually tried to minimize is not just L train theta, but L train theta plus this other quantity which was there in my equation. You get this my true equation was that my loss is equal to L trained theta plus this term, right which we approximated using Steins Lemma. So, I should have tried to minimize this quantity, but I do not know how to really compute this quantity.

So, I am going to just substitute it by omega theta and ensure that omega theta is such that it is high for complex models and low for simple models. Do you get the recipe? Everyone gets this? How many of you understand this? fine So, we can show that L1 regulation L2 regularization early stopping all of these are actually special cases of this particular formulation that we have.

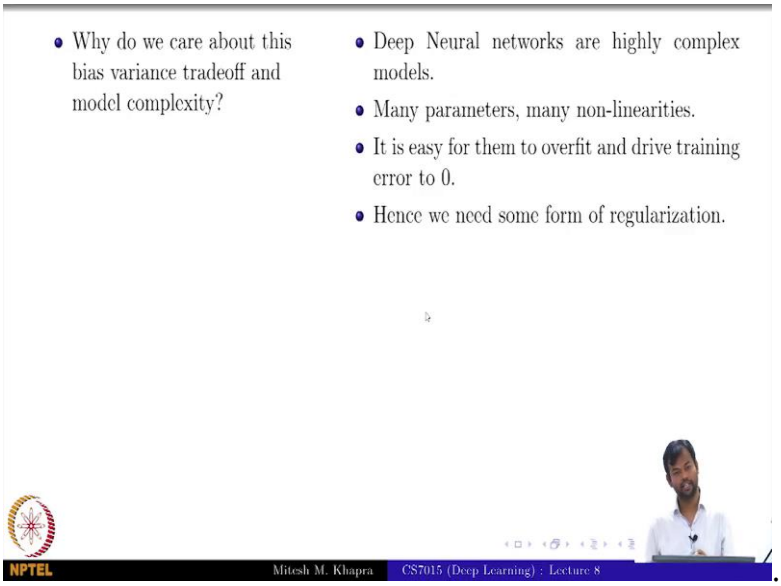
(Refer Slide Time: 06:09)



And remember that this is the sweet spot that we were aiming for ok, and this gap is actually, this quantity because we are making a very optimistic estimation of the error whereas, there is actually this quantity which we have been ignoring and that is why we see that the validation error is high, ok. So, is the full picture in terms of the diagram and all the equations that we have seen.

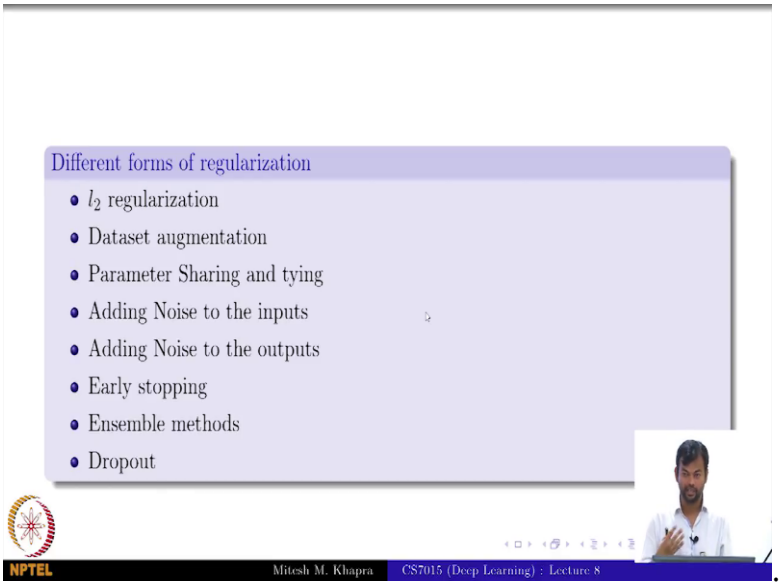
So, we should ensure using omega theta that this gap is also minimized. Therefore, our function should be minimized L theta plus omega theta. So, essentially what we are trying to do is minimize this gap and hence, the model would generalize better on the test data. Is this intuition clear to everyone?

(Refer Slide Time: 06:57)



Why do we care about this bias variance tradeoff model complexity? This is not a course on machine learning. They are highly complex models, they have many parameters, many non-linearities. In fact, now can you relate this back to the universal approximation theorem? What is the universal approximation theorem say give me any data, I will give you a deep neural network which will exactly over fit the data, right and that is exactly what we want to avoid. That is why regularization is important in the context of deep neural networks, fine. It is very easy for them to over fit the data and derive training error equal to 0 and that is why we need some regularization.

(Refer Slide Time: 07:35)



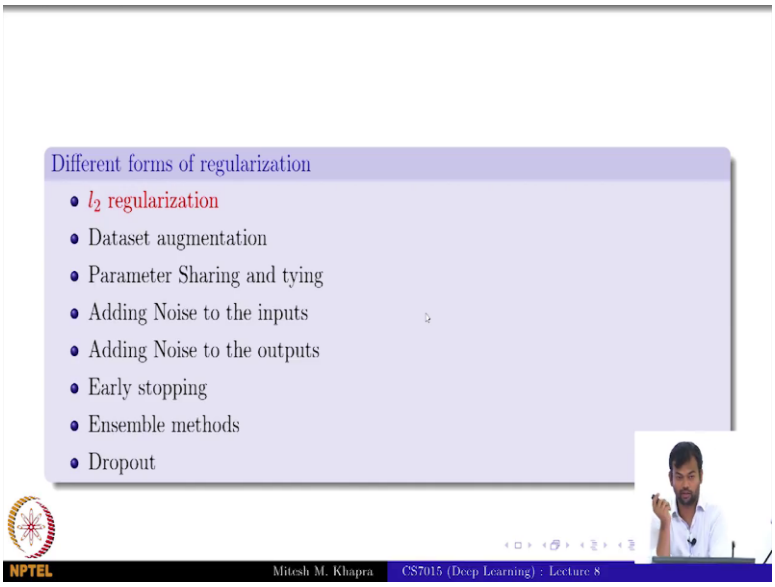
So, today we are going to look at different forms of regularization starting with L2 regularization; some simple tricks. So, some of these are going to be mathematically motivated, some of these are just going to be heuristics or empirical stuff. So, data set augmentation is one such empirical stuff. How many of you tried data set augmentation for the immunized assignment or the back propagation as parameter sharing and tying is something that? No, I am not.

Please, do not give me that look. Yeah I am not suggesting that, adding noise the inputs, adding noise to the outputs, early stopping, ensemble methods and drop off, right? So, these are the things that we are going to talk about this and all of this is in the context of regularization where you want to avoid some kind of model complexity.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

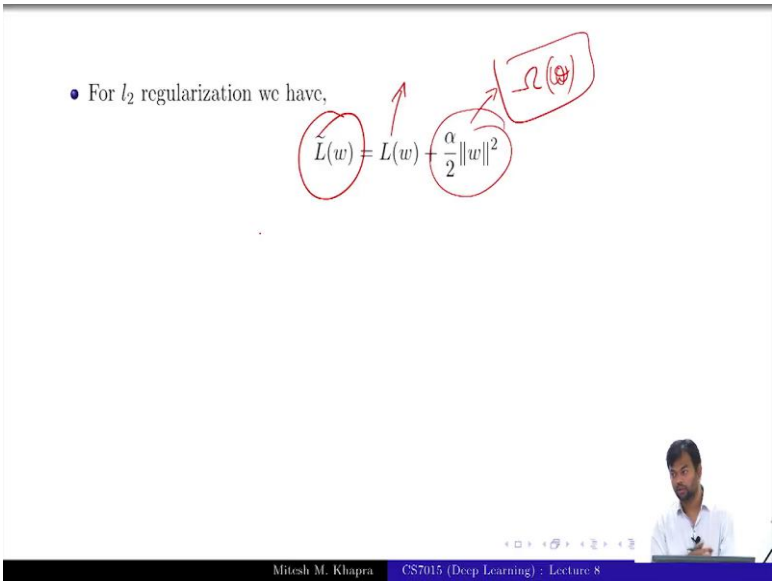
Module - 8.4
Lecture – 8
L2 regularization

(Refer Slide Time: 00:13)



So, let us start with L2 regularization. So, I have seen this before.

(Refer Slide Time: 00:15)

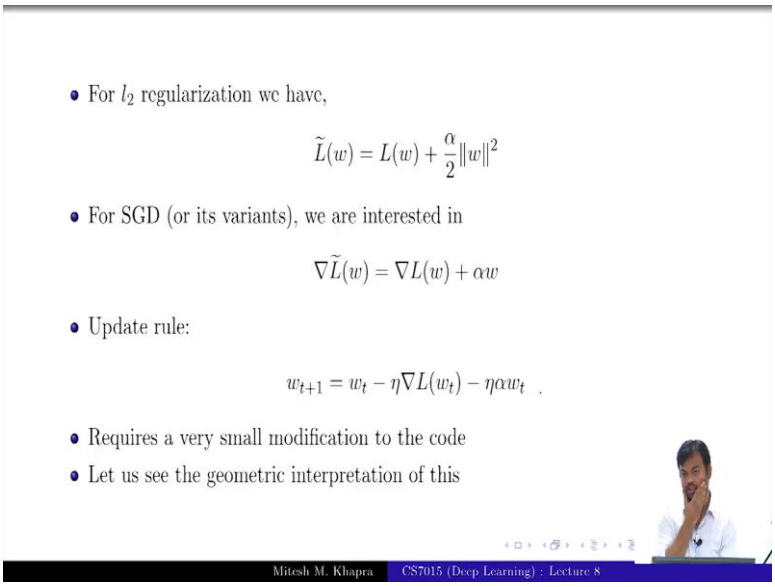


So, all of you see that this is L2 regularization right? What does L2 regularization does? Now tell me in the context of things that we have discussed today what is this? Empirical estimate of the train error ok and what is this. Is that fine right? So, everything that we are going to write is L, because of its w, but fine right ok. Now why does this relate to model complexity, what am I doing here actually by adding this?

So, they are going to see a very detailed analysis of this, but I just want to see first whether you get an intuition behind this. So, by doing that what you are trying to do? Not allow the model to become very complex right? You do not want a model where your weights can take any possible value; you just want the weights to be small. So, you are reducing the freedom on the model right, less freedom less complex, you get the intuition at least. We will see this in more detail, but at least you get the intuition why we are doing this.

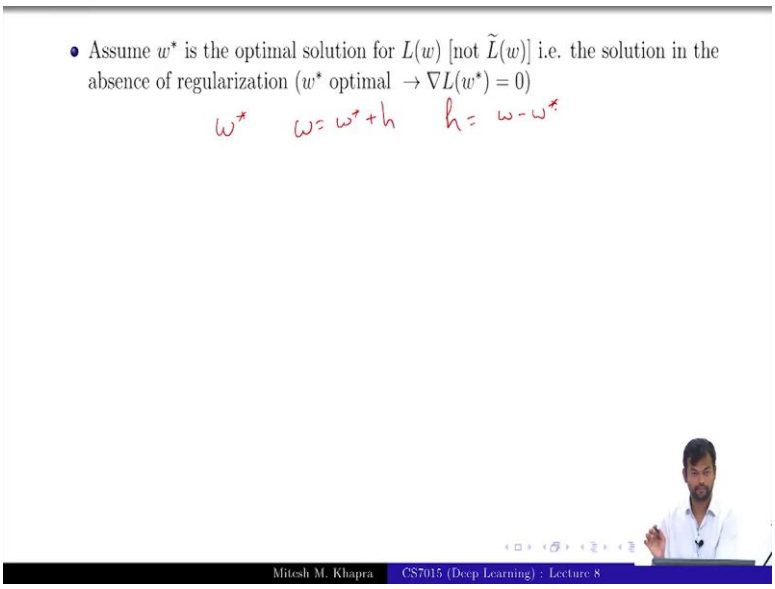
So, we are using omega. Remember that we are using this omega theta as a surrogate for model complexity. So, if you add something in all omega theta, just make sure you understand that this relates to model complexity ok fine. And now for SGD what would I need, for gradient descent. Just in case you have forgotten what SGD is, what do we need? Nothing, we have done it, [FL] gradient of this, which is a sum of the derivatives of the two quantities of which you know one right? You know this already, and what is the other guy; alpha w right.

(Refer Slide Time: 01:59)



So, you see this L2 regularization right one reason why it is preferred is now imagine you have already written code for gradient descent. All you need to do is change it at one place add this to your update rule; that is all you need and you can think of the vector form of this, where you have a vector of parameters, you can think of the matrix form of this variable vector matrix of parameters. All you need to do is add one term to your update rule, so it can be done with very minimalistic change and this would be your update rule. Now, let us see geometric interpretation of this.

(Refer Slide Time: 02:36)



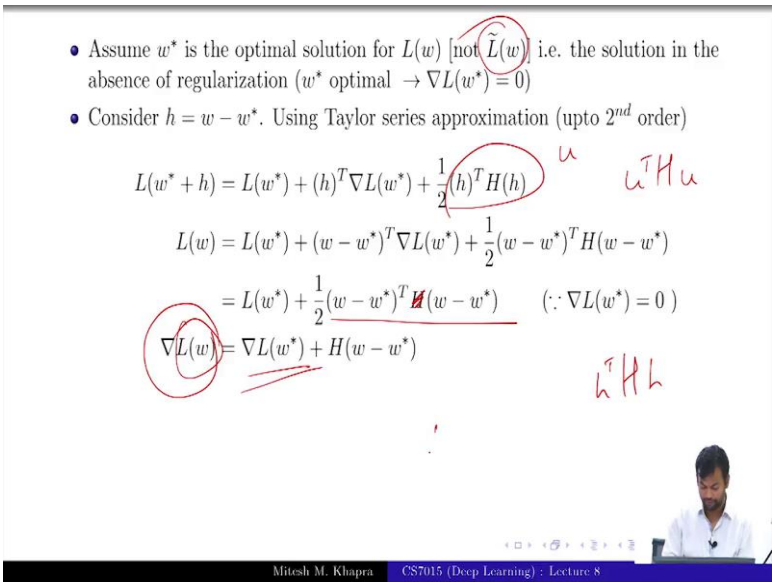
Now, from here onwards some of you will start getting a bit uncomfortable with some of the math, because of these assumptions that it only works for squared eggs in a vacuum right? So, you will see those kind of things, I will not tell you upfront what is the assumption I am making, because that will just spoil the analysis, you will just not enjoy it as much as you would ignorance is bliss right? So, if you do not know what the assumptions are, you will probably enjoy it more.

But for some of you will pick it up, just keep it to yourself, at the end I will tell you what are the assumptions I had made ok. There are some tricky assumptions that I want to make, but just live with it and just try to enjoy it while those assumptions last right ok. So, now, let us assume that w star is the optimal solution for L w, what is L w? The train error, not our regularized error, just the train error.

And, so if w star is the optimal solution what can you take tell about the derivative with respect to w star or derivative at w star sorry, it is going to be 0 from basic calculus right? So, which I say minimize x square, the minima is where derivative of x squared with respect to x is equal to 0 right.

So, now consider one point which is ok. So, what I actually want to consider is that, let me just see how to see this. So, let us see my w star ok and I want to consider some point in the neighborhood of w star ok; that is what I want do. So, one way of saying it is that h is equal to w minus w star, is that fine ok. So, that is what I am going to use in the next few steps.

(Refer Slide Time: 04:16)



So, suppose I have such an h which is equal to w minus w star; that means, I can move from w star to some point in its neighborhood by using h. And what does Taylor series tell us? This is what Taylor series tells us right, that the value of the function at this neighborhood point is equal to this. All of you know Taylor series well now, it is that fine I do not need to really go over this right?

This is approximation up to the second term second order derivative. Now what was h actually w minus w star. So, I will just substitute that and this is what I get, is that fine?. What is this quantity? 1 minus 0 infinity minus infinity 0 right we just did that ok. So, that term will disappear, what am I left with? This quantity ok and I have forgotten what is next?

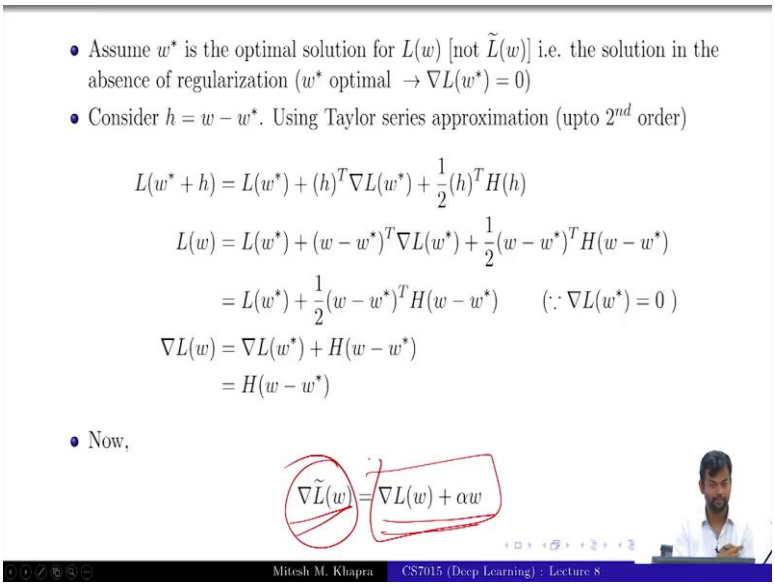
Now again I am interested in the derivative of this ok. So, what will happen if I take the derivative what would I get? I am interested in computing grad L w, what will the R H S be, how many of you fine with this? Remember this is a quadratic form right? So, this is of the form x square that is, I mean that is roughly how I remember it is not correct, because of the form x square. So, when you take the derivative one of the x is will disappear and this quantity will remain ok. So, everyone gets this ok.

So, now what do I have is, I have the formula for the gradient with respect to L w and it is in terms of the gradient with respect to or rather the gradient at L w star; that is what I have achieved. So, far, but what am I actually interested in, the regularized loss, I am, what I am still dealing with, is the non regularized loss. This is just the empirical estimate of the training error that is not what I am interested in, I am interested in the regularized loss.

How many of you lost at this point? h is the second order derivative oh. So, these are brackets just for clarity, but I see it is making it more unclear yeah, actually we should have used u and then call it u transpose H u. So, it is the brackets here are not indicating function ok this is just h transpose H. Now let us say it I realize how bad it is. So, last step what are we taking gradients with respect to is w right, is it fine.

So, we have a. So, I mean do not get too confused right? So, up till this point we have a formula for L w right, and I am just interested in the derivative of that ok. And all I have achieved by this is that, I have ok. In fact, I have one more step right.

(Refer Slide Time: 06:50)

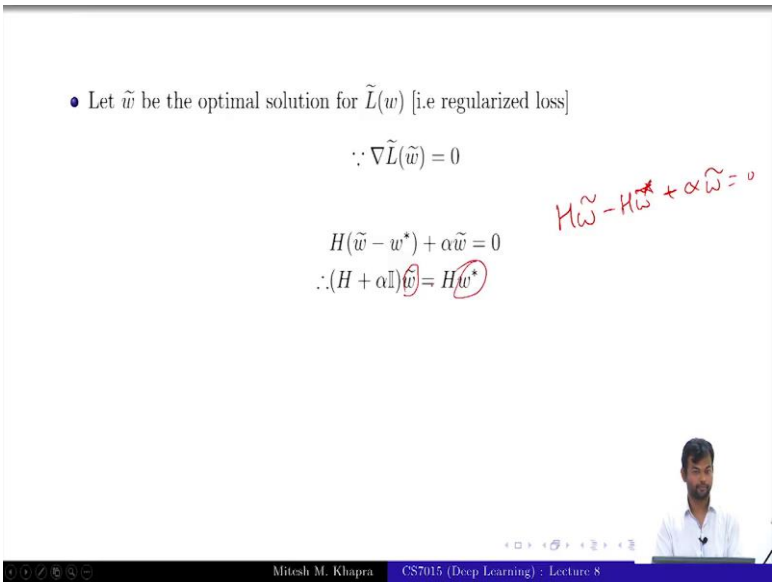


What is this quantity zero ok. So, we now know that the derivative of the loss function with respect to w can be written as this quantity. Is it ok, and I have just derived it step by step, there is nothing great about it, anyone is can. Why I am doing this is not clear that will become clear hopefully, but what I am doing is clear right, is that fine can I move ahead .

Now what we are actually interested in is this quantity, because this is the true loss that we are going to deal with right and we just saw in the previous slide that this quantity which is on the L H S is equal to this thing on the R H S, this is what we saw on the previous slide, can I just go back to the previous slide, because the derivative of this was just alpha w. Now let us start with this. So, on the next slide, let me just see if there is anything else that I need to see here ok.

So, far everyone is clear what I have derived so far why is not clear, but what is clear, what is being derived so far. So, I have said that the derivative of the loss function or the regular is loss function can be written as this quantity ok, is that fine, where w star is the optimal solution for with respect to the un regularized loss function ok. And now I have what I am interested in this solution with respect to the regularized loss function ok.

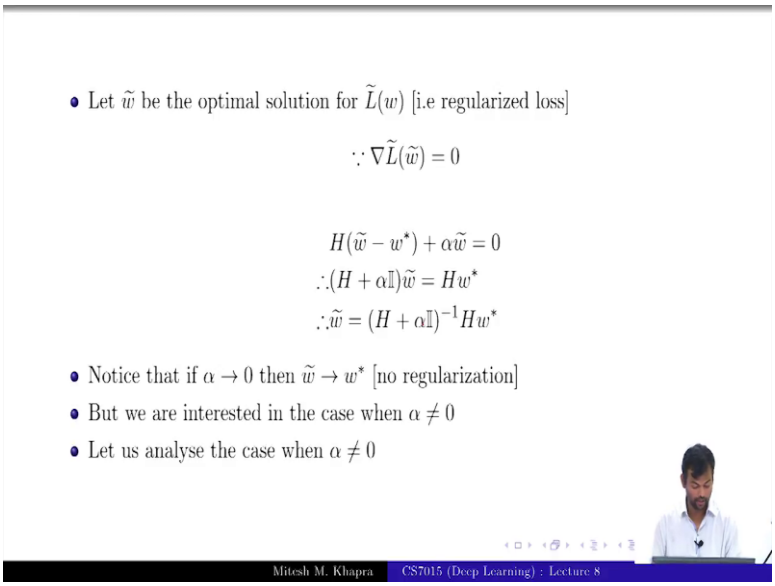
(Refer Slide Time: 08:08)



Now, let w tilde be that solution for the regularized loss function. So; that means, the derivative of the loss, the regularized loss function at w tilde is going to be 0, nothing great about this, but I just told you on the previous slide that I can write this quantity as this quantity that is what we derived on the previous slide ok. Just take my word that is what we derived on the previous slide ok, let just, no confidence in me ok that is fine. Now can you. Are you if I write it as this, just rearranging some terms oh sorry.

So, I am just grouping all the w tilde some terms and this is, a matrix is needed here right, because I need to, I can only add two matrices. So, what I am just doing is, putting the elements across the diagonal. Everyone understands this, everyone gets this step, ok.

(Refer Slide Time: 09:19)

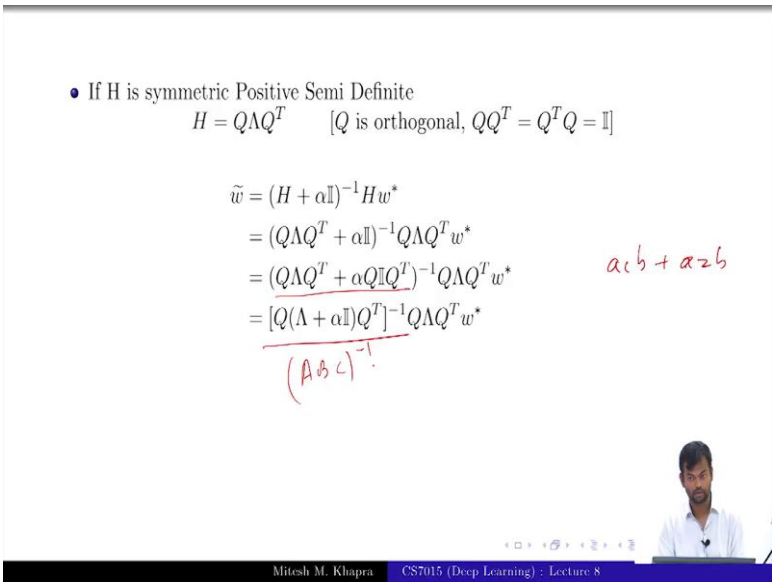


So, now I have a formula for w tilde in terms of w star ok. I am going to go a bit further and be a bit bold and compute the inverse also. So, now, I have a exact formula for w tilde in terms of w star. So, what is this, actually what is this relation that I am trying to establish? Suppose I know the solution with respect to the un regularized loss, and now I have added regularization what happens to the new solution.

So, I am telling you the new solution would be smaller weights and so on that is what L2 regularization tells you, now you are just trying to make an interpretation for that. So, I have given you a closed form solution that w tilde is actually equal to this quantity that you see on the right hand side ok. Why you are doing this is still not clear but right now I just focus on the what part of it this is just some mathematical steps that I am doing, anyone who is not comfortable with this.

Now notice what would happen if alpha tends to 0 what would be w tilde be w star, what do you mean by alpha equal to 0, no regularization right? So, that is just one corner case that I want to do, but that is not what we care about anything what that is stupid to do all this and tell you that if you do not use regularization you will get the same solution, but that is not what I am going to tell you right? We are interested in the case when alpha is not equal to 0 ok. So, let us look at that case.

(Refer Slide Time: 10:35)

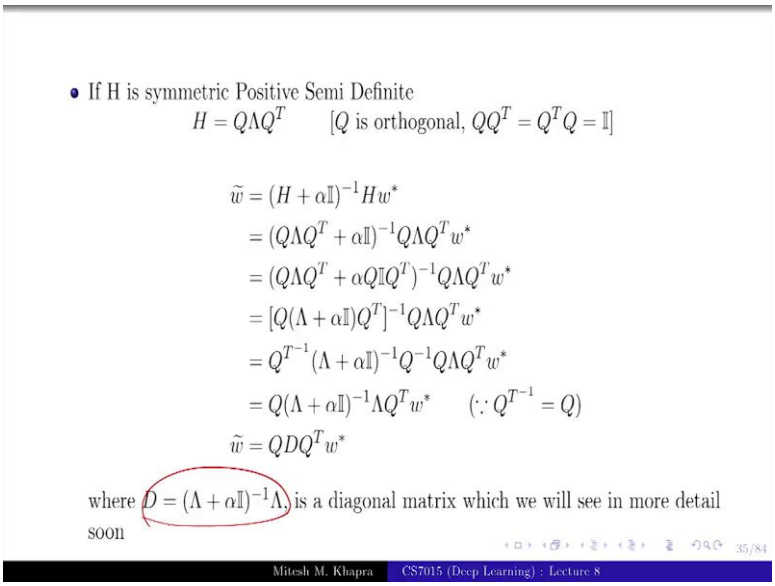


Now, I am going to assume that H is a symmetric positive semi definite matrix squared egg in a vacuum ok. So, if that is the case then I can write H as this, I have just done the dash of H, eigenvalue decomposition all right ok, and I know that since it is a squared symmetric matrix the eigenvalues are going to be eigenvalues are going to be orthogonal yes, eigenvalue vectors are going to be orthogonal and that is why I can write this that Q transpose is the inverse of Q.

Now, let us start with whatever we had on the previous slide and substitute what, what I am going to substitute? Instead of H I am going to use Q lambda Q transpose ok. So, I am doing that. So, is that ok, I will just go over the steps and let me know at any point if you have a problem. What I have done is, I have replaced this I by this and its valid, because Q Q transpose is just equal to I, I have just taken q and q transpose as common right? So, this is a c b plus some a z b. So, I have taken a and b out right, is that fine ok.

Now, what is the next thing I am going to do? This is of the form a b c inverse. So, I am going to write it as, and the inverses are neat right.

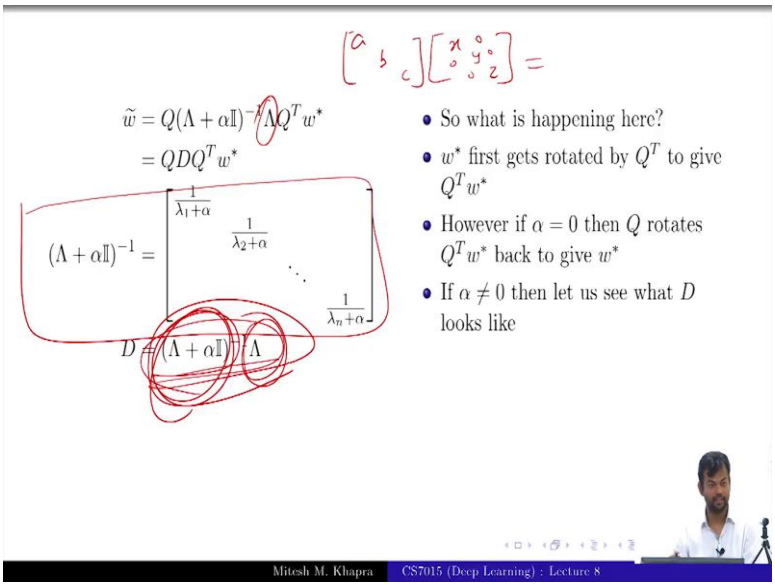
(Refer Slide Time: 12:08)



This is fine, what will happen to this quantity I, what is this quantity Q and this is what I am left with, but there is still something more I can do I guess let us see ok. So, I can write this entire thing as a diagonal matrix. How many of you see that it is a diagonal matrix, because lambda is a diagonal matrix, I of course, is a diagonal matrix, I is multiplied by a scalar which is also going to be a diagonal matrix and the whole thing is again multiplied by some diagonal matrix ok. What is the inverse of a diagonal matrix? The reciprocal of the diagonal elements.

So, I, its fine. So, I have a very neat formula for what w tilde looks like in terms of w star ok. Again why am I doing all this and God knows, but and here D is equal to this quantity.

(Refer Slide Time: 13:00).



So what exactly is happening here, in terms of linear algebra or in terms of geometric interpretations. So, let me just see if I have to do something first ok. So, what is happening to w star is getting

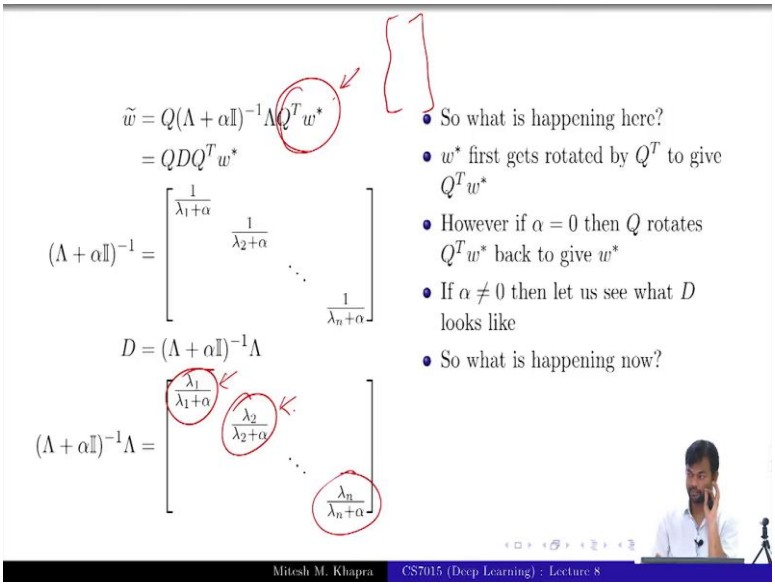
Student: (Refer Time: 13:15).

Rotated, remember what happens when a matrix where hits a vector, it gets rotated and scaled also. And then what is this diagonal matrix going to do, scale it, element wise scaling actually, everyone gets this operation ok and then I am again rotating it by Q. Again the same stupid question if alpha is equal to 0 what would happen? Q transpose would rotated by something and then Q would rotate it back way; that means, you will end up getting the same solution ok. If alpha is equal to 0 we understand.

Now if alpha is not equal to 0. First let us see what does this matrix look like. So, what is this matrix actually, it is a diagonal matrix, what are the diagonal elements, the. What is the first element in the diagonal? 1 by, everyone agrees with this, what is the second element ok fine, and what is the other matrix that I have? Lambda. So, D is equal to the product of these two things right? So, what is D going to be? What is the first element of this matrix is going to be, how many if you say lambda 1 by 1 lambda 1 plus alpha, this much is clear, everyone gets this.

So, this is a diagonal matrix of the form a b c, let us consider a 3 by 3 matrix. Now I am going to multiply it by another matrix x y z which is also a diagonal matrix right, because this is also it. So, this matrix I have already told you what it looks like, the other matrix is also a diagonal matrix. Now what is this product actually a x, b y, c z and everything else has 0. Now everyone gets it. now can you say what would this product look like if you can actually make out, it would be a diagonal matrix and what would the diagonal elements be?

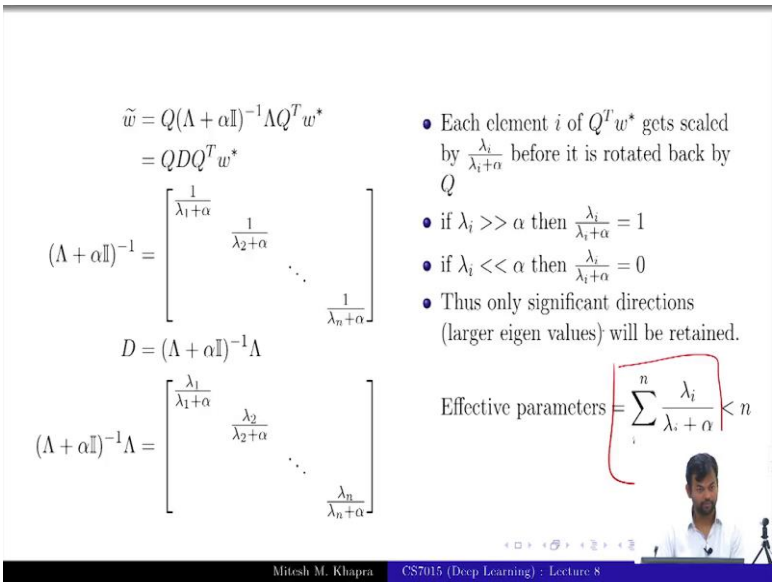
(Refer Slide Time: 15:18)



So, now what is happening? So, first this rotation is happening that no one is denying, after rotating what is happening, this is a, this product is actually a vector; that is fine ok. What are we doing to every element of the vector, scaling it, scaling it by what quantity these quantities that every element is getting scaled by the corresponding entry in the diagonal, in this diagonal right?

So, the first entry is getting scaled by this, the second entry is getting scaled by this and so on ok. I just want you to take some 30 seconds and try to figure out where I am headed from here.

(Refer Slide Time: 16:03)



Let us see if I can, yeah maybe look at this sentence and see. First of all everyone agrees with this sentence right? Is there anyone who does not agree with the sentence? I am just trying you to figure out the implication of the sentence, you get it, some people are nodding their heads just in, because if you scale it right, then there is no guarantee that what the vector has changed ok, what happens in the following case; that means, that dimension will be left as it is ok, but if the eigen, if this condition holds what would happen that dimension is almost getting multiplied by a 0 right.

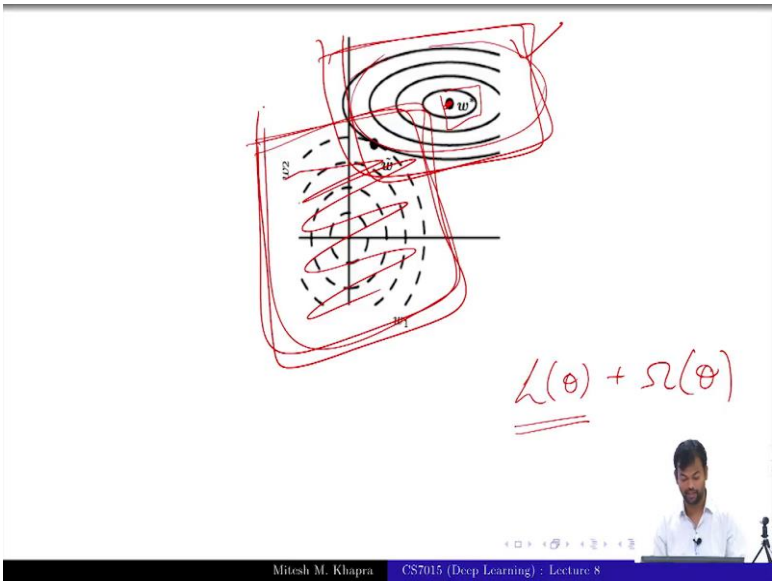
So, see these two extremes, when the eigen value is very large you will end up staying where you were, so those dimensions will not be affected. If the eigen value is very small then you are almost getting scaled down to 0. So, now, what will happen is actually, only the significant directions larger eigen values will be retained. So, what is the effective number of parameters in your model now?

See remember that this w vector is a vector of all the parameters, what am I telling you that some of these are going to disappear, when which condition holds, the third can, the third bullet holds, some of these are going to disappear. That means, the effective number of parameters, which remain in your model is going to be less right and you see that it is going to be given by this quantity, right?

So, that is sometimes known as the effective number of parameters in a neural network. If the effective number of parameters in your neural network is decreasing; that means,

what you are doing, making the model less complex right? So, that is what we have achieved, you see that ok.

(Refer Slide Time: 17:50)



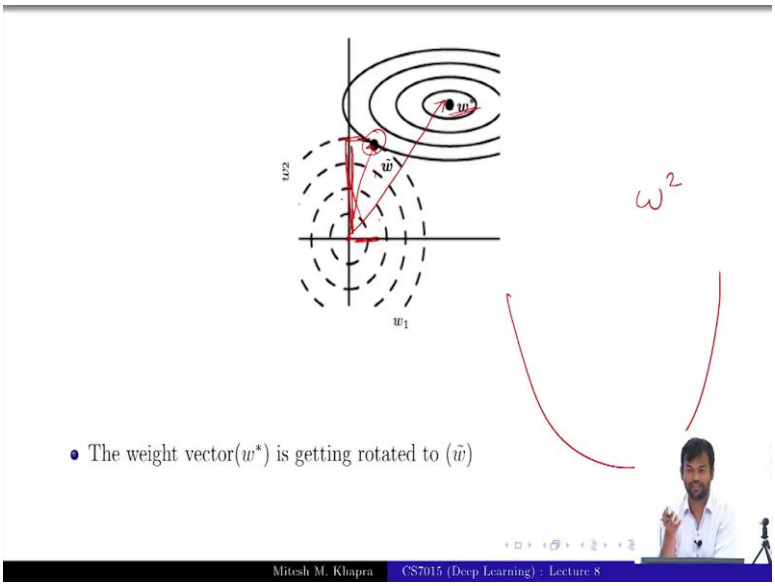
Now, let me end with a pictorial interpretation of this. You see two figures here and there is only one figure, but you see two different things here. Can you tell me what this is and what this is, that is the first question I want to ask you. The hint is that in this lecture we care about, the other hint is what was w star, the solution for the.

Student: (Refer Time: 18:23).

Unregulated loss which means which loss L theta, you need any more hints. Sorry, this box is the contours of L theta, this box contours of omega theta. So, this thing just ignore this part of the figure for now ok. This I have marked as w star, w star was the solution when I only had the un regularized loss ok. There is the solution when I had the un regularized loss ok.

So, remember the contour maps that we had seen. So, this is the minimum of that particular function. So, this is the contour map for L theta; that is clear. Now what probably is not clear is, why is this the contour map of omega theta, let me just go ahead actually.

(Refer Slide Time: 19:20)



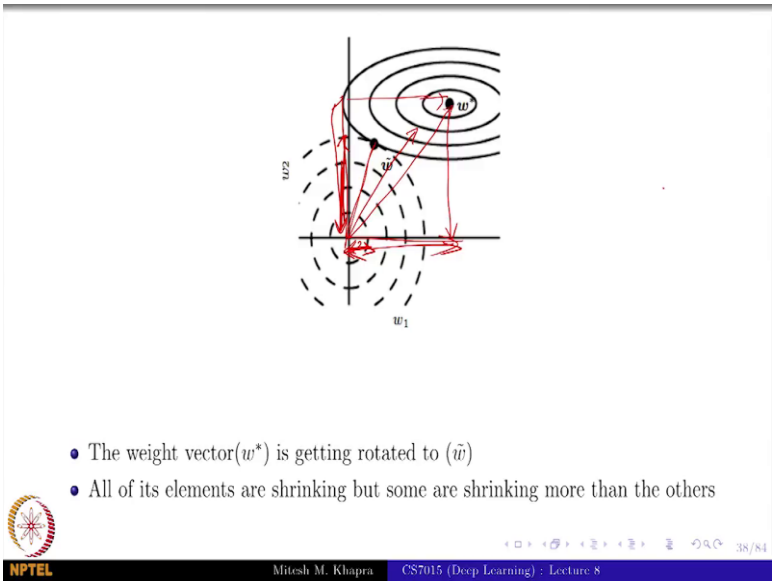
Please do not read this, this is the prestige ok. So, do not read that. So, this is the contour map of omega theta right, because omega theta in the, what is the minima for the omega theta, it is a function of the form w square, what is the minima? 0 and what does that function look like and what is this point, 0 the origin right? So, that is why this is the contour for omega theta ok.

Now, what is happening? This was the solution when you had without regularization and now this is w tilde which is a solution with regularization. So, can you make some commentary on this, with respect to not just general commentary, with respect to the things that we saw in the derivation. We talked about rotation, scaling, dimension specific scaling, so what is happening? This was my original solution vector; this was my original solution vector when I did not have the regularization term, now what has happened? The rotation has happened and we saw that there is a rotation operation happening, more importantly what has happened? Scaling has happened.

More importantly what has happened dimension specific scaling is happening right? One dimension has not, this dimension has scaled down this dimension has not scaled down enough; that is exactly what we wanted right? We wanted the less important weights to go down and the more important weights to stay there. We did not want a uniform scaling down; we wanted a dimension specific scaling down.

So, the weight vector has been rotated yes, each dimension after rotation has been scaled, some dimensions have been scaled down more, the other dimensions have been scaled down less, how many of you can make this interpretation from the figure, now that I have told you this interpretation.

(Refer Slide Time: 21:33)



Now, still if you do not how mean if you can still have a doubt with this, you still have a doubt what is doubt fine so. So, this was the original solution vector right? The map told us that what actually happens is when you add this omega theta the solution vector gets rotated ok, at the same time there is also some scaling down and that scaling down is for dimension.

How many dimensions do you have here? Two dimensions right? So, this is one dimension, this is the other dimension. Now in the original case both these weights actually seemed almost equal right? I mean if you look at the w 1 coordinate and the w 2 coordinate they were same. Now after this regularization what has happened is, what are the new coordinates for w 1 and w 2, this is the coordinate for w 1 right, this is the value of w 1 and this is the value for w 2.

Both of them are admittedly smaller than the original values for w 1 and w 2 in the absence of regularization or both of them equally smaller. No they are being scaled differently, one rate has been scaled down more, the other weight has been scaled down lesser right and that is exactly what the math was telling us that they get scaled in

proportion to those lambda 1 by lambda 1 plus alpha and that is exactly what we see in the figure is that fine.

How many if you get this interpretation now is that ok. So, all of its elements are shrink oh. You have a question. So, this final resultant right it is. So, what would have happened is that there would have been first rotation then scaling down and then again rotation. So, what you are seeing here is the final rotation right? So, it is not, it should have been showed in three steps by just shown the final step.

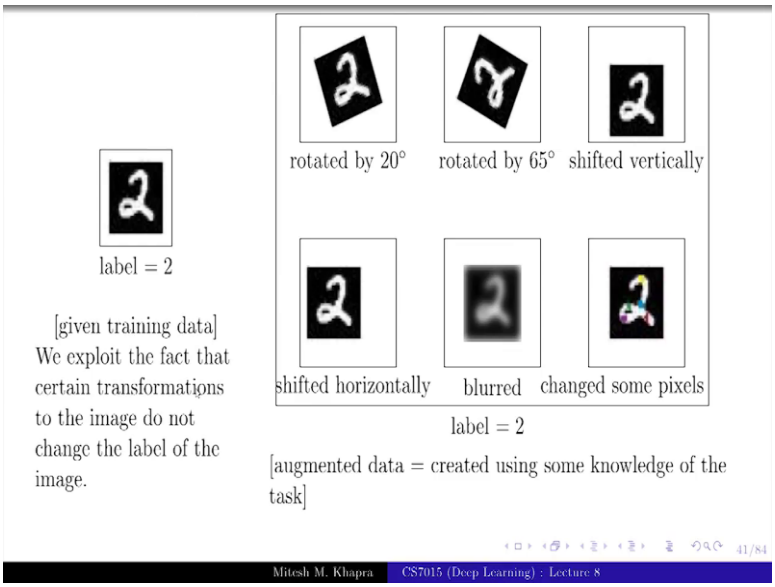
So, its question was that we first had a rotation, then had a scaling and then again a rotation, but I even as explained in the figure I spoke only about one rotation. So, I basically clubbed both the rotations, and so what you see finally is rotations scaling down and again rotation.

Deep Learning
Prof. Mitesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module - 8.5
Lecture – 08
Dataset augmentation

So, how with that heavy math I will just interline with this, something very simple, which is, something known as dataset augmentation.

(Refer Slide Time: 00:21)



So, what is dataset augmentation mean? So, you always given some training data. So, in the case of MNIST, you had this training data, where you are given these digits, images of digits and you wanted to train some classifier. So, in dataset augmentation, what we do is. So, now, we have, what is happening here right. Conceptual is that there some seeing, some training data and try to build a classifier and what you doing actually is minimizing the empirical train error.

That mean it will ensure that whatever you have seen in training is going to look. It is going to be perfectly classified. Whatever we have seen in training that is going to look very good. It is going to be the training error on, that is the error of those training examples, it is going to be very easy.

Now, my question is this; if a training time you are seeing all this 2's which are roughly vertically drawn right and a test time, you see at 2, which is written like this, which is slightly tilted what would happen? It will not be able to do a good job on that; that means, your model is not think of terms that you have used in this lecture, not generalizing.

Can you think of a simple trick based on your domain knowledge of how people right digits to kind of overcome overcome this problem you get the question right. I am telling you that it is possible that someone writes to in a very tilted manner, can you prepare for eventuality, eventuality the title of this module was dataset augmentation. So, what would happen is, are given some training data.

You can always generate for training data from that, see here is another training instant that I have created, I have just rotate it to 2 by some random angle. I took this image, I just rotate it and this is a simple operation that all pixels are moving by a certain angle, I could have rotate it more. I could have shifted it vertically; that means, in all my image the 2 was actually exactly at the centre. I just shifted at a bit vertically.

So, I am. So, think that you are reading, one of those KYC forms or bank forms, most people would write at the center of the block provided, but some people could write to the extreme right or extreme left right. So, you are preparing for that, they saying that ok. All my data, the digits are well written at the centre, but let me just shift them bit; so, that I can also deal with people who write it at the corner.

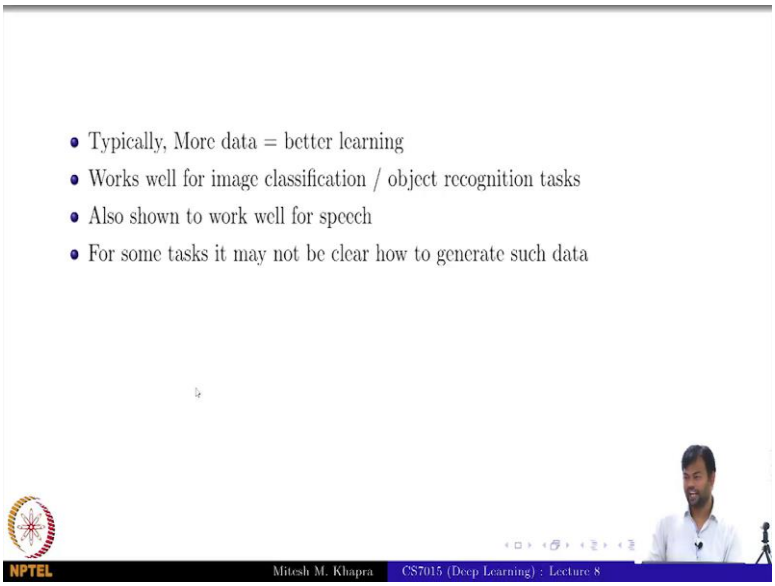
Left align or right align instead of center align. I could have even shift it horizontally. Most people would write at the center, but some people would write at the top or at the bottom. I could blur the image, but someone has taken a photo and send it to me and the photo is not very clear or I could just change, some pixels randomly right. I could add noise. All of this is dataset augmentation, with the hope that I am capturing with these variations. I am capturing enough variations in the data.

So, that I have a better chance of doing something better on the test data. Is that fine? This is all still training data, mind you, I am still going to compute the empherical train error. It is just that. Now, I have blown up my data, but much more than what I had initially do? You all see by doing this, you could have done better on the MNIST assignment, you could have done better again, I am not asking you to do this.

So, now I will do this then I will have supervised data, because I know that by this small variations, the label is not going to change and what am I using, there I am using my domain knowledge right. I cannot do this always right. I hope you appreciated that suppose, that changes the domain a bit and I am given images of defects of motor parts right, where I have taken a image and there is a black spot somewhere, which indicates, defect I cannot go about doing the same thing, there I cannot change some other pixels. It will just means that the defects is at a different location right, but in many cases you can do that.

So, if you are given picture, because of dogs and cats, because the entire world case about classifying cats and dogs then you could do some rotations, you could blur them a bit, you could occlude certain questions of the picture and so on and still generate training data right. And what you are trying to do is, trying to take care of cases that you would end up dealing at test (Refer Time: 04:27) right. Is that clear ok, and please be aware that we are exploiting, some domain knowledge here.

(Refer Slide Time: 04:32)



Typically, more data is better learning, works well for image classification in object recognition. These are the task, where this is already been tried out and they have shown to work very well in these tasks also shown to work well for speech, where the people have some speech training data. They try to augment it for some task; it may not be very easy to generate such data right?

So, you could think of various NLP applications, whereas given you a data document right, because always do what Joe does in that Friends episode. Do you remember what I am talking you, see what I am talking about. See you wants to write a recommendation letter for Monika and Chandler ok, and he has a letter written, any replaces, every word by it's best synonyms from the thesaurus (Refer Time: 5:20) right, that says a way of generating noisy data and in that case, it was actually noisy right? So, you could think of doing here, but as happened in that case, it will not result in very good transformation.

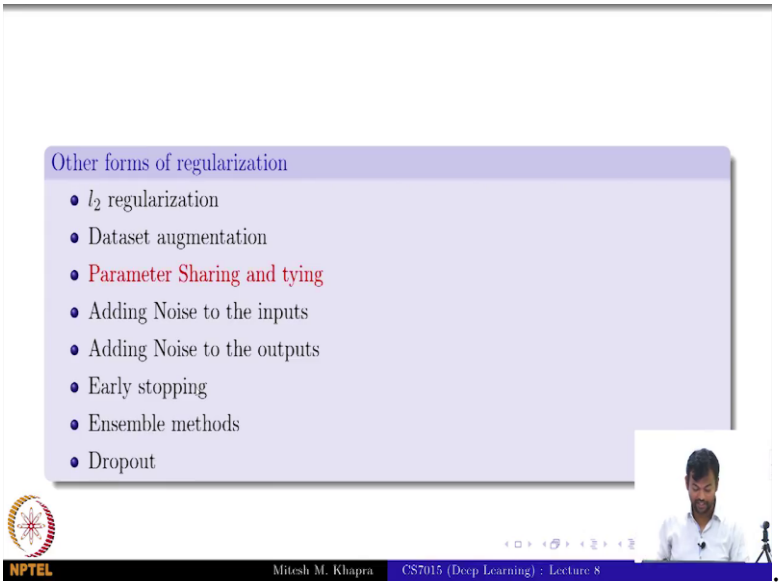
Next, for example, I remember something right? They are very warm hearted people got translated as they have some warm cardiograph or something like that which do not make sense. So, it is not very easy, in almost all application should do it, but in some applications, typically in vision application, this is easy to do and you would gain a lot by doing this right?

Deep Learning
Prof. Mitesh M. Khapra
Indian Institute of Technology, Madras
Department of Computer Science and Engineering

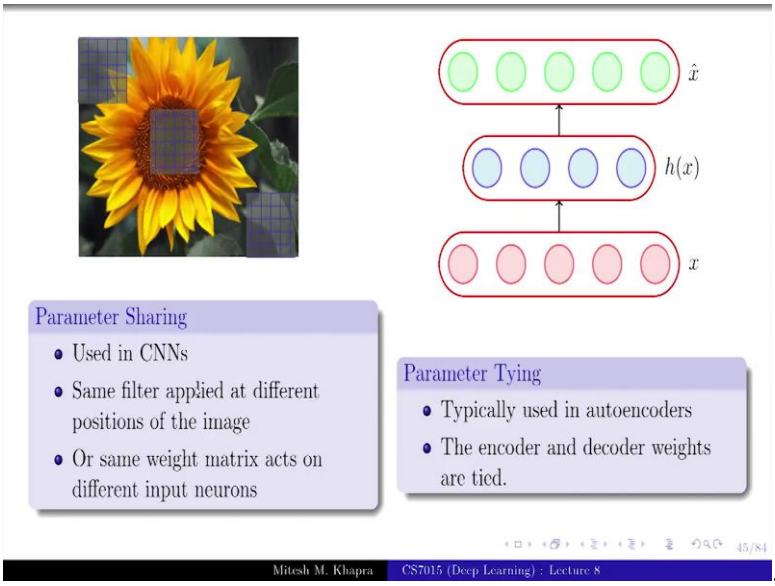
Module - 8.6
Lecture - 08
Parameter Sharing and Tying

The next thing that I would like to talk about, and this quickly go over this Parameter Sharing and Tying.

(Refer Slide Time: 00:19)



(Refer Slide Time: 00:21)



So, parameter sharing and tying; I will just quickly go on this because for the sake of completeness it is there in this lecture, but it should, it would really make sense when I do convolutional neural networks. So, for the time being just take my word for it that in convolutional neural networks, you do a lot of parameter sharing where as the other place that you have seen parameter tying. So, that is again something that I am not going to talk about.

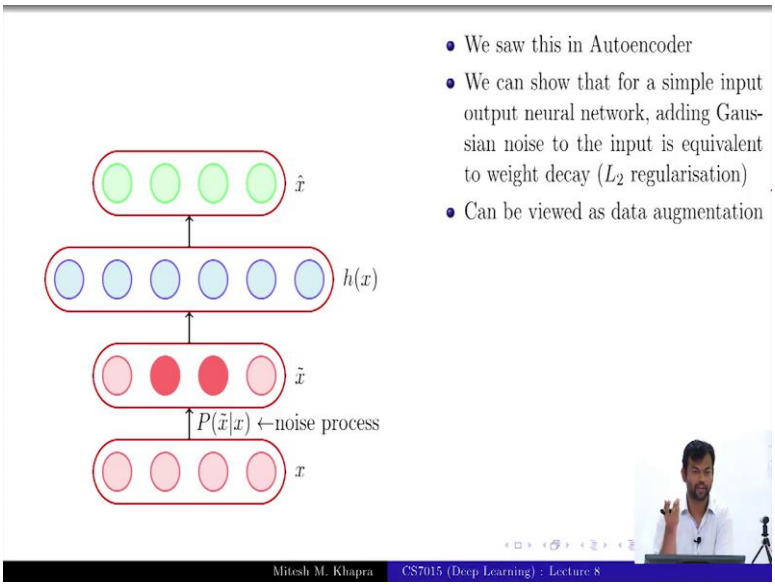
So, this is typically used in auto encoders, where the encoder and decoder weights are shared and that effectively reduces the number of parameters in the model which effectively reduces the complexity on the model. If the complexity of the model goes down, omega theta goes down because that is what which wise man told us that time, Steins Lemma.

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 8.7
Lecture – 08
Adding Noise to the Inputs

We go down the next module, which is adding noise to the inputs right?

(Refer Slide Time: 00:17)



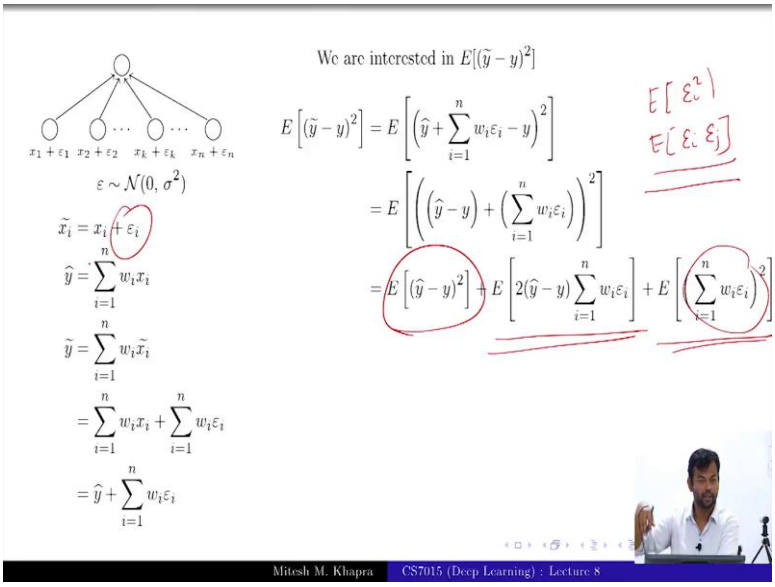
So, we have some kind of a noise process and now can you relate that how that was related to regularization, that was exactly the motivation in that case that we could have an over complete auto encoder which is a very complex model, because it has a large number of parameters.

And to avoid that we were adding this noise to the inputs so that even if it tries to minimize the training error, it is not actually minimizing the true training error right? Because you have fed some noise to it everyone gets this, right? Ok. Now, actually we can show that for a simple input output neural network right; that means, you do not have any hidden layer you just have a set of inputs and you have the output layer. Then adding noise to the input or rather adding Gaussian noise to the input, it is equivalent to weight decay.

So this can also be viewed; so, we will do this part right. So, we will just quickly do a small derivation, where we show that adding Gaussian noise to the inputs is the same as doing a L2 regularization, that is a very neat idea. So, this can also be viewed as data augmentation right. Exactly what I shown on the previous slide you added 2 you just corrupted some inputs of it that is the same as adding noise to the data.

So, the essentially augmenting the data right, you have some training data and just augmenting it. So, to get more training data is that fine.

(Refer Slide Time: 01:41)



Now, about this smallest derivation this is again just a set of steps, I will go over it reasonably fast. I will give you the set up and then it is quickly work through the derivation right.

So, what I was trying to say is that if you have a simple input output neural network; that means, you just have inputs and the output you do not have a hidden layer, right? Then adding a Gaussian noise to the input units where the noise comes from this distribution; it is a Gaussian distribution 0 mean. I want to show that doing this is effectiveness the same as doing L2 regularization.

Now, again see this is the same thing squared eggs in vacuum because this is not the kind of networks that we deal with, but it is good to see what happens at least in these neat conditions, because we will never have a simple input output network, at least not in this

course. We will have a deep neural network always. So, but at least see what happens in the simple case right. So, what we are doing is from the x i's we are creating a noisy x i by just adding some epsilon noise to that. And what is our model going to be? It is just an aggregation of all the inputs ok? So, this is what our original model would have been without the noise fine.

I would have just aggregated all the inputs, I am assuming there is no non-linearity at the output and I am just taking y i is equal to summation of all my inputs everyone fine with this side. Or this is too simple for you guys to understand because we have been doing a lot of deep neural networks. So, suddenly one-layer network I do not know what it is everyone gets it, right?

And instead of y hat, now I have y tilde because instead of x i I have x i tilde ok, but what is x i tilde x i plus epsilon i right? So, I can write it as this just fine. So, actually y tilde is nothing but y hat plus some quantity, ok. What are we interested in? Always this quantity the expected mean square error ok? I mean expected squared error and why not y hat?

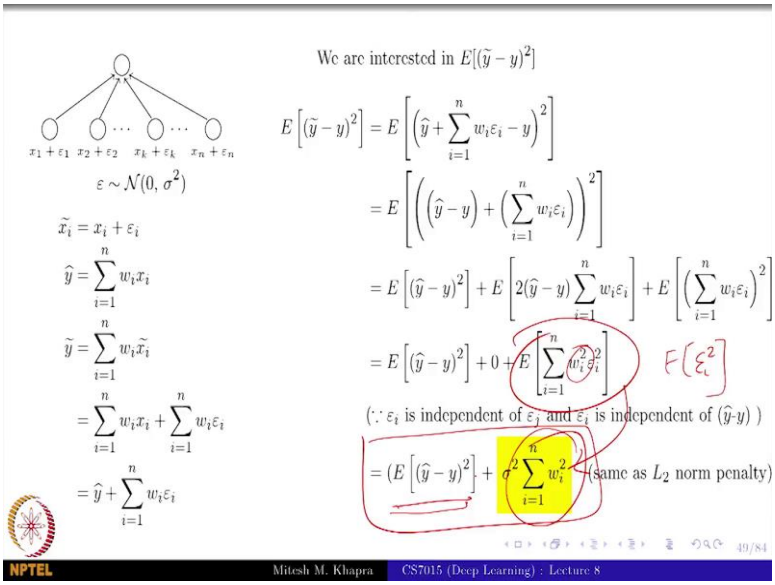
So, we have added noise to the input. So now, y tilde are the outputs that we are going to tilde. So, let us see what that quantity is; and again just going to be some simple stuff. So, I replaced y tilde by this that we just derived on the right hand side, on the left hand side ok. So, I am going to take these 2 terms together; so, I can write it as this plus this the whole square fine. And I am going to keep this as it is. What is this quantity? The original squared error expected squared error right, when I was not adding noise to the inputs ok. and you see how we got these 2 quantities, this is just a plus b the whole square is equal to whatever it is equal to right? Now let us look at the last term this is a square of a sum right?

So, what kind of terms would you have inside? You will have some terms which are epsilon i squares and you would have some terms which were epsilon i epsilon j right ok. So, we will have some expectations which are going to be something into epsilon i square, and some expectations which are going to be epsilon i, epsilon j. Everyone gets this? Some terms there; now which of these terms would disappear?

Student: (Refer Time: 05:09).

These terms right, why? Because the noises are independent ok. I am not if I have drawn a noise for one instance; it does not have any influence on the noise that I am going to add to the next instance. If I have taken one x i, corrupted it with some noise there is no bearing on the noise that I am going to use for the next epsilon i right? All these features are the noise added to the features are independent.

(Refer Slide Time: 05:38)



So now from these terms only the square terms are going to remain, is that fine? And similarly this quantity, what can you say about this? We just did something similar, why I am a saying that this is going to 0? Again I can show that this is the covariance between this random variable and this random variable ok? And now are these 2 random variables dependent? What is epsilon i? The noise that I am adding to the input, does it have any effect on y hat no, right? Because y hat does not depend on the noise, what is y? True output does it have anything to do with the noise? No right.

So, that is why these 2 random variables are independent. So, I can again write there the expectation of their product as a product of expectations, and then the expectation of this is going to be 0, because epsilon i was drawn from a 0 mean distribution is that fine everyone gets that the same trickery that we did earlier. So, this is the quantity that we are left with, you see how I got from here to here. This is an expectation of a sum, which is equal to a sum of expectations, w i has nothing to do with it is not a random variable.

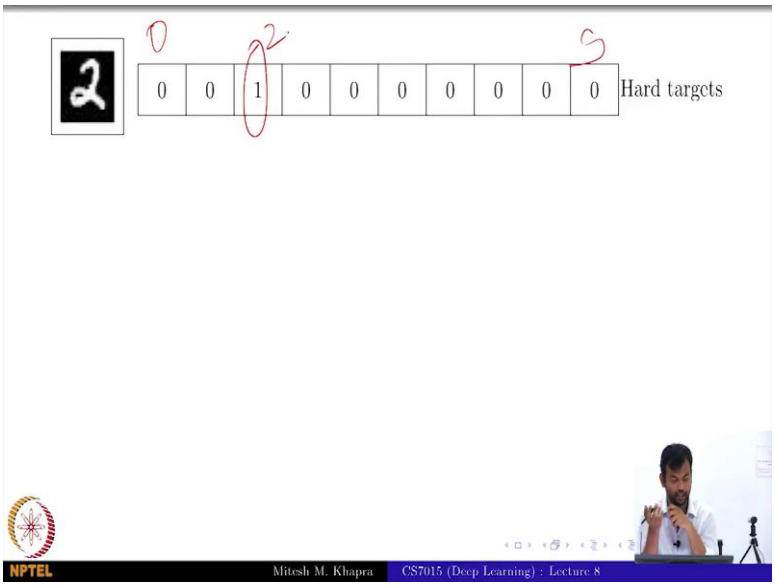
So, it is just the expectation of sigma i square which is nothing but the variance right. So, I get this what does this look like? I already told you the answer before starting right, this looks like L 2 regularization this is the true error, I mean this is the empirical estimate from the training error. And this is the weight decay term everyone get this? How you see that this is an equivalent thing? So, at least in this neat set up you get the intuition that adding noise to the inputs is a same as adding a L2 regularization term.

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

Module – 8.8
Lecture – 08
Adding Noise to the Outputs

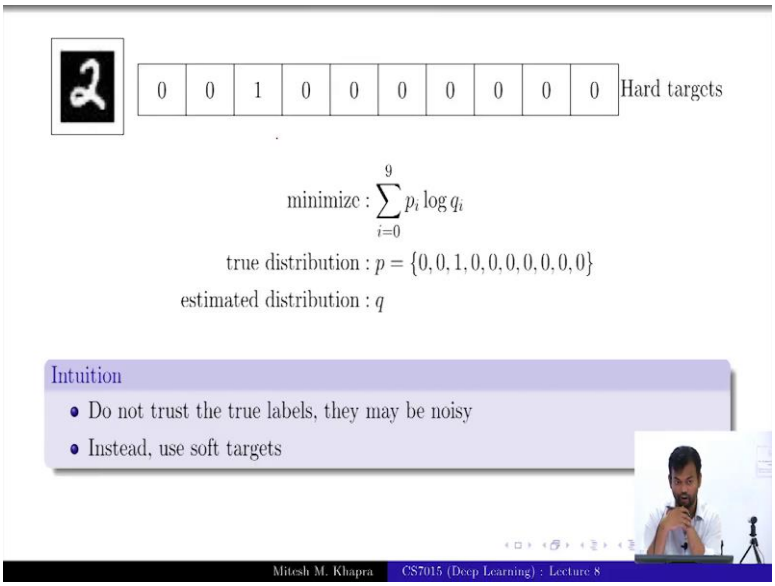
So now going on to the next module which is Adding Noise to the Outputs.

(Refer Slide Time: 00:19)



So, here when you are given some training data, this is the label vector that you have been given, right? Where one of these elements is 1; so, these are like 0 to 9 8, where which digit it is and in this case it happens to be digit 2. So, that element is 1 right that is the true training data given to you.

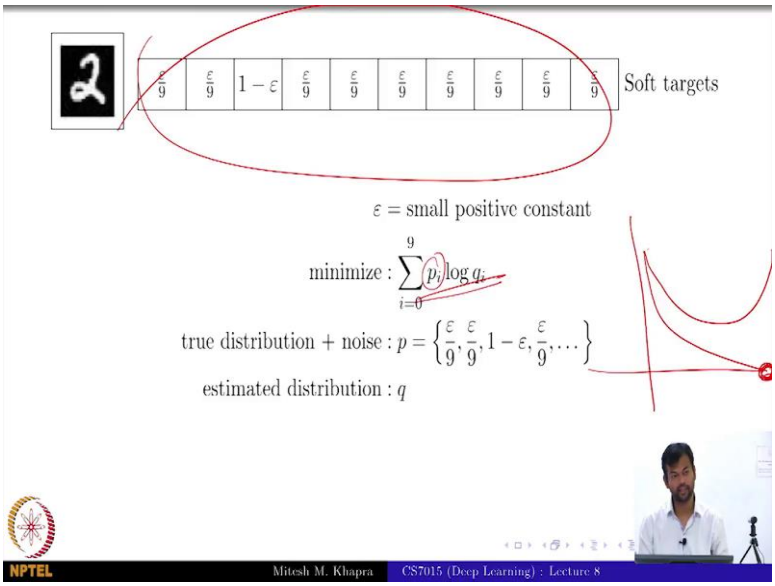
(Refer Slide Time: 00:40)



So, what you could do is actually and actually what you try to do is, minimize this quantity p i log q i, where what is p i? p i is the vector which was given and, what is q i? The predicted probabilities ok. So now, when you try to add noise to the output, what you actually do is, you see that I do not trust the true labels, they may be noisy.

Whatever data you have given to me that is one way of looking at it. That I do not trust it, I will just say that it is noisy. The other way of looking at it is that in some way I am ensuring that; I do not try to over fit to this label, right? Because now my true whatever I am trying to optimize, let me just go to that and let us see. So, instead what we will do is we will use soft targets.

(Refer Slide Time: 01:19)



So, this is what i mean by soft target. Assume that there was some epsilon noise in your labels; so, instead of treating this as 1 and all 0s. Treat the true label as 1 minus epsilon and divide that among the remaining 9 entities right that probability mass divided among the remaining 9 entities.

So now when you are trying to minimize this, what is p i? This soft distribution right and q i is the predicted distribution. So, you see why this acts as a regularization, why does it act as a regularization? What is the aim of regularization? Do not over fit on the training data right. To over fit on the training data, what should it have done? It should have treated only the correct label. Now if I am giving it this information then I am not allowing it to over fit on the training data right?

Because now with this distribution, this quantity will not get minimized, when q i is equal to the 1-hour distribution where all the masses on 2. Do you get that? So, in some sense we are making sure that, now if it tries to over fit on the training data; it will not get the minimized error right. So, you have this corrupted the outputs of it everyone gets this, is ok? The trainer no that is the whole point

Student: (Refer Time: 02:40).

No.

So, that is thing right? So, some of these are heuristics based. So now, we have started with this whole derivation, where we try to show the relations between trainer error tested or not, but things that we have seen some of these things right, even whatever unfortunately, I tried to prove on the previous slide the weight decay thing; even that is only for these neat networks where you do not have any hidden layer and so on right.

So, most of these are just heuristics, you are just saying that the principle is that; you will not allow the true training error as computed from the training data to go to 0. If you do that you know that you are going to over fit. So, try whatever you can to avoid that ok. That is the idea, do you agree that doing this is going in that direction?

Student: (Refer Time: 03:25).

Training data. The hope is that if you do not do that then it will not under fit on the test it right?

There is no I mean I have you are you looking for a proof, where I say that doing this we will ensure that a training error does not go to 0, but the test error comes close to the training error. There is no such proof right, just a heuristic. It is going by the principle that if I do not allow the training error to go to 0. Then hopefully I will over fit, I will not over it as much as I would have otherwise right?

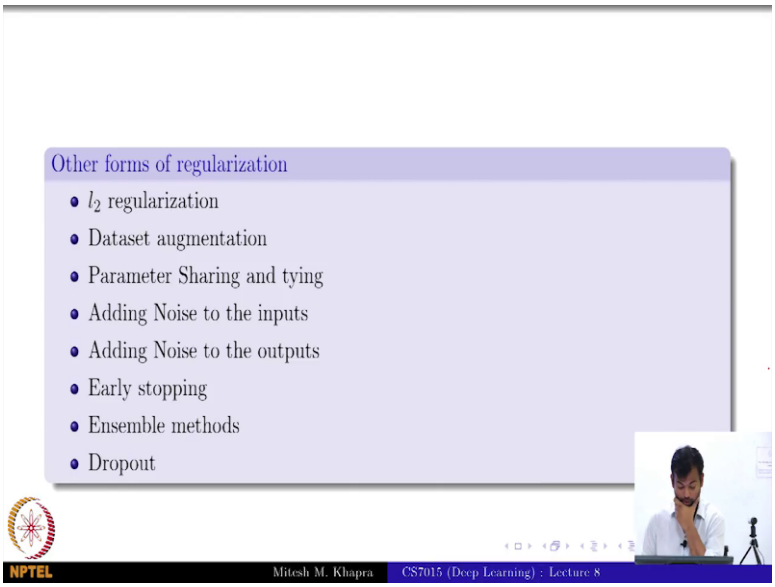
So, that you can think of it as this way right; so, this is the curve that you are seeing it. This was a training curve; this was your test curve. You are preventing from entering this region where the error is 0; that means, you will end up somewhere here right? And you know that that is a more preferred point as compared to this. That is the intuition that you are going right, is that?

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

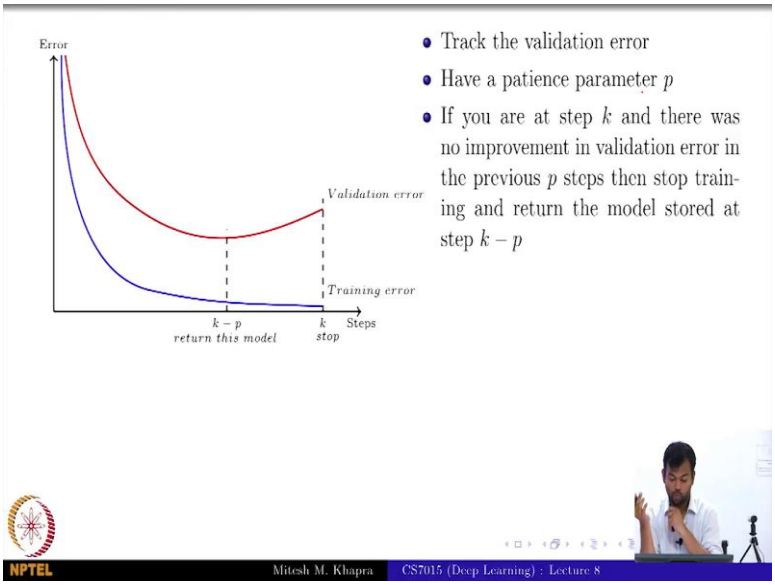
Module – 8.9
Lecture – 08
Early Stopping

I will do, will do early stopping where again we will get into some of these eigenvector analysis. So, let us see that.

(Refer Slide Time: 00:18)



(Refer Slide Time: 00:21)



So, the idea been early stopping is actually very simple in principle what needs to be done. So we know that, this that this trend exists between the training error and the tester right. So in practice, what you will do is you will continue to optimize the training error; the empirical training error which is the sum of the errors on the m training points.

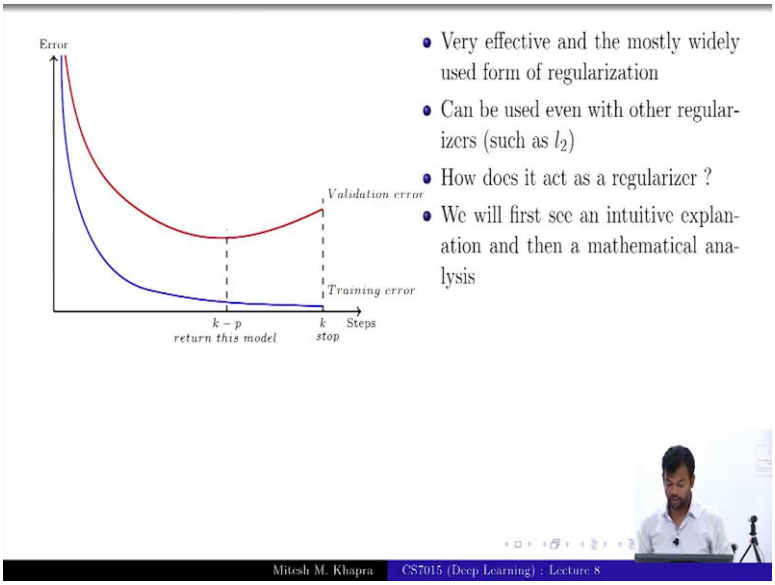
You will also continuously keep track of the validation error; that means, the same quantity you will compute over the n validation or test points. Everyone get this, you can do this and you are actually doing this in your back propagation assignment. Keeping track of the training error as well as the validation error and you keep plotting them, ok. I will keep running for various epochs and keep something known as a patients parameter p.

So, if you are at the 20th epoch and if your patients parameter p is equal to phi; And just do a check whether, in the phi last phi epochs has my validation error ever gone down or it has been staying the same or has it been increasing ok. Now I will give you a condition that it was either staying the same or it was actually increasing is this good or bad.

What does it tell you while your training error was of course, decreasing may the more you train your training error will keep going down? So, what does this tell it is just over fitting you are fitting the training error you are just making it 0 or as close to 0 as possible, but that is not helping your validation error. So, the validation error is either worst case increasing or remaining the same right.

So, this is a very commonly used trick which is known as early stopping you keep this passions patients parameter. And you make sure that if you have cross this patients right and the patients here is that I was waiting for the validation error to go down, but it is not going down for some p epochs; So, no point in continuing training anymore I will just stop it does not make sense.

(Refer Slide Time: 02:21)



So, and this can also be used in conjunction with other regularizers, right? So, in the quiz also we had this question sorry for bringing up the quiz, but we also at this question where you have the sparsity regularization and I was asking whether I can add the L2 regularization along with it. So, these regulations can be added or used in conjunction it is not that you can only use one of them.

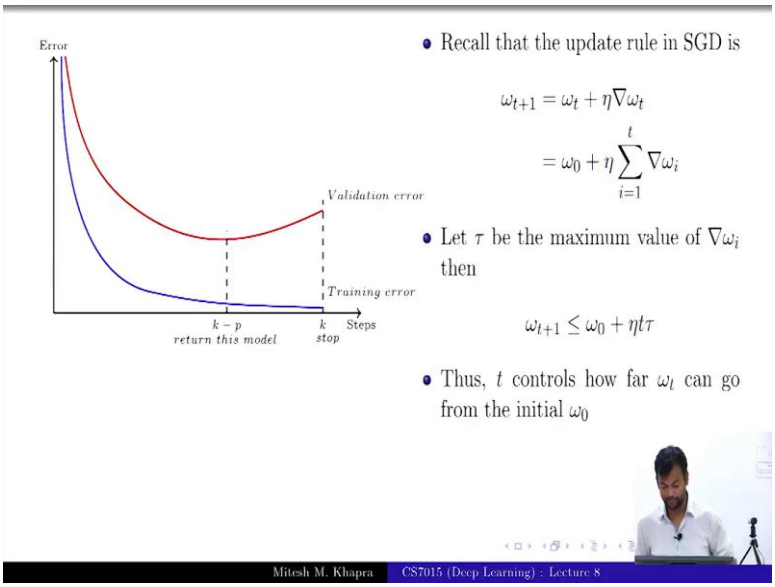
So, early stopping is a way of regularizing, but you could also use it in conjunction with L2 regularization or any other regularization technique that you do not want right so, but how does this act as a regularizer from the picture? It is probably clear and is the same as the explanation. I was trying to give to his question right? That you are preventing yourself from entering in these regions and trying to enter into more favorable stop at more favorable regions.

But can you think of slightly more in terms of, what happens in gradient? And what would happen if you stopped it early and so on. Can you try it to connect it to the update rule of gradient descent, what happens as you keep doing it for more and more epoch?

No gradient descent has nothing to do with validation error or backtracking error gradient descent only works on the training data let us think in those terms.

Gradient star diminishing to 0; so, what happens how does gradient descent progress? Where do you start? I started a random point at every epoch which is a collection of iterations right, or you go or many training points. What happens to this, I start moving. I keep moving now, if I fix the number of epochs or do not allow it to change any more after a number of epochs. What am I doing? I am restricting the boundary around the weight right. I am not allowing it to grow beyond a certain boundary do you get that.

(Refer Slide Time: 04:13)



Let us see that. So, we will first see an intuitive explanation and then, go to a more mathematical analysis are update. So, the update rule for gradient descent is, I always make this mistake, this has to be minus oh the t H have disappeared ok is there; so, sorry other to have disappear.

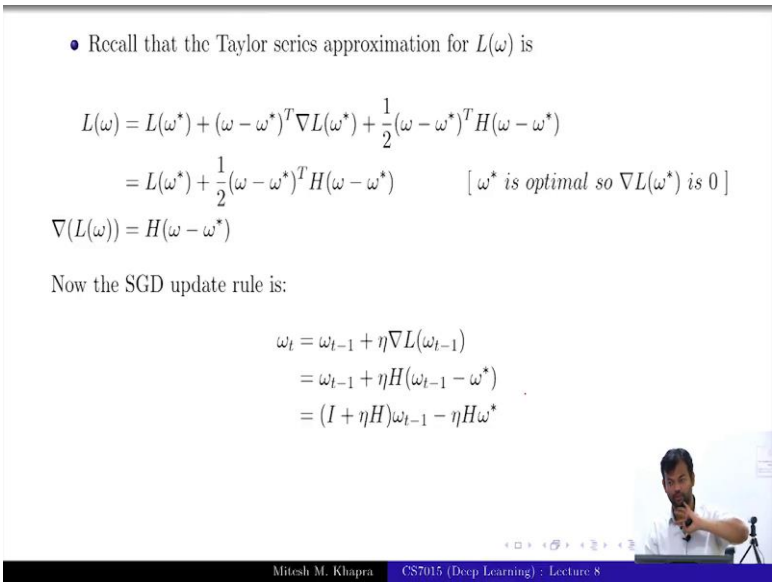
So now, what would actually happen at the t H step is we have w naught 3 plus or minus does not matter. It just tells you that, how much it is going to change? This is what is happening actually at the t H step right? You have just subtracted all the previous derivatives that you had so far right. From where you started off now, you are looking at t steps. So, at every point you are computing a certain gradient, but had a certain magnitude.

Now, let me say that across all these steps, the maximum gradient that you had. I will just call it by tau right? So; that means, in this summation there are t terms, I am saying the maximum of those was tau that was the maximum rate gradient that I got at any one point.

Now, what I am going to do after this? I am going to replace this by something. This summation is always going to be less than or equal to this right. Because, I am assuming that each of my steps is less than tau, there are t such steps. So, I could have at matched moved t into tau right, but I would have moved less than that, because tau was the maximum gradient that I had.

So, this is going to be less than equal to is that do you get the change from the equality to less than equal to ok. So now, what am I restricting actually in early stopping, what is being restricted? There are only so many symbols there I just speak one t tau is of course, not in your hands w naught is not in your hands w. So, t is the one right. So, I am only allowing that many updates so; that means, from w naught you can only moves that much this looks. You see that analogy that, this is something similar to you not allowing the weights to really grow a lot.

(Refer Slide Time: 06:27)



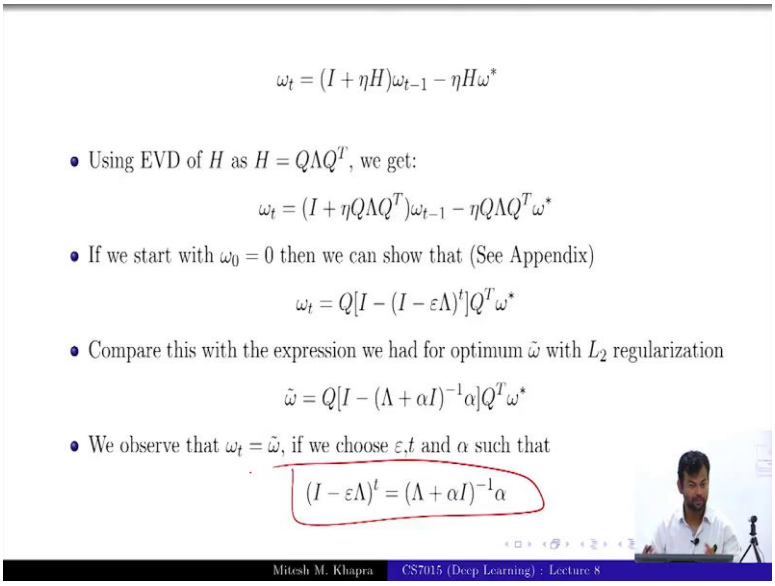
So now, but will not end here you will. Of course, do some more stuff on this right ok. So, we now see a mathematical analysis of this. So, recall that a Taylor series approximation for L w is the following. The same thing which I wrote a few slides back

or many slides back everyone remembers this right. And now again I am going to do the same thing that, if I know the optimal w star then the gradient at that point is going to be 0.

So, this term disappears, and now if I take the derivative, this is what will remain. This is exactly what we did earlier also right. So, we will have derivative of this and derivative of this. So, the derivative of this quantity is just this and the derivative of this is 0. Because, that is exactly what we started off with right that w star is the optimal solution.

Now, SGD update rule is the following ok, which I can write as this. I just replaced this by this ok, I am just rearranging some terms is that ok. How many if you are fine with this? How many feels to tired to even care about this?

(Refer Slide Time: 07:32)



So, this is what w t would be this is again some simple steps leading to some conclusion. The conclusion is, what matters the steps are very easy you can go back and look at them right. So again, I will use the EVD the same trick that I did earlier and it will give me this instead of H ok. Again I will just do some rearrangements and actually, I can show that if I start with w naught equal to 0, then w 2 is actually given by this quantity ok. And there is a proof of this in the appendix you can go and look at it.

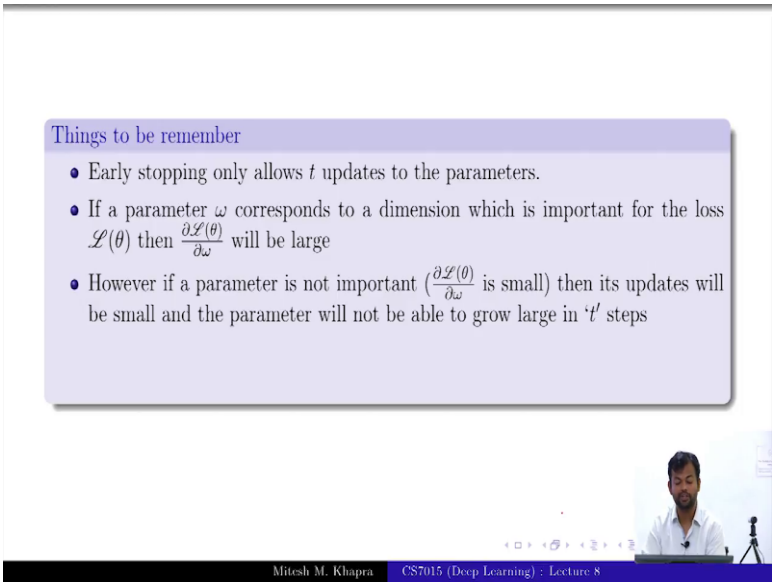
Now, what does this look similar to rotation diagonal rotation exactly similar to the analysis that we did for L2 regularization right. And in fact, if you can you can show

that, if we compare this expression with the while we had for L2 regularization. And this is the expression that we had for L2 regularization right. Rotation some scaling and then again rotation right; Then we can show that, early stopping is actually equivalent to L2 regularization, if the following condition is satisfied.

This does not mean much because, god knows how you will satisfy this condition? Right, but all it is saying is that there is some equivalence. At under certain conditions and that is what is the intuition was also telling us that, it is somehow preventing the weights from going large and it is doing this; In this very convoluted way where this condition holds for it to be equivalent to L2 regularization.

As I said for you and me is going to be very hard to create this condition right. How do I make sure that, something like this is true right, but that does not matter what matters is that, there is some equivalence between them?

(Refer Slide Time: 09:34)



So, when you are doing early stopping. It is not just a heuristic or a blind thing that you are doing. You know that, it is somehow related to L2 regularization. Hence, that you are doing it and hence it also works in practice is it fine, we will that work for all of you ok right? So, the things to remember is that early stopping only allows t updates to the parameters. This is the important thing rights? So, now, if a parameter w corresponds to a dimension which is important for the loss. Then what would this quantity be the partial

derivative of the loss with respect to that parameter it is going to be, if there is a parameter.

For example, let us take the Amir Khan an example right. That whatever weight you gives to whether, the actor was Amir Khan or not. If that is very important because, if that feature is on you are lost completely changes and so on right. If you do not learn the weight correctly that feature is very sensitive.

So, for important features the loss would be very sensitive to the changes. In the weights of these features, is that intuition correct right? That means, this gradient would be large ok and if a parameter corresponds to a feature which is not important. What would this derivative be small now, what is the net effect of this you have some parameter which are important. So, the derivatives are large some parameters which are not important.

So, the derivatives are going to be small and you are going to only allow t updates. So, what is going to happen? The parameters which are important; we will end up getting effectively more updates right; because, each of these magnitudes was higher and you did t of those. The parameters which are not important we will end up getting effectively lesser movement.

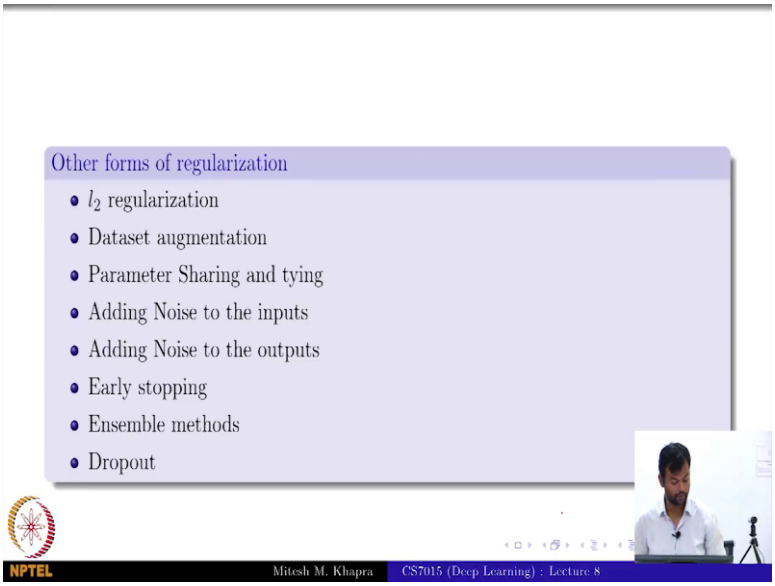
Because, each of these gradients were small and you did only t of those right? So, you again see this that, it is a weird way of ensuring that your important parameters get more updates than your non important parameters right? So, it is very important to see these connections between these different regularization methods, all of you are fine with this fine.

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

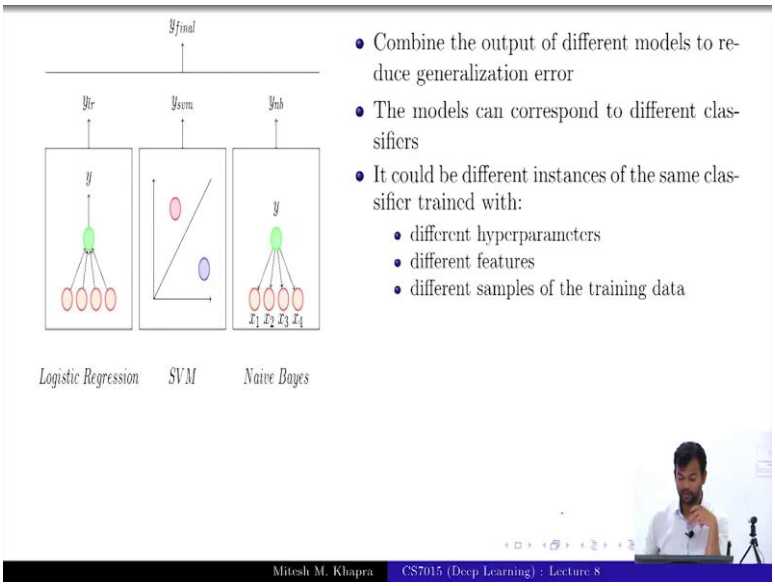
Module – 8.10
Lecture – 08
Ensemble methods

So next we look at on Ensemble methods, and this is just to build the intuitions for something known as dropout which is very popular technique in deep neural networks and convolution neural networks and even recurrent neural networks.

(Refer Slide Time: 00:23)



.(Refer Slide Time: 00:27)

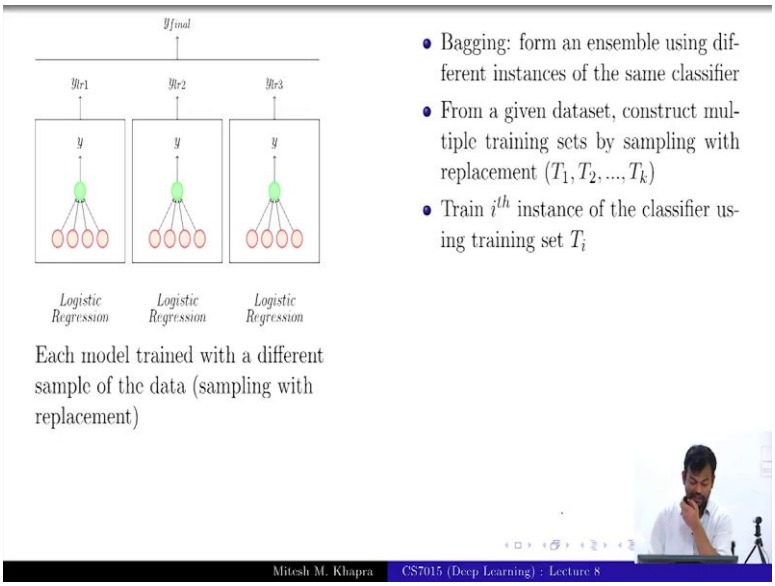


So, how many you have seen ensembles before? Seen it in machine learning ensemble was not done in machinery done with ok. Ravi did it. So, as a combine so the ensemble is essentially just the combining the output of different models, to reduce the generalization error right. Why does that make sense? Have these different models all of these would have different biases and variances right?

So now you are combining them. So, I will end up with a better thing on the test error right? So, that is the idea behind ensemble, now the models could correspond to different classifiers right for example, here I have a Logistic Regression and SVM and a Naive Bayes. I have trained them independently using the same data or different subsets of the data. And a test time I am taking a prediction from all of them and then, taking an ensemble of those predictions that is the basic idea.

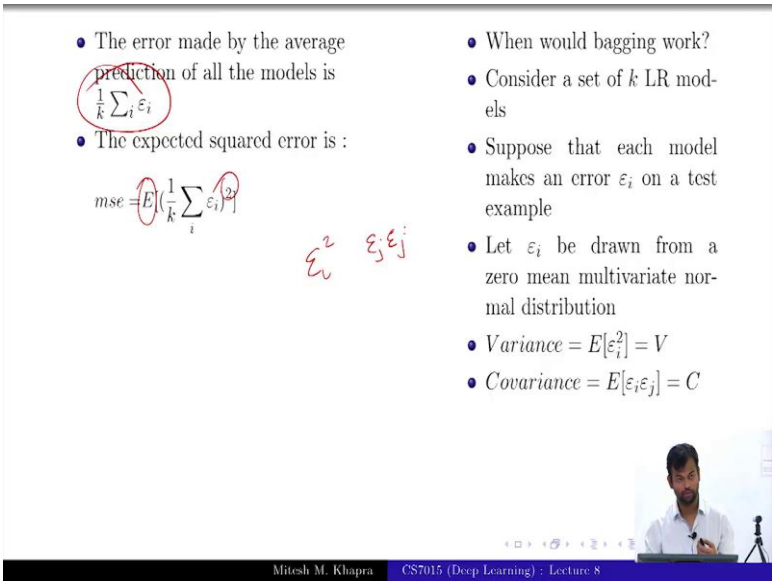
Now it could be different instances of the same classifier, trained with different hyper parameters. I could have the same neural network a 3-layer neural network, but trained with different hyper parameters. So, the hyper parameters could be learning rate, it could be batch size, it could be the number of neurons in each layer and so on right. So, it could be same classifier, but different hyper parameters, different features right. So, instead of looking at all the 100 features that i have given, I could train these classifiers with different subsets of the features ok or different samples of the training data.

(Refer Slide Time: 01:46)



So, bagging is one such ensemble method where you have different instances of the same classifier, which are trained on different samples of the training data ok. So, I have one classifiers trained on a subset T 1 of the training data another classifier trained, on a subset T 2 of the training data and so on right and so, each of these model is trained with a different sample of the data.

(Refer Slide Time: 02:12)



Now, when would bagging actually work, what would you want these classifiers to be? So, each classifier is going to make certain errors.

What do you want these errors across classifiers to be dependent, independent?

Student: Independent.

Independent right; so, if one classifier makes the errors on certain test instances, other classifier makes errors on a different set of test instances and the third classifier makes errors on a very different set of instances, that is the condition that you are looking for right? There is errors if all of them make error on the same instance then all of them are collectively going to make an error on the final prediction also right?

Because it is like I asked 3 guys all of them gave me the wrong answer. So, my final answer is going to be wrong, but at least 2 of these 3 guys gave me the correct answer then my final answer is going to be correct right? So; that means, the errors that these models make, I want these errors to be independent if I treat error as a random variable I want these errors to be independent.

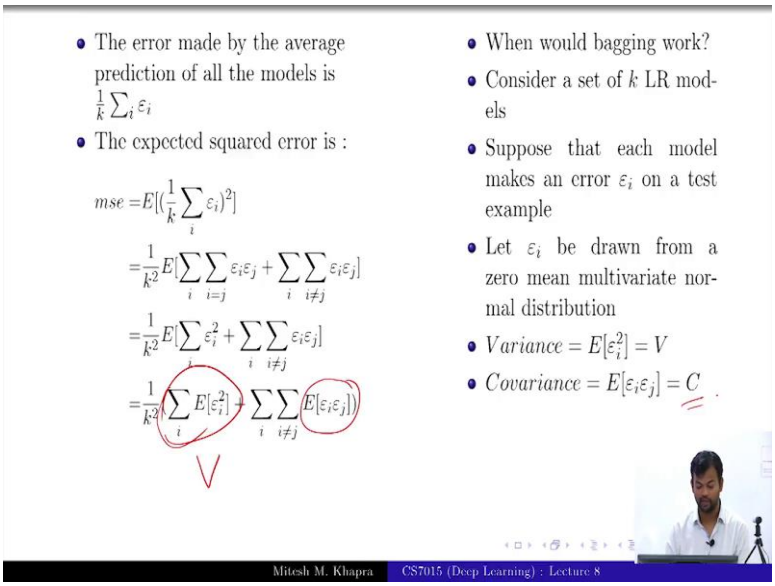
So, so consider a set of k such logistic regression models, suppose that each model makes an error epsilon i on the test example; now let epsilon i be drawn from a 0 mean multivariate normal distribution. So, the variance is equal to V; and how many such epsilons do i have? How many such distributions I am considering?

Student: k.

K right because for each classifier there is a distribution. So, then I can compute the covariance between these random variables ok. I will add that let that covariance be C. Is that fine?ok Now the true the error made by the average prediction of all the models is going to be given by this model one made an error of epsilon 1, model 2 made an error of epsilon 2.

So, the average error is going to be given by this. Now what is this expected squared error? This is the error; this is the expectation this is the square. That is the expected squared error is that fine? Again this is a square of a sum; so, it will lead to a lot of terms of the form epsilon i squares and what will happen now which terms will go to 0.

(Refer Slide Time: 04:23)

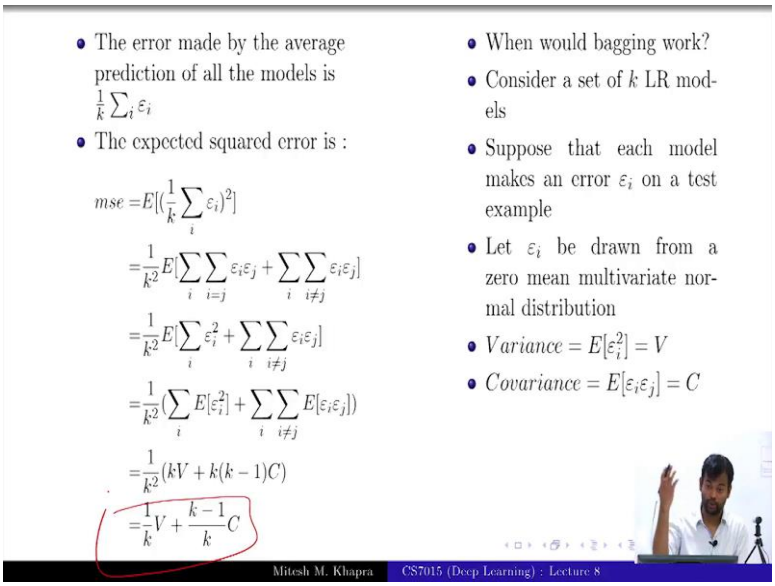


The terms having epsilon i epsilon j again the same thing they are independent. So, I can write the expectation of a product as the product of expectations and those expectations are 0. So, this is what it is going to look like, what is this? Oh sorry actually we had not assumed that the covalence.

What is this right? And what is this? Covariance I am sorry I have not we had assumed that there is some covariance said wed not assume they are independent right. We would want it to be independent, but in the general case we will assume some covariance and then I will show you the special case where they are independent.

So, then how many Vs do i have here? K right, and how many Cs do i have here?

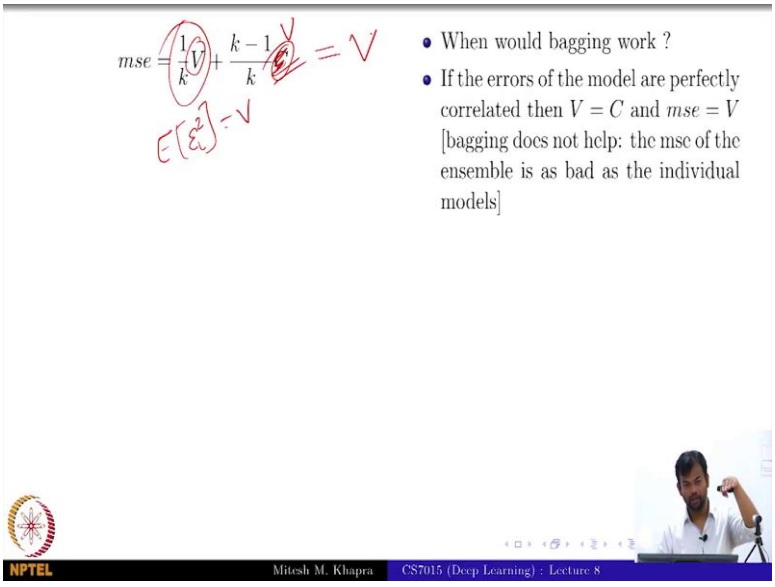
(Refer Slide Time: 05:08)



This summation is k into k minus 1 right or i equal to 1 to k and j equal to i plus 1 to k fine. And so, this is what it looks like now can you make some inferences from this equation, this is what the expected mean square error is going to be. Now think in terms of variance, covariance and tell me when would this be beneficial. I have already told you the answer, if the errors are independent what would covariance be? 0 right?

So, then what is the mean square error 1 by k 1 by k into V right so; that means, bagging would work when your classifiers the k classifiers that you are combining.

(Refer Slide Time: 05:33)

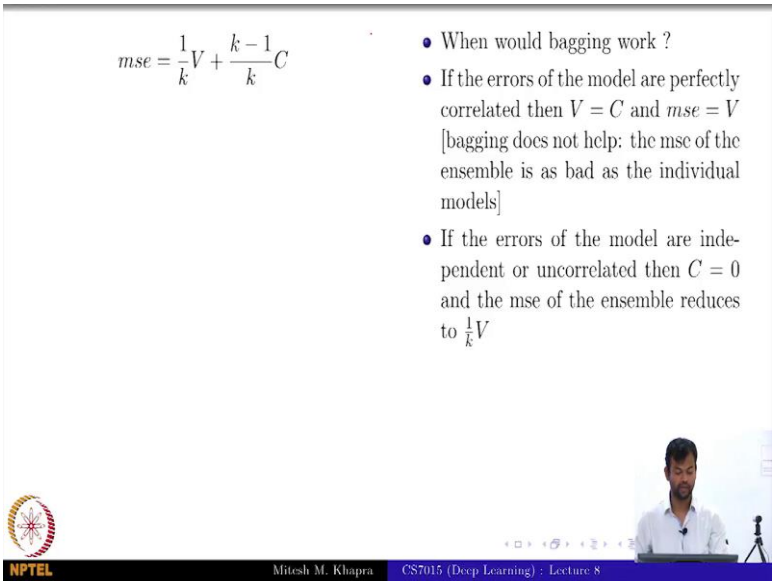


If the errors are independent, then the mean square error should actually have been V right, for a single classifier it was V right because mean square error is nothing but the expectation of the error expectation of epsilon i square which is nothing but V.

But if you are if you are combining k classifiers and if these classifiers are independent in terms of their errors, then your mean square error is going to be 1 by k into V, because this term is going to disappear ok. Now if your classifiers are perfectly correlated, then what would happen? And basically C is equal to V right, is that fine. So now what would happen? What is the net result if I substitute this as V? Going to be V right?

So, if you are all your classifiers are perfectly correlated, that is the other case we had tried taken. And all of them are making errors on the same test instances and the same errors, right? Then you will not get any benefit of doing bagging, but if you look at the other extreme, where all your errors are independent or all your classifiers are making independent errors, then you will get a benefit your expected mean square error would go down from V to 1 by k into V; everyone gets that?

(Refer Slide Time: 07:10)

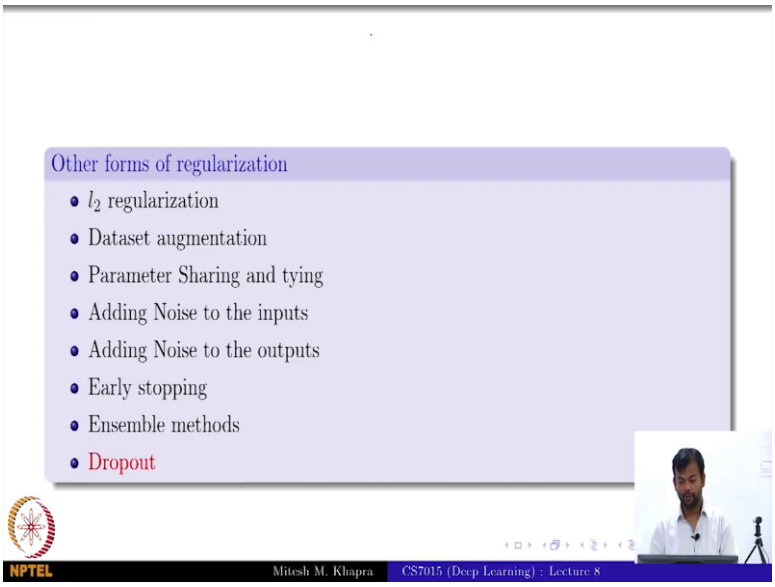


So, this was just to develop an intuition that taking an ensemble helps right? And using this intuition now we are going to see at how to do this ensemble in the case of deep neural networks.

Deep Learning
Prof. Mithesh M. Khapra
Department of Computer Science and Engineering
Indian Institute of Technology, Madras

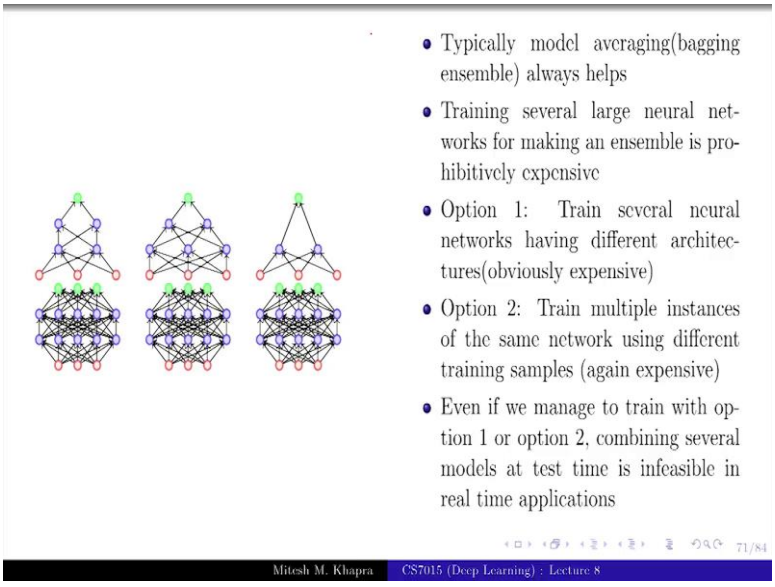
Module – 8.11
Lecture – 08
Dropout

(Refer Slide Time: 00:15)



So, in this module we will look at dropout now.

(Refer Slide Time: 00:17)



So, the intuition that we have developed in the previous module which was about ensemble methods, is what that is that ensemble makes sense in most cases. Because, you do not expect the errors of these k models that, you are using to be perfectly correlated. And we saw that, whenever they are not perfectly correlated you are going to get some advantage.

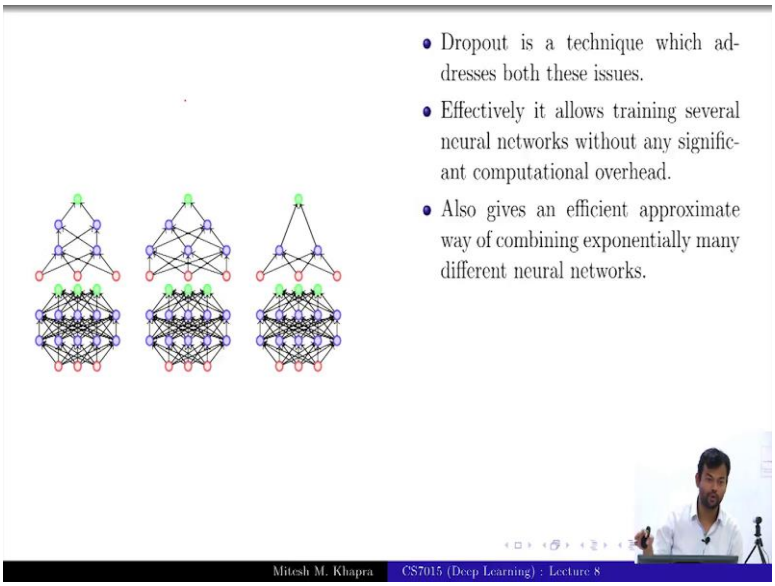
Now, how do you do this in the context of neural networks? So remember, what was bagging multiple instances of the same network trained on different subsets of the data? What is the problem with this in the context of neural networks? Each of these neural networks is very complex training, each of these is going to take time and I going to train k of them is that, fine right?

So you decide ok. Sorry, so, one option that you have is you train several different neural networks having different architectures right, but this is going to be expensive because, you have to train k of them. The other option that you have is, you train the same network, but on different subsets of the data this is also going to be expensive.

So, whatever ensambling sampling techniques you can think, if in the think of in the context of neural networks which are essentially these two techniques different architectures and take an ensemble or train the same architecture on different subsets of the data both of them are going to be expensive right?

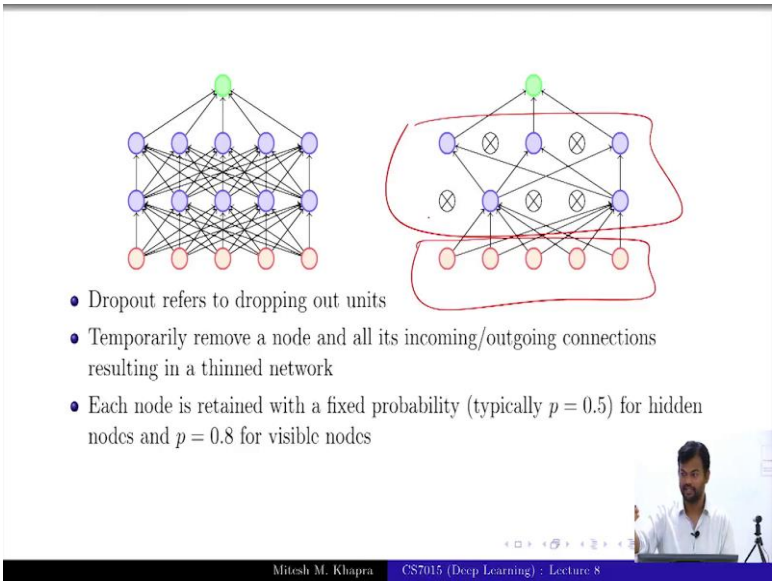
So now how do you go about it? And it is not just training time, expensive it even if we manage to train it at test time again. When you are given a test instance you have to pass it through all of these complex neural networks, each of which is going to take some computation, and then take the ensemble of the outputs right? So, even at test time it is expensive it is not just that, that training time it is expense.

(Refer Slide Time: 01:57)



So now, dropout is a technique which addresses both these issues, which issues? Train time computation as well as test time computation. So, it effectively allows training several neural network architectures without any significant computational overhead. So, we will see how that works and it just not training time as I said it also allows us to do this quickly at test time.

(Refer Slide Time: 02:21)



So again let us see, so again here ok. I will get to it when I know. So, drop out actually refers to dropping out units from the neural network.

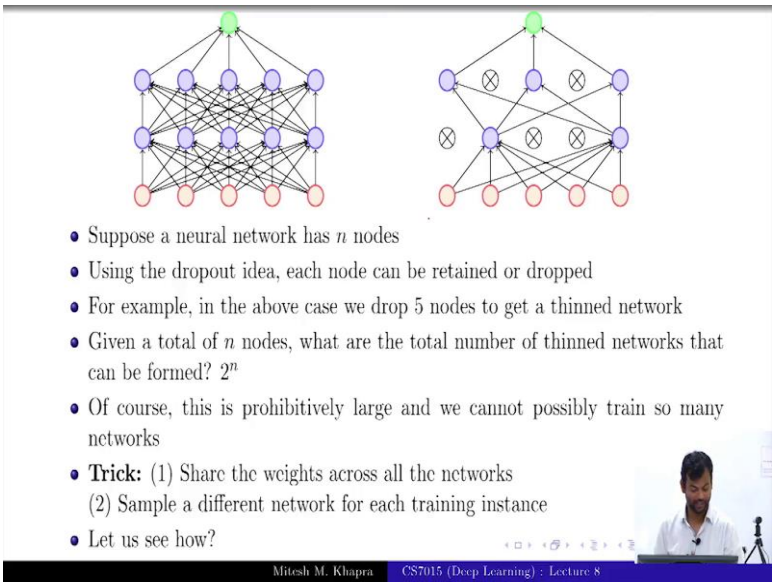
So, this is my original neural network and I am just talking about one neural network, forget about ensembles just one neural network is what I have. Now what dropout says this. You dropout some units from this neural network; that means, dropout some neurons and when I dropout some neurons, I am also going to drop out the incoming and the outgoing edges. Otherwise, where are they headed right, so I am just dropping out.

So basically, what is effectively happening here? I am getting a new network architecture right? At least that is clear that is what dropout effectively does, but I have already made a case that I do not want so many architectures. That because, it is a headache to train all of them and again a test time I have to pass it through all of them right.

So, I need to still fill that gap, but drop out says that, drop some units and you will get a new architecture. But how does that simplify life, we will see that. And now each node is actually retained with a fixed probability for the hidden nodes and even further input nodes.

So, then we were not wrong in actually dropping out the visible node. Because, you can do dropout at the visible nodes also ok. Anyways yeah so for, the hidden units you would drop them with a probability 50 percent and the input units you will drop them with a probability of 20 percent typically it again is some hyper parameter that you will have to tune, but typically this is what you will do and I hope you see that dropping nodes from the hidden unit. From the input unit is same as corrupting the input data right it is same as adding noise to the input data is that fine.

(Refer Slide Time: 03:58)

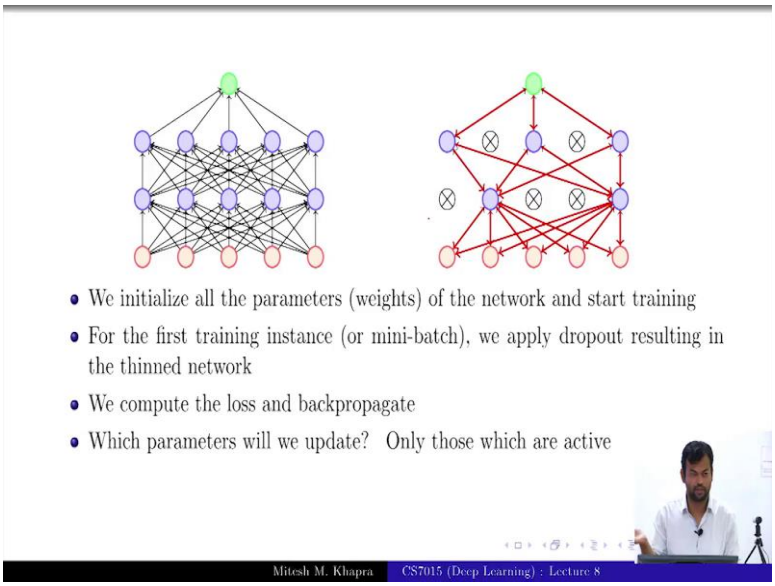


So this is the idea, now let us see how to actually implement this idea. okSo, suppose a neural network has n nodes using the dropout idea each node can be retained or dropped, an example in the above case I have dropped some 5 nodes to get a thinned network.

So, if there are n nodes what are the total number of thin networks that I can get from it? And so; that means, I can get 2 raise to n different neural networks. Am I happy about this, or sad about this? Sad, there is just too many neural networks. How can I train them actually right?

So how do I do this? I am just creating a lot of suspense without giving you the answer ok. So, first trick is, share the weights across all these networks ok. We will see what; that means, and the second trick is sample a different network for each training instance ok. None of which is clear at this point, I can see I can read your faces I am good at it ok. So, let us see how to do that.

(Refer Slide Time: 04:50)

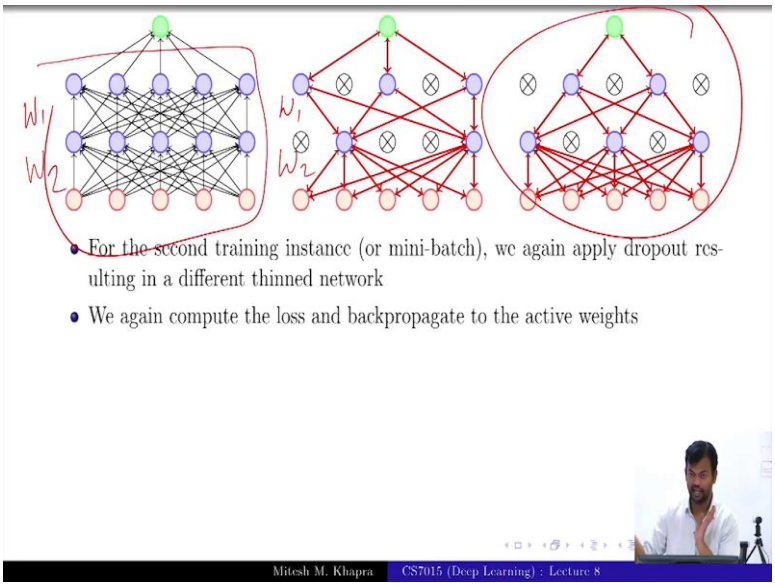


So, we initialize all the parameters of the network randomly or whatever may be used and start training. When I start training, I will pick up the first training instance or the mini batch or whatever I am doing we apply dropout resulting in this network.

What will I do and they forward prop, forward propagation right ok. Now, ok we compute the loss and back propagate how? Some weights are missing right how do I do back propagation now. I have deliberately dropped up some of these connections, they did not participate in the forward propagation. This back propagate, which are the parameters which will update now? Only the ones which actually participated right?

So, I will just do back propagation. Just look at the red arrows, I will just do it over the paths which are actually present in my network fair enough right. That is what you meant by normally ok, that is normal ok. So, I will just do it over the weights which actually participated that is fair enough that is the only thing you could; obviously do.

(Refer Slide Time: 05:51)



Now, I take the second instance, again I apply dropout and quite naturally I will get a different thinned network as you see the figure 3 in this slide ok. What would I will do now?

Student: Forward propagation.

Forward propagation, then compute the loss back propagate to compute the loss ok, and then.

Student: Back propagate.

Back propagate again back propagate only to the.

Student: Active nodes.

Active nodes, so these other nodes which will get activated. So, what is happening here, is now trying to relate it to what we were doing in bagging right; where we are trying to train these different networks on different subsets of the training data right? Do you see something similar happening here? There are many such thin networks; each time I am sampling a different network and updating it right?

So, it is equivalent to training these large number of networks on different subsets of the data right, but then the problem is that some of these networks may never even get

sampled. There are 2 raised to n of those, my amount of data is definitely to be less than 2 raised to n.

So, some of these networks might just not even get sampled. Then, what is happening? Or they would get sampled very rarely right? For example, what is the probability that again? I will end up with the same network we are computing it. Good, it is very less ok, I am fine with that at 730 right?

So, it is a very less right. So, it is quite likely that this network will never be sampled again; that means, for that network the parameters are getting updated very few times, am I fine with it. Yes, I am. Why? Because the same weights will get updated for a different network; I am just using the same weight matrix throughout remember that, my W matrix or W 1, W 2 is the same throughout.

It is just that at different depth subsets different instances; I am just touching some portions of this W 1 and I am not touching the other portions of W 1. So now, what would happen, so I have shown you 2 training instances right? What would happen to the weights which were active for the first training instance as well as, the second training instance? It will get updated twice and which are active only once?

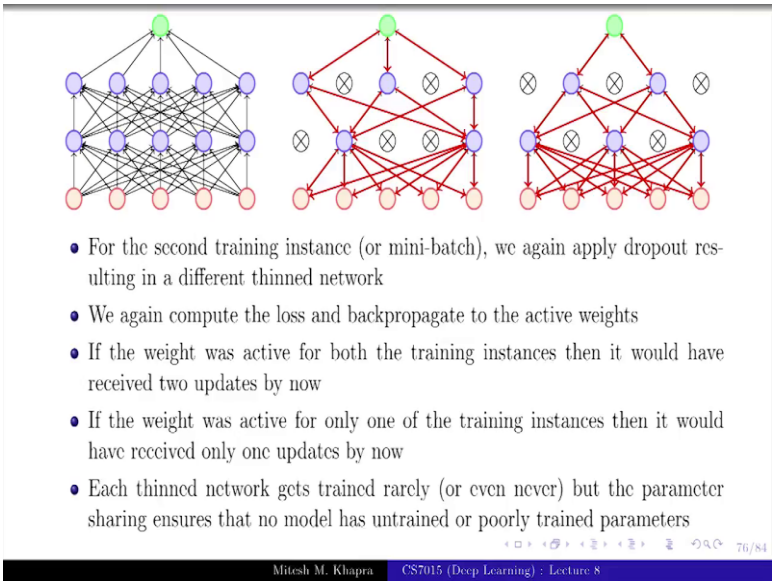
Student: (Refer Time: 07:59).

Only once right. So, over a period of time many of these weights are shared across all these networks that, I am sampling right? So, even though a particular network is sampled only a few times, its weights will get updated many times via these other networks which are similar to it. Do you get that? How many of you get this? Ok; good, so what is happening, I will just repeat that I have just one weight matrix. I am sampling a thinned out network which only uses some of these weights.

So for that training instance, I will update those weights. Now, I know that the likelihood of the same network getting sampled again is very less, but I do not care about it because, I could sample a different network, but I am sure that some of these weights will again repeat in that right? And in that, I told they will get updated. So, even though each of these networks is seemingly getting very few updates. Overall, all the weights shared by these networks are getting updated as much as they should be is that fine?

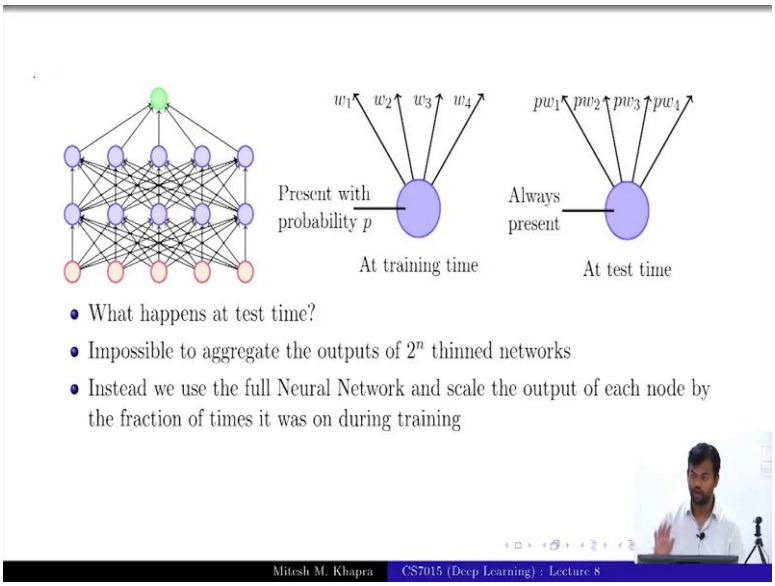
Everyone gets this idea? Ok, fine and while I am also taking care that, similar things like early stopping or weight regularization L2 regularization where, I am not allowing a single weight to continuously grow or something. Otherwise because, these weights will be off for some networks. Is that fine? You see the connection between early stopping, l 2 regularization and this is that ok?

(Refer Slide Time: 09:18)



And so, each thinned network gets trained rarely or sometimes even never, but I am not worried about it. Because, it is weights will get updated through some of these other thin networks.

(Refer Slide Time: 09:27)



This is all finite training time. At training time what is happening is this is one of these blue guys introduce on with the probability p; that means, the weights going out of it, who are available with a probability p right and other times they were not available.

Now, what do I do, it test time? I cannot let me finish this ok. I cannot take an ensemble of d ok, the answer would have been that, at test time instantiate all these 2 raised to n networks pass the training passed the test example, through all of them and then take an ensemble right, but of course, that is probablitivly expensive. So, what will I do at test time? What is the simple trick that I will do? So, he says that just use this network.

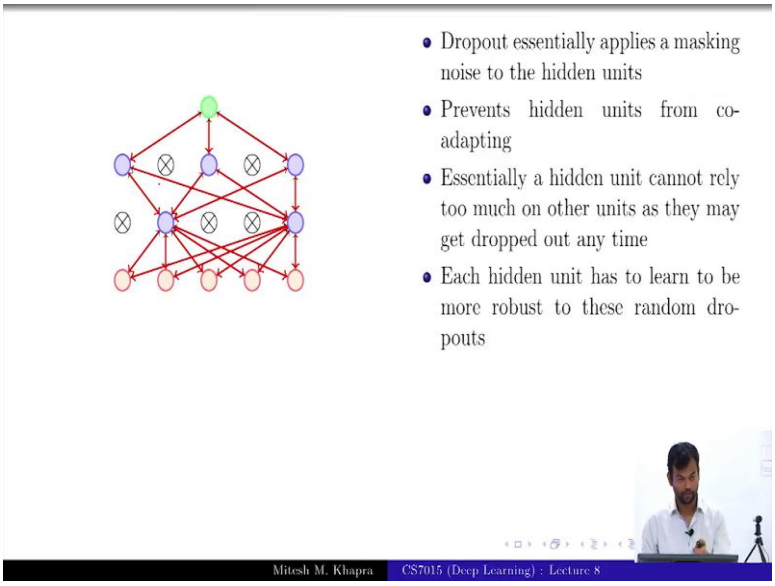
And just use the final net matrix, that you had no, but then you have guessing out of the 2 raised in the sample, some small number of those and do it. Actually, dropout uses something very simple than this. What it says is, that each of my nodes was present only p fraction of the times in the training data ok; that means, one way of looking at it is that. So, imagine that you could think of this as the analogy is that all these nodes are participating in a discussion right where they trying to see how to do this job properly, but with probability p they all sleep off, right?

So, at the end of the meeting, you will trust each of them only with probability p. So, that is the simple trick with dropout uses; it says that just scale their weights by p. Because, that is how much I trust this node; it only participated in p faction of the decisions. So, that is the confidence that I have in it.

So, if it is saying that with W1 weight, do this I will only do it with p into W 1 weight does that make sense ok. And there is again a squared egg with vacuum kind of explanation for this ok. Which was there in the quiz, last year which is very convoluted, it does not really give you the true picture because, you can derive some math and so, that this is mathematically proper, but that again works in very specific conditions, but at least if you get the intuition that is fine that what we are saying is that. These nodes will leave an active a few number of times. So, I will only trust them that much and I will just scale their weights by that factor.

So, at test time I will just pass my test instance through one network, which is the full network with the weights scaled according to the rule which, I just said that is exactly what dropout does.

(Refer Slide Time: 12:03)



So, what dropout actually does is, we will apply some kind of masking noise to the hidden units, right? Since the same as seeing that you are computing the hidden unit, but then you are masking it off ok.

So, what is the effect of this? I will give you the answer and I like; I like you to think about it. The answer is that it prevents the neurons from becoming lazy. What do lazy people do? They depend on others yeah actually yeah; they depend on others now. So, let me answer that give the answer for this and then tell me whether that is still contradict ok.

So, let us see right consider this layer of neurons. All of these are collectively responsible for what happens to this guy right; Now you see, what I mean by neurons becoming lazy? I could just see ok. I will not give my input these other neurons will take care of it, they will adjust their weights.

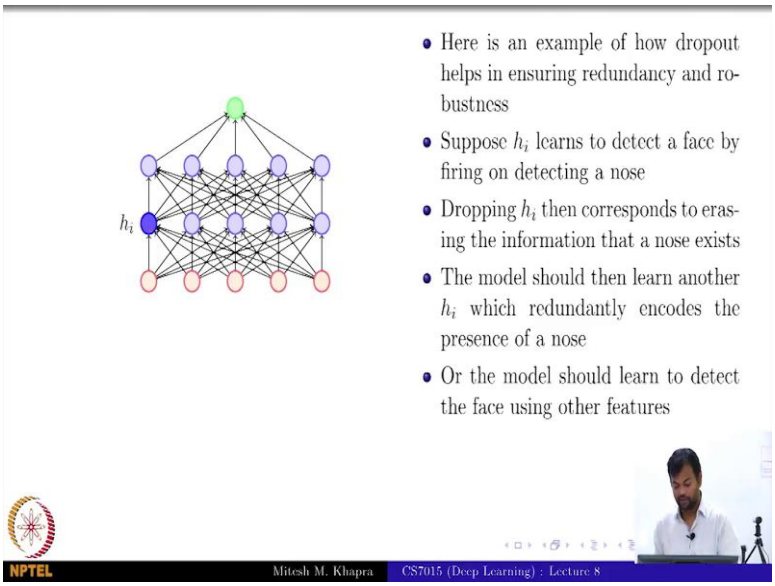
So, that they eventually, it will fire or not fire or whatever right you see that could happen, but now these neurons cannot rely on their neighbors. Because they do not know when their neighbors are going to ditch them right? They will suddenly drop off ok. And now I was waiting for my neighbor to actually do something and he is not going to do it. So, I have to be alert always do you get the analogy.

So, these guys are collectively responsible for something and they know that some people in the collection are going to betray them. So, each of them has to be more careful; so, the more technical term for this is that does not allow the neurons to co adapted.

So, it does not allow them to get into this mutual agreement that you take care of certain things, I will take care of certain things and together we will do the job right. You do question 1, I will do question 2, I am ok, it does not allow them to do this.

So, let us just concretize that intuition a bit for. So, essentially a hidden unit cannot rely too much on other units as they may get dropped out at any time. Each hidden neuron has to learn to be more robust right. It has to do the job as if it is the only guy responsible for the job ok and let us consider one of these neurons h i.

(Refer Slide Time: 14:00)

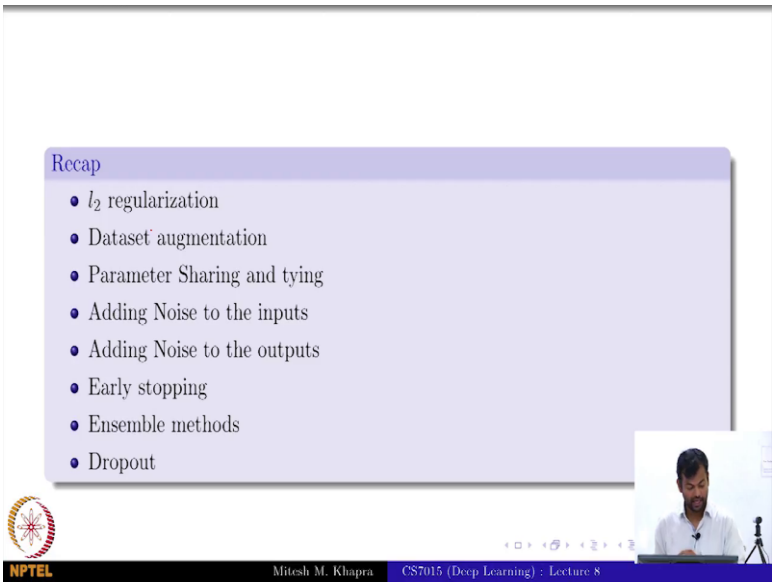


And let us see that, a h i learns to detect faces; sorry, it learns to detect a nose. So, I am trying to do face detection whether, an image is about a face or not and h i is the feature which fires. If there is a face somewhere, if there is a nose somewhere in the image is that fine.

Now, if all these guys start acting lazily ok. This guy is going to detect a nose; that means, definitely face will be there. So, I do not need to do anything right, what would happen now? Suddenly this guy is going to go away dropped out; so then, these other guys need to do one of 2 things; either add redundancy; that means, one of them should also take responsibility for detecting a nose or do it in a different way. Take responsibility for detecting the lips or the eyes or some other part do you get that? Right because, you know that I cannot co adopted with my other neurons; I cannot say that ok, in these front facing faces you just detect the nose and will be done and we will all keep quiet right?

I do not know whether, you will do your job properly. So, I will have to add more redundancy. You detect a nose, I will also detect a nose or you detect a nose and I will detect something else which helps detecting the feature right. So, that is why these networks become more and more robust as you add this dropouts?

(Refer Slide Time: 15:26)



So, that is all that I had to say I still do not know whether I have answered your question or not. All of them try to detect nose; see as long as that helps reducing the final loss it is fine. It is just the case that you would have some training images, where the nose is not visible maybe that person is drinking something right?

So, for at least for those training instances someone else has to take care that you detect from the other images right. Otherwise, a loss would not be 0 for that training instance. So, as long as you have some training instances see, if all your training instances can be detected just by detecting the nose. Then, there is nothing wrong in all of them trying to detect the nose. So, if the training it is like that it will happen, but the hope is the training data is not like that right; is that fine? So we will end here.